

SEMINAR HASIL

SISTEM PELAPORAN DAN MANAJEMEN KASUS KEKERASAN TERHADAP PEREMPUAN DAN ANAK BERBASIS *WEB* MENGGUNAKAN ARSITEKTUR *MICROSERVICES* (STUDI KASUS: DPPPA KOTA KENDARI)

**Diajukan Untuk Memenuhi
Salah Satu Syarat Memperoleh Gelar Sarjana Teknik**



MUH. AFDAL ZIQRI RAMADHAN

E1E122065

**JURUSAN INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HALU OLEO**

2026

LEMBAR PENGESAHAN

SEMINAR HASIL

SISTEM PELAPORAN DAN MANAJEMEN KASUS KEKERASAN TERHADAP PEREMPUAN DAN ANAK BERBASIS *WEB* MENGUNAKAN ARSITEKTUR *MICROSERVICES* (STUDI KASUS: DPPA KOTA KENDARI)

Adalah benar dibuat oleh saya sendiri dan belum pernah dibuat atau diserahkan sebelumnya baik sebagian ataupun seluruhnya, baik oleh saya maupun orang lain, baik di Universitas Halu Oleo ataupun institusi pendidikan lainnya.

Kendari, September 2026

Muh. Afdal Ziqri Ramadhan

E1E122065

Pembimbing I

Pembimbing II

Bambang Pramono, S.Si., M.T

NIP.197104252008011010

Isnawaty, S.Si., MT.

NIP: 197611172008122001

Mengetahui,
Ketua Jurusan Informatika,
Fakultas Teknik, Universitas Halu Oleo.

Rizal Adi Saputra, S.T., M.Kom..

NIP: 199104062019031021

INTISARI

MUH. AFDAL ZIQRI RAMADHAN, E1E122065

SISTEM PELAPORAN DAN MANAJEMEN KASUS KEKERASAN TERHADAP PEREMPUAN DAN ANAK BERBASIS *WEB* MENGGUNAKAN ARSITEKTUR *MICROSERVICES* (STUDI KASUS: DPKPA KOTA KENDARI)

Proposal, Fakultas Teknik, 2026

Kata Kunci: Kekerasan Terhadap Perempuan dan Anak, Manajemen Kasus, *Microservices*, *Rational Unified Process*, *Website*.

Proses pelaporan dan penanganan kasus kekerasan terhadap perempuan dan anak pada Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPKPA) Kota Kendari masih menghadapi pencatatan pada media yang terpisah, penginputan ulang, dan keterbatasan pemantauan status oleh pelapor. Penelitian ini bertujuan membangun sistem pelaporan dan manajemen kasus berbasis web serta menerapkan arsitektur *microservices* untuk memisahkan pengelolaan laporan dan kasus.

Sistem dikembangkan menggunakan *Rational Unified Process* (RUP) dan terdiri atas *Report service* serta *Case service*. Komunikasi perubahan status antarlayanan menggunakan *RabbitMQ*, sedangkan layanan dijalankan dalam container *Docker* dan diakses melalui API gateway *Nginx*. Evaluasi dilakukan melalui load testing pada 100, 500, dan 1.000 pengguna virtual, *fault injection* dengan menghentikan *Report service*, serta *User Acceptance Testing* (UAT).

Hasil load testing menunjukkan bahwa *microservices* memiliki rata-rata waktu respons lebih rendah pada Login, Submit Laporan, dan Master Kekerasan, sedangkan monolitik lebih rendah pada Get Penanganan. *Microservices* menghasilkan *error rate* 0% hingga 1.000 pengguna virtual, sementara monolitik mencatat *error rate* 0,10% pada Submit Laporan pada beban 1.000 pengguna virtual. Pada *fault injection*, dua pesan tertampung ketika *Report service* dihentikan dan antrian kembali kosong setelah layanan diaktifkan. Indeks UAT pelapor sebesar 80,87% dan indeks UAT admin sebesar 88,21%. Hasil tersebut berlaku pada konfigurasi dan skenario pengujian yang digunakan.

ABSTRACT

MUH. AFDAL ZIQRI RAMADHAN, E1E122065

WEB-BASED REPORTING AND CASE MANAGEMENT SYSTEM FOR VIOLENCE AGAINST WOMEN AND CHILDREN USING MICROSERVICES ARCHITECTURE (CASE STUDY: DPPPA KENDARI CITY)

Proposal, Faculty of Engineering, 2026

Keywords: *Case Management, Microservices, Rational Unified Process , Violence Against Women and Children, Website.*

The reporting and management of violence against women and children cases at the Kendari City Office of Women's Empowerment and Child Protection (DPPPA) still involve records kept in separate media, repeated data entry, and limited status tracking for reporters. This study develops a web-based reporting and case management system and applies a microservices architecture to separate report and case management functions.

The system was developed using the Rational Unified Process (RUP) and consists of a Report service and a Case service. RabbitMQ carries status-change messages between services, while the services run in Docker containers behind an Nginx API gateway. Evaluation used load testing with 100, 500, and 1,000 virtual users, fault injection by stopping the Report service, and User Acceptance Testing (UAT).

Load-testing results show that microservices produced lower average response times for Login, Submit Report, and Violence Master, whereas the monolithic system was faster for Get Case Handling. Microservices recorded a 0% error rate up to 1,000 virtual users, while the monolithic system recorded a 0.10% error rate for Submit Report at 1,000 virtual users. In the fault-injection test, two messages remained queued while the Report service was stopped, and the queue became empty after recovery. The reporter UAT index was 80,87%, and the admin UAT index was 88,21%. These findings apply to the tested configuration and scenarios.

DAFTAR ISI

LEMBAR PENGESAHAN	i
INTISARI.....	ii
ABSTRACT.....	iii
DAFTAR ISI.....	iv
DAFTAR GAMBAR	vii
DAFTAR TABEL.....	ix
DAFTAR LAMPIRAN.....	x
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Batasan Masalah.....	4
1.4 Tujuan Penelitian	5
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan.....	6
1.7 Tinjauan Pustaka	7
BAB II LANDASAN TEORI	12
2.1 Kekerasan terhadap Perempuan dan Anak.....	12
2.1.1 Definisi Kekerasan terhadap Perempuan dan Anak	12
2.1.2 Dampak Kekerasan terhadap Perempuan dan Anak.....	13
2.2 Sistem Informasi dan Manajemen Kasus	13
2.2.1 Pengertian Sistem Informasi.....	14
2.2.2 Sistem Pelaporan Kasus.....	15
2.2.3 Manajemen Kasus Kekerasan.....	16
2.3 Arsitektur <i>Microservices</i>	17
2.4 <i>API Gateway</i>	18
2.5 REST	18
2.6 <i>Containerization</i>	20
2.6.1 <i>Container</i>	20
2.6.2 Docker.....	21
2.7 Node JS	22
2.8 JavaScript	22
2.9 MongoDB.....	23
2.10 ExpressJS	24
2.11 React JS	24
2.12 RabbitMQ.....	25

2.13	<i>Unified Modeling Language (UML)</i>	26
2.12.1	<i>Use case Diagram</i>	26
2.12.2	<i>Activity Diagram</i>	27
2.12.3	<i>Class Diagram</i>	28
2.12.4	<i>Sequence Diagram</i>	29
2.12.5	<i>Deployment Diagram</i>	30
2.14	<i>Rational Unified Process (RUP)</i>	30
2.15	<i>Load testing</i>	32
2.16	<i>Fault Injection</i>	34
2.17	<i>Apache JMeter</i>	35
2.18	<i>User Acceptance Testing (UAT)</i>	35
2.19	<i>Skala Likert</i>	36
BAB III	METODOLOGI PENELITIAN	37
3.1.	Metode Pengumpulan Data	37
3.2.	Metode Pengembangan Sistem	37
3.3.	Waktu dan Tempat Penelitian	38
3.3.1.	Waktu Penelitian.....	38
3.3.2.	Tempat Penelitian	39
BAB IV	ANALISIS DAN PERANCANGAN	40
4.1.	Analisis Sistem yang Sedang Berjalan.....	40
4.2.	Analisis Kebutuhan Sistem	42
4.2.1.	Kebutuhan Fungsional	42
4.2.2.	Kebutuhan Non Fungsional	44
4.3.	Analisis Perancangan Sistem	45
4.3.1	<i>Flowchart</i>	45
4.3.2	Analisis Perancangan <i>Unified Modeling Language (UML)</i>	46
4.3.3	Perancangan Antarmuka (<i>User Interface</i>)	67
4.4.	Pengujian <i>Load Testing</i>	76
4.5.	Pengujian <i>Fault Injection</i>	78
4.6.	Pengujian <i>User Acceptance Testing</i>	79
4.6.1	Instrumen <i>User Acceptance Testing</i> pelapor	80
4.6.2	Instrumen <i>User Acceptance Testing</i> admin	82
BAB V	IMPLEMENTASI DAN PENGUJIAN SISTEM	84
5.1.	Implementasi Sistem	84
5.2.	Implementasi <i>Interface</i>	84

5.2.1 Halaman Login	84
5.2.2 Antarmuka Pelapor	85
5.2.3 Halaman Dashboard Petugas	86
5.2.4 Halaman Penanganan Kasus	87
5.2.5 Halaman Arsip Laporan	88
5.2.6 Halaman <i>dashboard</i> Super Admin	89
5.2.7 Halaman Manajemen Petugas	89
5.2.8 Halaman Pusat Kendali Layanan	90
5.2.9 Halaman Data Pelaporan	91
5.3. Implementasi <i>Message Broker</i>	92
5.4. Implementasi Docker	94
5.5. Pengujian <i>Load Testing</i>	99
5.6. Pengujian <i>Fault Injection</i>	106
5.7. Pengujian <i>User Acceptance Testing</i>	108
5.7.1 Hasil Pengujian UAT Pelapor	108
5.7.2 Hasil Pengujian UAT Admin	110
BAB VI PENUTUP	113
6.1. Kesimpulan	113
6.2. Saran	114
DAFTAR PUSTAKA	115
LAMPIRAN	122

DAFTAR GAMBAR

Gambar 2. 1 Hubungan klien, API Gateway dan Services	17
Gambar 4. 1 Flowchart Sistem yang Sedang Berjalan.....	41
Gambar 4. 2 Flowchart Umum.....	45
Gambar 4. 3 Use Case Diagram.....	47
Gambar 4. 4 Class Diagram	49
Gambar 4. 5 Activity Login	50
Gambar 4. 6 Activity Buat Laporan.....	51
Gambar 4. 7 Activity Registrasi Laporan	52
Gambar 4. 8 Activity Input Asessment.....	53
Gambar 4. 9 Activity Renca Intervensi.....	54
Gambar 4. 10 Activity Kelola Kasus	55
Gambar 4. 11 Activity Selesaikan Kasus	56
Gambar 4. 12 Activity Menu Manajemen Petugas	57
Gambar 4. 13 Activity Kelola Metode Penanganan.....	58
Gambar 4. 14 Activity Kelola Jenis Kekerasan	59
Gambar 4. 15 Activity Export Data Laporan	60
Gambar 4. 16 Sequence Login	61
Gambar 4. 17 Sequence Input dan Cek Lapora.....	62
Gambar 4. 18 <i>Sequence Diagram</i> Registrasi Laporan	63
Gambar 4. 19 <i>Sequence</i> Selesaikan Kasus	64
Gambar 4. 20 <i>Sequence</i> Manajemen Petugas	65
Gambar 4. 21 <i>Sequence</i> Pusat Kendali Layanan.....	66
Gambar 4. 22 <i>Deployment Diagram</i> arsitektur <i>microservices</i>	67
Gambar 4. 23 Halaman Pelaporan	68
Gambar 4. 24 Halaman Cek Status Laporan.....	69
Gambar 4. 25 Halaman <i>Dashboard Admin</i> Petugas.....	70
Gambar 4. 26 Halaman Penanganan Kasus	71
Gambar 4. 29 Halaman Arsip Kasus.....	72
Gambar 4. 30 Halaman Dashboard untuk Super admin.....	73
Gambar 4. 31 Halaman Manajemen Petugas	74
Gambar 4. 32 Halaman Pusat Kendali Layanan	75
Gambar 4. 33 Halaman Data Pelaporan	76
Gambar 5. 1 Halaman Login.....	85
Gambar 5. 2 Halaman pelaporan.....	85
Gambar 5. 3 Halaman pelaporan.....	86
Gambar 5. 4 Halaman Dashboard Petugas.....	86
Gambar 5. 5 Halaman Penanganan Kasus	87
Gambar 5. 6 Halaman Detail Laporan	88
Gambar 5. 7 Halaman Arsip Laporan	89
Gambar 5. 8 Halaman Dashboard Admin	89
Gambar 5. 9 Halaman Manajemen Petugas	90
Gambar 5. 10 Tab Kategori Kekerasan.....	90
Gambar 5. 11 Tab Metode Penanganan	91
Gambar 5. 12 Halaman Data Pelaporan.....	91

Gambar 5. 13 Antrean Pesan ketika Report Service Dihentikan	107
Gambar 5. 14 Antrean Pesan ketika Report Service Diaktifkan Kembali	107

DAFTAR TABEL

Tabel 2.1 Simbol Diagram Use case	27
Tabel 2.2 Notasi Activity Diagram	28
Tabel 2.3 Simbol Class Diagram	29
Tabel 2.4 Simbol Sequence Diagram	29
Tabel 2.5 Simbol Deployment Diagram	30
Tabel 2.6 Tabel Skala Likert	36
Tabel 3.1 Waktu Penelitian	39
Tabel 4.1 Kebutuhan Fungsional Sistem	42
Tabel 4.2 Kebutuhan Perangkat Keras	44
Tabel 4.3 Kebutuhan Perangkat Lunak	44
Tabel 4.4 Skenario Pengujian <i>Load Testing</i>	77
Tabel 4.5 Rancangan Pengujian <i>Fault Injection</i>	79
Tabel 4.6 Instrumen <i>User Acceptance Testing</i> pelapor	80
Tabel 4.7 Instrumen <i>User Acceptance Testing</i> admin	82
Tabel 5.1 Titik publikasi pesan perubahan status kasus	92
Tabel 5.2 Kode program pengiriman pesan pada <i>Case service</i>	92
Tabel 5.3 Kode program pengiriman pesan pada <i>Case service</i>	93
Tabel 5.4 Konfigurasi Arsitektur Monolitik	95
Tabel 5.5 Konfigurasi <i>Report service</i>	96
Tabel 5.6 Konfigurasi <i>Case service</i>	97
Tabel 5.7 Konfigurasi Database	98
Tabel 5.8 Konfigurasi <i>Message Broker</i>	98
Tabel 5.9 Data dasar perhitungan pada 100 pengguna virtual	100
Tabel 5.10 Perbandingan hasil pengujian pada 100 pengguna virtual	101
Tabel 5.11 Data dasar perhitungan pada 500 pengguna virtual	102
Tabel 5.12 Perbandingan hasil pengujian pada 500 pengguna virtual	103
Tabel 5.13 Data dasar perhitungan pada 1.000 pengguna virtual	104
Tabel 5.14 Perbandingan hasil pengujian pada 1.000 pengguna virtual	104
Tabel 5.15 Hasil Pengamatan Antrean pada Pengujian <i>Fault injection</i>	106
Tabel 5.16 Hasil Pengujian <i>User Acceptance Testing</i> (UAT) Pelapor	108
Tabel 5.17 Hasil Pengujian <i>User Acceptance Testing</i> (UAT) Admin	110

DAFTAR LAMPIRAN

Lampiran 1 Form <i>User Acceptance Testing</i>	122
Lampiran 2 Hasil Penilaian UAT Responden.....	128
Lampiran 3 Wawancara Dengan Petugas DPPPA.....	129
Lampiran 4 Dokumentasi Responden UAT.....	130
Lampiran 5 Bimbingan Dosen Pembimbing 2.....	131

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kekerasan terhadap perempuan dan anak masih menjadi salah satu isu sosial yang memerlukan perhatian serius di Indonesia. Berdasarkan data Sistem Informasi Online Perlindungan Perempuan dan Anak (Simfoni PPA) milik Kementerian Pemberdayaan Perempuan dan Perlindungan Anak (KemenPPPA), sepanjang tahun 2025 tercatat lebih dari 25.658 kasus kekerasan terhadap perempuan dan anak yang dilaporkan di seluruh Indonesia. Namun, angka tersebut tidak sepenuhnya mencerminkan realitas yang ada, mengingat banyak korban yang memilih untuk tidak melapor karena ketakutan, tekanan dari lingkungan sosial, atau kurangnya akses terhadap layanan pelaporan yang mudah diakses (Putri, Fahmi, dan Ardiansah 2025).

Pada tingkat daerah, Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA) Kota Kendari memiliki peran penting dalam menangani kasus kekerasan melalui mekanisme pelaporan, pendampingan korban, dan pelaporan kasus ke tingkat provinsi maupun pusat. Berdasarkan hasil wawancara dengan pihak DPPPA Kota Kendari (2025), permasalahan utama yang sering dihadapi meliputi keterlambatan laporan dari kelurahan yang menyebabkan proses tindak lanjut menjadi lambat, serta minimnya kesadaran masyarakat dalam melapor akibat rasa takut dan stigma sosial. Permasalahan ini juga diperburuk oleh sistem pelaporan yang masih bersifat manual dan belum terintegrasi antarbidang, sehingga data sering kali tidak sinkron dan sulit untuk dipantau secara cepat.

Kondisi serupa juga terjadi di berbagai daerah di Indonesia, di mana sistem pelayanan pengaduan kekerasan terhadap perempuan dan anak masih menghadapi kendala akses dan informasi. Banyak korban belum memahami hak mereka untuk melapor, tidak mengetahui mekanisme pengaduan yang tersedia, dan terkendala oleh lokasi tempat tinggal yang sulit dijangkau oleh penyedia layanan (Adi, Terttiavini, dan Marcelina, 2023). Selain itu, sistem yang belum terintegrasi menyebabkan penanganan kasus menjadi lambat dan koordinasi antarinstansi tidak berjalan efektif. Permasalahan sosial ini menegaskan perlunya dukungan sistem

informasi berbasis teknologi untuk membantu proses pelaporan, validasi, dan tindak lanjut kasus secara lebih terstruktur dan cepat. Hambatan lain yang sering muncul dalam proses penanganan kasus kekerasan adalah kurangnya integrasi antarinstansi dan lemahnya manajemen kasus, di mana pencatatan dan pelaporan masih dilakukan secara terpisah antarunit. Kondisi ini menghambat komunikasi lintas lembaga dan memperlambat penyelesaian kasus. Koordinasi yang belum optimal menyebabkan tumpang tindih data, serta mengurangi efektivitas pelayanan publik dalam memberikan perlindungan bagi korban (Fitriana dan Kriswibowo, 2022).

Seiring dengan perkembangan teknologi informasi, pemanfaatan *website* sebagai sarana digitalisasi pelayanan publik menjadi langkah strategis untuk meningkatkan efisiensi dan transparansi layanan pemerintah. *Website* mampu menyediakan informasi secara terbuka, mempercepat proses pelayanan, serta mempermudah interaksi antara masyarakat dan instansi pemerintah. Melalui media berbasis *web*, masyarakat dapat melakukan pelaporan, pemantauan, maupun memperoleh informasi layanan tanpa harus datang langsung ke kantor pelayanan. Hal ini sejalan dengan pandangan bahwa penggunaan *website* dalam pelayanan publik dapat mempercepat proses *administrasi*, memperluas jangkauan layanan, dan mendukung keterbukaan informasi di lingkungan pemerintahan (Nurrahman dkk., 2021).

Dalam konteks pelayanan DPPPA, sistem berbasis web dapat membantu menyediakan saluran pelaporan yang lebih mudah diakses serta mendukung pengelolaan laporan dan perkembangan kasus secara lebih terstruktur. Sistem juga dapat menyediakan informasi laporan yang diperlukan oleh pihak terkait melalui antarmuka dan API yang sesuai dengan hak akses pengguna. Dalam pengembangannya, pemilihan arsitektur sistem memainkan peran krusial dalam menentukan keandalan dan skalabilitas aplikasi, khususnya di instansi pemerintah yang masih banyak mengadopsi arsitektur monolitik. Arsitektur monolitik mengintegrasikan seluruh komponen seperti antarmuka pengguna, logika bisnis, dan basis data dalam satu kesatuan, sehingga meskipun sederhana untuk tahap awal, ia menimbulkan kelemahan signifikan seperti kesulitan pengembangan fitur baru,

pembaruan yang rumit, serta ketidakefisienan pengelolaan sumber daya. Akibatnya, kerusakan pada satu modul berpotensi menyebabkan kegagalan total sistem, yang memperburuk risiko downtime di lingkungan pemerintahan (Iqbal, Syahputra, dan Handoko, 2022).

Sebagai alternatif, arsitektur *microservices* memberikan pendekatan yang lebih fleksibel, di mana sistem terdiri dari serangkaian layanan kecil yang independen dan saling berkomunikasi melalui *Application Programming Interface* (API). Setiap layanan dapat dikembangkan, diuji, dan diperbarui secara terpisah tanpa mempengaruhi layanan lainnya. Pendekatan ini membuat sistem lebih mudah untuk dikembangkan, lebih tahan terhadap kesalahan, dan mampu beradaptasi dengan peningkatan jumlah pengguna (Saputra dan Nabil, 2021). Dalam konteks sistem pelaporan kasus kekerasan, *microservices* memungkinkan modul pelaporan, validasi data, manajemen pengguna, dan laporan statistik dijalankan secara mandiri tetapi tetap saling terhubung melalui integrasi data yang aman.

Berdasarkan uraian tersebut, diperlukan pengembangan Sistem *Integrated Reporting and Case Management* Kekerasan Terhadap Perempuan dan Anak Berbasis *Web* Menggunakan Arsitektur *Microservices* (studi kasus: DPPPA Kota Kendari). Sistem ini bertujuan untuk menyediakan *platform* pelaporan online yang mudah diakses masyarakat, mempercepat koordinasi antarunit kerja, serta meningkatkan efektivitas dalam pengelolaan dan pelaporan kasus. Dengan dukungan arsitektur *microservices*, sistem dapat dikembangkan secara modular, lebih stabil, serta siap beradaptasi dengan kebutuhan pengguna dan perkembangan teknologi di masa mendatang.

Dengan adanya sistem ini, diharapkan DPPPA Kota Kendari dapat mempercepat proses pelaporan, memperkuat koordinasi dengan kelurahan, dan meningkatkan transparansi dalam penanganan kasus kekerasan terhadap perempuan dan anak yang sejalan dengan visi pembangunan digital pemerintah daerah yang inklusif dan berkelanjutan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini yaitu:

1. Bagaimana merancang dan membangun sistem pelaporan serta manajemen kasus kekerasan terhadap perempuan dan anak berbasis web pada DPPPA Kota Kendari?
2. Bagaimana menerapkan arsitektur *microservices* untuk memisahkan fungsi pelaporan dan manajemen kasus?
3. Bagaimana kinerja dan perilaku sistem yang dibangun berdasarkan waktu respons, *throughput*, *error rate*, mekanisme komunikasi antar layanan saat terjadi gangguan, dan penerimaan pengguna?

1.3 Batasan Masalah

Berdasarkan rumusan masalah yang ada, adapun batasan masalah dalam penulisan tugas akhir ini adalah :

1. Sistem yang dikembangkan berfokus pada pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak pada Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA) Kota Kendari, dan tidak mencakup instansi lain di luar ruang lingkup studi kasus.
2. Arsitektur *microservices* yang dikembangkan dibatasi pada dua *service* independen, yaitu *Report Service* yang menangani pelaporan dan statistik kasus, serta *Case service* yang menangani registrasi, *assessment*, dan penanganan kasus.
3. Proses pelaporan dilakukan secara *online*, dengan data yang diinput berupa identitas pelapor, informasi kejadian, dan dokumen pendukung. Sistem tidak mencakup layanan pelaporan melalui telepon atau pesan singkat (SMS).
4. Sistem hanya mendukung pengelolaan data dan alur penanganan kasus secara administratif, dan tidak mencakup aspek psikologis, medis, maupun proses hukum lanjutan secara langsung.
5. Arsitektur *microservices* dibatasi pada dua layanan independen, yaitu *report service* untuk mengelola laporan dan data pelaporan serta *case service* untuk mengelola registrasi, *assessment*, intervensi, dan penyelesaian kasus.

1.4 Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut :

1. Merancang dan membangun sistem pelaporan serta manajemen kasus kekerasan terhadap perempuan dan anak berbasis web pada DPPPA Kota Kendari.
2. Menerapkan arsitektur *microservices* untuk memisahkan fungsi pelaporan dan manajemen kasus.
3. Mengevaluasi kinerja dan perilaku sistem berdasarkan waktu respons, *throughput*, *error rate*, mekanisme komunikasi antar layanan saat terjadi gangguan, dan penerimaan pengguna.

1.5 Manfaat Penelitian

Adapun manfaat dari penelitian ini dalam beberapa bidang adalah sebagai berikut:

1. Bagi akademisi, penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan ilmu pengetahuan di bidang sistem informasi, khususnya terkait perancangan dan penerapan sistem pelaporan dan manajemen kasus berbasis *web* menggunakan arsitektur *microservices*. Hasil penelitian ini juga dapat menjadi bahan referensi bagi mahasiswa dan peneliti lain yang ingin melakukan penelitian lanjutan mengenai digitalisasi pelayanan publik, integrasi sistem antarinstansi, serta penerapan arsitektur *microservices* pada sistem informasi pemerintahan.
2. Bagi praktisi, khususnya Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA), penelitian ini diharapkan dapat membantu dalam meningkatkan efektivitas proses pelaporan dan penanganan kasus kekerasan terhadap perempuan dan anak. Sistem yang dikembangkan dapat dimanfaatkan untuk mempercepat alur pelaporan, meningkatkan koordinasi antara kelurahan dan DPPPA, serta mempermudah pemantauan status dan perkembangan kasus secara terstruktur dan *real-time* sebagai dasar pengambilan keputusan.

3. Bagi masyarakat dan lingkungan sosial, penelitian ini diharapkan dapat memberikan dampak positif dalam meningkatkan akses masyarakat terhadap layanan pelaporan kekerasan terhadap perempuan dan anak. Dengan tersedianya sistem pelaporan berbasis *web* yang mudah diakses, masyarakat diharapkan lebih terdorong untuk melaporkan kasus kekerasan, sehingga dapat mendukung upaya pencegahan, perlindungan korban, serta peningkatan kualitas pelayanan sosial di tingkat daerah.

1.6 Sistematika Penulisan

Adapun sistematika penulisan laporan proposal penelitian ini yaitu sebagai berikut :

BAB I PENDAHULUAN

Bab ini berisi tentang deskripsi umum isi laporan yang meliputi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, sistematika penulisan dan tinjauan pustaka.

BAB II LANDASAN TEORI

Bab ini menjelaskan teori-teori yang menjadi dasar dan acuan dalam penelitian ini. Seluruh teori yang digunakan diperoleh dari berbagai sumber terpercaya, baik dari media cetak maupun media elektronik, yang relevan dengan topik penelitian. Teori-teori tersebut meliputi konsep kekerasan terhadap perempuan dan anak, sistem informasi, sistem pelaporan dan manajemen kasus, *website*, arsitektur *microservices*, *Application Programming Interface*(API), basis data, serta *Unified Modeling Language* (UML) sebagai alat bantu perancangan sistem.

BAB III METODE PENELITIAN

Bab ini menjelaskan secara rinci metode yang digunakan dalam penelitian ini, yang mencakup tahapan dan prosedur pengumpulan data, metode analisis yang digunakan, serta tahapan pengembangan sistem yang diterapkan. Selain itu, bab ini juga memaparkan lokasi dan waktu penelitian, analisis kebutuhan sistem, perancangan arsitektur dan antarmuka sistem, serta skenario pengujian yang

dilakukan untuk memastikan sistem pelaporan dan manajemen kasus yang dibangun dapat berjalan dengan baik dan sesuai dengan kebutuhan pengguna.

1.7 Tinjauan Pustaka

Beberapa penelitian yang dijadikan rujukan dalam penelitian ini dapat dilihat pada 1.1.

Tabel 1. 1 Penelitian Terdahulu

No	Judul	Hasil
1	Sistem Informasi Pelayanan Pengaduan Kekerasan Terhadap Perempuan dan Anak Berbasis <i>Web</i> (Adi, Terttiavini, dan Marcelina 2023)	Penelitian ini menghasilkan sistem informasi berbasis <i>web</i> yang mempermudah masyarakat dalam melakukan pengaduan kasus kekerasan terhadap perempuan dan anak serta membantu pihak instansi dalam pengelolaan data pengaduan secara terpusat dan terstruktur. Sistem yang dibangun mampu meningkatkan efektivitas dan efisiensi layanan pengaduan dibandingkan proses manual, serta mendukung penyajian laporan berdasarkan identitas korban, pelaku, lokasi, usia, dan pendidikan korban.
2	Perancangan Sistem Informasi Laporan Kegiatan Penanaman Modal Dengan Menggunakan Arsitektur <i>Microservices</i> Pada Kementerian Investasi/Badan Koordinasi Penanaman	Penelitian ini merancang sistem informasi pelaporan kegiatan penanaman modal pada Kementerian Investasi/BKPM menggunakan arsitektur <i>microservices</i> . Sistem ini memecah proses pelaporan ke dalam layanan-layanan independen (<i>microservices</i>) yang masing-masing menangani fungsi spesifik seperti input data laporan, validasi, dan penyajian data. Dengan arsitektur <i>microservices</i> , setiap layanan dapat dikembangkan, diuji, dan di- <i>deploy</i> secara mandiri, sehingga meningkatkan skalabilitas,

Tabel 1.1 Penelitian Terdahulu (Lanjutan)

No	Judul	Hasil
	Modal (Tjahyono dkk., 2022)	modularitas, dan kemudahan pemeliharaan sistem pelaporan yang sebelumnya bersifat monolitik.
3	Sistem informasi penanganan kekerasan terhadap perempuan dan anak yang memfasilitasi pengaduan online dan pemantauan penanganan kasus di PW Aisyiyah DKI Jakarta (Annisa dan Adharani, 2023)	Penelitian ini menghasilkan sistem informasi berbasis web yang memfasilitasi pengaduan kasus kekerasan terhadap perempuan dan anak secara online. Sistem memungkinkan petugas dari berbagai divisi untuk melakukan pemantauan perkembangan penanganan kasus, berkoordinasi melalui sistem, serta mengarsipkan pencatatan laporan penanganan secara digital dan dapat diakses secara daring
4	Pengembangan Arsitektur Microservice pada Learning Management System E-learning Menggunakan Metode Web Service (Dinova dan Utomo, 2024)	kelemahan dalam mengadaptasi perubahan kebutuhan sistem dan kompleksitas kode. Dengan arsitektur microservice, setiap layanan dapat dibangun, diuji, dan di-deploy secara terpisah, sehingga meningkatkan fleksibilitas dan maintainability sistem.
5	Perancangan dan Evaluasi Arsitektur Microservices Menggunakan Microservices Migration Pattern dan Microservices	Penelitian ini merancang dan mengevaluasi migrasi sistem informasi akademik dari monolitik ke arsitektur microservices. Dengan menerapkan Microservices Migration Pattern dan evaluasi Microservices Scorecard, hasil rancangan mencapai skor kesiapan 81,7% (kategori high readiness). Penelitian ini memberikan pendekatan

Tabel 1.1 Penelitian Terdahulu (Lanjutan)

No	Judul	Hasil
	Scorecard (Ramadhan Putra dkk., 2025)	terukur dalam transisi ke microservices untuk meningkatkan skalabilitas dan pemeliharaan sistem di lingkungan perguruan tinggi.
6	Evaluating Service-Oriented and Microservice Architecture Patterns to Deploy eHealth Applications in Cloud Computing Environment (Calderón-Gómez dkk., 2021)	Penelitian ini membandingkan performa arsitektur SOA dan MSA pada platform eHealth berbasis Cloud menggunakan spike testing dengan Apache JMeter. Hasil menunjukkan bahwa MSA lebih unggul dalam performa dan waktu respons (sekitar 54,21% lebih efisien) serta mendukung skalabilitas horizontal yang lebih baik dibandingkan SOA. Namun, MSA mengonsumsi bandwidth lebih besar (sekitar 73,80%).
7	Monolithic Vs Microservice Architecture A Performance and Scalability Evaluation (Blinowski, Ojdowska, dan Przybylek, 2022)	Penelitian ini membandingkan performa dan skalabilitas arsitektur monolitik dan microservices secara empiris. Hasil menunjukkan bahwa pada mesin tunggal, arsitektur monolitik memiliki performa lebih baik, namun arsitektur microservices unggul dalam hal skalabilitas horizontal, fault tolerance, dan fleksibilitas pengembangan. Studi ini memberikan panduan bagi organisasi dalam menentukan arsitektur yang sesuai berdasarkan skala pengguna dan kebutuhan sistem. Pemilihan arsitektur disimpulkan bergantung pada kebutuhan dan karakteristik organisasi

Tabel 1.1 Penelitian Terdahulu (Lanjutan)

No	Judul	Hasil
8	Case Reporting System with Tracking and Chat Machine Learning for The Department of Women's Empowerment and Child Protection Manado (Rumondor dkk.. 2025)	Penelitian ini berhasil mengembangkan sistem pelaporan kasus berbasis web bernama LABRRAK yang mengintegrasikan chatbot berbasis Machine Learning (NLP menggunakan Google Dialogflow) dan pelacakan status laporan berbasis Finite State Machine (FSM). Hasil pengujian menunjukkan chatbot memiliki akurasi klasifikasi intent sebesar 91%, waktu respon rata-rata 0,8 detik, serta sistem mampu meningkatkan transparansi, kecepatan penanganan kasus, dan kepercayaan publik. Sistem dinilai stabil, fungsional, dan efektif untuk layanan publik yang menangani kasus kekerasan terhadap perempuan dan anak.
9	Online and Technology-Facilitated Violence Against Women (Rigotti dan McGlynn, 2025)	Penelitian ini membahas kekerasan terhadap perempuan yang terjadi melalui media digital sebagai bentuk pelanggaran hak asasi manusia. Hasil kajian menekankan pentingnya perlindungan hukum, pemantauan kebijakan, serta peran negara dan platform digital dalam mencegah dan menangani kekerasan terhadap perempuan di ruang digital.

Tabel 1.1 Penelitian Terdahulu (Lanjutan)

No	Judul	Hasil
10	Development and Implementation of Violence against Women and their Children Report System Mobile Application (Balahadia dkk., 2022)	Penelitian ini mengembangkan dan mengimplementasikan aplikasi web dan mobile untuk pelaporan kekerasan terhadap perempuan dan anak (VAWC) di Provinsi Laguna, Filipina. Aplikasi memiliki lima fitur utama: modul darurat, pelacakan lokasi, pelaporan, diseminasi informasi, dan asesmen. Hasil pengujian menunjukkan seluruh fitur berfungsi dengan baik dan aplikasi berhasil membantu masyarakat khususnya perempuan dan anak melalui pelaporan digital. (Jurnal Internasional - IJSTEM)

BAB II

LANDASAN TEORI

2.1 Kekerasan terhadap Perempuan dan Anak

Kekerasan terhadap perempuan dan anak merupakan isu global yang menjadi perhatian serius di berbagai negara, termasuk Indonesia. Permasalahan ini tidak mengenal batas usia, latar belakang pendidikan, maupun status sosial ekonomi, sehingga dapat terjadi di berbagai lapisan masyarakat. Pemahaman yang komprehensif mengenai definisi dan dampak kekerasan terhadap perempuan dan anak menjadi landasan penting dalam upaya pencegahan dan penanganan kasus secara efektif. Bagian ini akan membahas konsep dasar kekerasan terhadap perempuan dan anak serta dampak yang ditimbulkan bagi korban maupun lingkungan sosialnya.

2.1.1 Definisi Kekerasan terhadap Perempuan dan Anak

Kekerasan terhadap perempuan dan anak merupakan segala bentuk tindakan yang menimbulkan penderitaan atau kerugian, baik secara fisik, psikis, seksual, maupun sosial, yang dilakukan terhadap individu atau kelompok yang berada dalam posisi rentan. Tindakan tersebut tidak hanya berdampak pada kondisi korban secara langsung, tetapi juga menghambat pemenuhan hak-hak dasar dan kebebasan yang seharusnya dimiliki sebagai manusia. Kekerasan dalam konteks ini dipahami sebagai pelanggaran terhadap nilai kemanusiaan dan hak asasi manusia (Wijaningsih dan Farida, 2022).

Kekerasan terhadap perempuan umumnya berkaitan dengan ketimpangan relasi kuasa dan diskriminasi berbasis gender yang berkembang dalam struktur sosial. Kekerasan ini dapat terjadi di ranah privat maupun publik, termasuk dalam lingkungan keluarga, masyarakat, dan institusi. Sementara itu, kekerasan terhadap anak mencakup segala bentuk perlakuan yang mengancam keselamatan, martabat, serta mengganggu proses tumbuh dan berkembang anak secara optimal. Oleh karena itu, kekerasan terhadap perempuan dan anak tidak hanya dipandang sebagai

persoalan individu, tetapi juga sebagai masalah sosial dan kebijakan publik yang memerlukan penanganan dan perlindungan secara sistematis (Jannah, Nasrullah, dan Rachmad, 2024).

2.1.2 Dampak Kekerasan terhadap Perempuan dan Anak

Kekerasan terhadap perempuan dan anak menimbulkan dampak yang luas, baik secara fisik, psikis, maupun sosial. Dampak tersebut tidak hanya dirasakan langsung oleh korban, tetapi juga berpengaruh terhadap kehidupan keluarga dan lingkungan sekitar. Kondisi ini terjadi karena perempuan dan anak berada pada posisi sosial yang rentan, serta masih kuatnya nilai-nilai patriarki dan budaya diam yang membuat korban enggan melapor dan sulit memperoleh perlindungan (Gea, Mardhiyah, dan Jasri, 2025).

Secara fisik, korban sering mengalami luka, gangguan kesehatan, bahkan pada kasus tertentu dapat mengancam keselamatan jiwa. Sementara itu, dampak psikologisnya berupa trauma mendalam, depresi, kecemasan, dan perasaan takut yang berlarut-larut. Anak korban kekerasan juga cenderung mengalami perubahan perilaku, kehilangan semangat belajar, serta menarik diri dari lingkungan sosialnya (Hidayat, 2021).

Selain itu, kekerasan terhadap perempuan dan anak juga menimbulkan dampak sosial jangka panjang seperti stigmatisasi, kehilangan rasa aman, dan rusaknya hubungan sosial dalam keluarga maupun masyarakat. Korban kerap menanggung beban sosial akibat pandangan negatif masyarakat, sedangkan pelaku sering kali tidak mendapat hukuman yang setimpal. Kondisi ini memperkuat siklus kekerasan dan menurunkan kepercayaan publik terhadap lembaga perlindungan dan penegakan hukum yang ada (Hanapi, Moonti, dan Ahmad, 2025).

2.2 Sistem Informasi dan Manajemen Kasus

Sistem informasi memiliki peranan penting dalam mendukung proses pengelolaan data, pelaporan, dan pengambilan keputusan di berbagai sektor pelayanan publik, termasuk dalam bidang perlindungan perempuan dan anak. Melalui penerapan sistem informasi, lembaga dapat mengelola data kekerasan

secara terstruktur dan terintegrasi, sehingga proses identifikasi, pelaporan, hingga penanganan kasus dapat dilakukan secara lebih cepat dan akurat. Penerapan sistem informasi di lingkungan lembaga pelayanan publik juga meningkatkan efisiensi kerja dan transparansi antar-unit pelayanan, terutama dalam upaya penanganan kekerasan terhadap perempuan dan anak yang membutuhkan koordinasi lintas lembaga.

Dalam konteks penanganan kasus kekerasan terhadap perempuan dan anak, integrasi antara sistem informasi dan manajemen kasus sangat dibutuhkan untuk memantau setiap tahapan penanganan, mulai dari pelaporan, asesmen, pendampingan, hingga proses hukum. Sistem informasi manajemen kasus memungkinkan penyimpanan, pelacakan, dan pembaruan data korban secara *real time*, yang dapat diakses oleh instansi berwenang sesuai tingkat kewenangan masing-masing. Dengan adanya sistem tersebut, proses pelaporan menjadi lebih efisien, akurat, dan transparan karena seluruh pihak terkait dapat berkoordinasi dalam satu *platform* digital yang terintegrasi (Triana dkk., 2025).

2.2.1 Pengertian Sistem Informasi

Sistem informasi merupakan konsep yang mendasari berbagai aktivitas pengelolaan data dalam organisasi modern. Sistem informasi dapat didefinisikan sebagai serangkaian komponen yang bekerja bersama untuk mengumpulkan, mengelola, menyimpan, memproses, dan menyebarkan informasi yang diperlukan untuk mendukung pengambilan keputusan dalam suatu organisasi atau entitas. Komponen utama dalam sistem informasi mencakup pemanfaatan teknologi berbasis perangkat keras dan perangkat lunak yang terintegrasi untuk menghasilkan informasi yang bermakna bagi penggunaanya (Adham, 2024)

Dalam pengertian yang lebih luas, sistem informasi merupakan suatu kombinasi teratur dari orang perangkat keras, perangkat lunak, jaringan komunikasi, dan *database* yang mengumpulkan, mengubah, dan menyebarkan informasi di dalam suatu organisasi. Sistem informasi tidak hanya terbatas pada aspek teknologi semata, tetapi juga melibatkan unsur manusia dan prosedur kerja

yang saling terintegrasi untuk mencapai tujuan organisasi (Effendi, Sagalai, dan Rezeki, 2023).

Sistem informasi berfungsi sebagai sarana utama untuk mengumpulkan, mengolah, dan menyajikan informasi yang relevan dan akurat kepada manajemen, sehingga dapat membantu dalam pengambilan keputusan yang lebih baik. Berdasarkan berbagai definisi tersebut, dapat disimpulkan bahwa sistem informasi adalah sekumpulan komponen yang saling terintegrasi yang bekerja sama untuk mengumpulkan, memproses, menyimpan, dan mendistribusikan informasi guna mendukung pengambilan keputusan, koordinasi, dan pengendalian dalam suatu organisasi.

2.2.2 Sistem Pelaporan Kasus

Sistem pelaporan kasus merupakan suatu mekanisme yang memungkinkan masyarakat untuk menyampaikan laporan atau pengaduan terkait suatu kejadian kepada pihak berwenang secara terstruktur dan terdokumentasi. Dalam konteks kekerasan terhadap perempuan dan anak, sistem pelaporan berperan penting sebagai pintu masuk bagi korban atau saksi untuk melaporkan kasus yang terjadi. Sistem pelaporan berbasis *web* memungkinkan masyarakat untuk menyampaikan laporan secara *online* tanpa harus datang langsung ke kantor instansi terkait, serta memberikan kemudahan bagi petugas dalam mengelola dan memantau status pengaduan secara *real-time* (Rahmawati dan Setiawan, 2025)

Di Indonesia, Kementerian Pemberdayaan Perempuan dan Perlindungan Anak telah mengembangkan Sistem Informasi *Online* Perlindungan Perempuan dan Anak (SIMFONI-PPA) sebagai sistem informasi yang digunakan untuk melakukan pencatatan dan pelaporan kekerasan terhadap perempuan dan anak yang terjadi di seluruh wilayah Indonesia. Sistem ini bekerja sama dengan instansi pemerintahan di setiap provinsi, kabupaten, dan kota sehingga aplikasi dapat diakses oleh semua unit layanan penanganan korban kekerasan secara *real-time*. Masyarakat dapat melaporkan peristiwa kekerasan yang dilihat atau dialami melalui telepon *hotline* maupun aplikasi berbasis *web*.

Penerapan sistem pelaporan berbasis *web* dalam penanganan kasus kekerasan memberikan beberapa keuntungan, antara lain meningkatkan efisiensi dalam pencatatan dan pemantauan pengaduan, mempercepat proses tindak lanjut, serta meningkatkan transparansi antara lembaga penanganan dan masyarakat (Rahmawati dan Setiawan, 2025). Selain itu, sistem pelaporan digital juga dapat meningkatkan kepercayaan dan keberanian korban dalam melaporkan kasus kekerasan karena proses pelaporan dapat dilakukan secara aman dan terjaga kerahasiaannya (Kartaputra dan Sulistiawati, 2024).

2.2.3 Manajemen Kasus Kekerasan

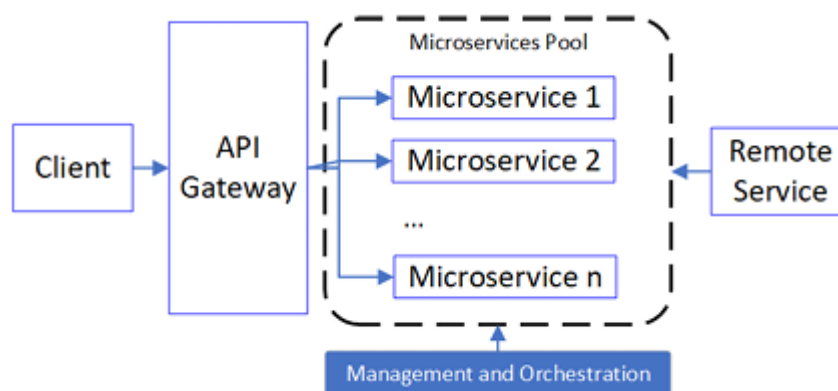
Manajemen kasus kekerasan merupakan serangkaian proses sistematis yang dilakukan untuk menangani kasus kekerasan terhadap perempuan dan anak secara komprehensif, mulai dari penerimaan laporan, asesmen, perencanaan intervensi, pelaksanaan layanan, hingga pemantauan dan evaluasi. Dalam konteks perlindungan perempuan dan anak, manajemen kasus bertujuan untuk memastikan korban mendapatkan layanan yang tepat, terkoordinasi, dan berkelanjutan. Penanganan kasus kekerasan memerlukan koordinasi dan komitmen antar lembaga dalam menyelenggarakan upaya perlindungan yang dibutuhkan korban, meliputi layanan penegakan hukum, layanan kesehatan, layanan rehabilitasi sosial, serta pendampingan psikologis.

Di Indonesia, Unit Pelaksana Teknis Daerah Perlindungan Perempuan dan Anak (UPTD PPA) menjadi ujung tombak dalam manajemen kasus kekerasan di tingkat daerah. UPTD PPA bertugas memberikan layanan pengaduan, penjangkauan, pengelolaan kasus, penampungan sementara, serta mediasi bagi korban kekerasan. Proses penanganan kasus di UPTD PPA dilakukan sesuai dengan Standar Pelayanan Minimum yang ditetapkan oleh pemerintah, mulai dari pelayanan pengaduan hingga pemulihan dan pemberdayaan korban. Dalam kinerjanya, UPTD PPA menjalin kerjasama secara horizontal dengan lembaga pemerintah lainnya, maupun secara vertikal dengan LSM, masyarakat, dan pemangku kepentingan lainnya dalam penanganan kasus (Putri dan Ritonga, 2024).

Penanganan kasus kekerasan yang efektif memerlukan kolaborasi berbagai pihak dengan pendekatan yang terkoordinasi. Keterlibatan aktif para aktor baik pemerintah, swasta, maupun masyarakat dalam penanganan kasus kekerasan menjadi kunci keberhasilan manajemen kasus. Selain itu, sinergi dalam pendokumentasian kasus kekerasan diharapkan mampu menghasilkan data yang lengkap, akurat, dan akuntabel, sehingga dapat memenuhi kebutuhan para pengambil kebijakan dalam upaya penanganan dan pencegahan kasus kekerasan terhadap perempuan dan anak (Rahmawati dan Hertati, 2023)

2.3 Arsitektur *Microservices*

Arsitektur *microservices* merupakan pendekatan arsitektur perangkat lunak yang memecah aplikasi menjadi layanan-layanan kecil yang saling terhubung dan dapat bekerja secara independen. Setiap layanan memiliki tanggung jawab yang spesifik dan dapat dikembangkan, diuji, serta di-*deploy* secara terpisah tanpa mempengaruhi layanan lainnya. Layanan-layanan ini berkomunikasi satu sama lain menggunakan *Application Programming Interface* (API) atau protokol komunikasi lainnya seperti gRPC. Arsitektur ini berbeda dengan arsitektur monolitik di mana seluruh komponen aplikasi digabungkan dalam satu kesatuan, sehingga perubahan pada satu bagian dapat mempengaruhi keseluruhan sistem (Arief dan Chaniago, 2025)



Gambar 2. 1 Hubungan klien, API Gateway dan *Services*.

Karakteristik utama arsitektur *microservices* meliputi autonomous di mana setiap layanan merupakan entitas terpisah yang bertanggung jawab terhadap proses

deployment-nya sendiri, *single responsibility* di mana setiap layanan fokus pada satu fungsi bisnis tertentu, *loose coupling* yang menjadikan layanan terpisah namun tetap terintegrasi dengan baik melalui API, serta *technology diversity* yang memungkinkan setiap layanan menggunakan bahasa pemrograman, *database*, atau *framework* yang paling sesuai dengan kebutuhannya. Konsep ini bertujuan agar sistem yang dibangun dapat menangani kegagalan secara parsial, di mana jika terdapat satu layanan yang bermasalah, layanan lainnya tetap dapat berjalan dengan normal (Dinova dan Utomo, 2023).

2.4 API Gateway

API Gateway merupakan sebuah layanan yang berperan sebagai pintu masuk utama dalam sistem dan berada di antara klien (*client*) dan layanan *backend* (*server*). Proses komunikasi dimulai ketika klien mengirimkan permintaan ke API gateway, kemudian layanan ini meneruskan permintaan tersebut ke endpoint yang sesuai untuk memperoleh data yang dibutuhkan. Setelah proses selesai, API gateway mengirimkan kembali respons hasil pemrosesan kepada klien (Arief dan Chaniago, 2025).

API gateway merupakan lapisan arsitektur kritis dalam sistem *microservices* yang berfungsi sebagai titik akses tunggal (*single entry point*) bagi seluruh permintaan dari klien, seperti *browser web* masyarakat atau dashboard DPPPA, menuju berbagai layanan *backend* yang independen. Secara konseptual, API Gateway bertindak sebagai pintu gerbang pintar yang menangani proses *routing*, transformasi permintaan, dan pengelolaan lalu lintas, sehingga menyembunyikan kompleksitas internal *microservices* dari pengguna akhir. Pendekatan ini mendukung prinsip *loose coupling* dan *scalability*, di mana klien tidak perlu mengetahui lokasi maupun protokol spesifik setiap layanan, melainkan cukup berinteraksi melalui RESTful API yang terstandarisasi di gateway (Neelan, 2025).

2.5 REST

REST atau *Representational State Transfer* merupakan gaya arsitektur yang digunakan dalam pengembangan *web service* untuk mendukung pertukaran data

antar sistem melalui protokol HTTP. REST tidak didefinisikan sebagai standar atau protokol tertentu, melainkan sebagai pendekatan arsitektural yang menekankan kesederhanaan dalam komunikasi antar aplikasi. Arsitektur ini memanfaatkan konsep *resource* sebagai pusat pertukaran data, di mana setiap *resource* dapat diakses melalui *Uniform Resource Identifier* (URI). Pendekatan REST memungkinkan sistem informasi yang dibangun dengan *platform* dan bahasa pemrograman berbeda untuk saling berkomunikasi secara efisien, sehingga mendukung interoperabilitas dan pengembangan sistem yang bersifat fleksibel (Novianto dan Munir, 2022).

Prinsip-prinsip dasar REST meliputi pemisahan antara *client* dan *server*, sifat *stateless*, orientasi terhadap *resource*, serta penggunaan metode HTTP standar. Pemisahan *client server* bertujuan agar antarmuka pengguna dan pengelolaan data dapat dikembangkan secara independen. Sifat *stateless* mengharuskan setiap permintaan dari *client* ke *server* membawa seluruh informasi yang dibutuhkan tanpa menyimpan status sesi di sisi *server*. Selain itu, setiap data direpresentasikan sebagai *resource* yang diakses melalui URI dan dimanipulasi menggunakan metode HTTP seperti GET, POST, PUT, dan DELETE yang merepresentasikan operasi *create*, *read*, *update*, dan *delete*. Penerapan prinsip-prinsip tersebut menjadikan REST API lebih sederhana, konsisten, dan mudah diintegrasikan dalam sistem informasi akademik berbasis *web* maupun *mobile* (Prayogi dkk., 2020).

Penerapan REST API dalam sistem informasi juga ditemukan pada sistem pengelolaan aset ruang, di mana REST digunakan sebagai penghubung antara *client* dan *server* dalam mengelola data ruang, pengguna, dan transaksi peminjaman. Data diperlakukan sebagai *resource* yang dapat diakses melalui HTTP *method* standar dengan format pertukaran data JSON. Pola penerapan ini memungkinkan sistem berjalan secara multiplatform serta mendukung proses pengelolaan data secara *real-time* dan terintegrasi. Pendekatan REST API tersebut menunjukkan bahwa arsitektur REST efektif digunakan dalam sistem informasi yang membutuhkan konsistensi data, kemudahan integrasi, dan efisiensi pengelolaan layanan (Achsan dan Susetyo, 2022).

2.6 *Containerization*

Containerization merupakan pendekatan modern dalam pengemasan dan penyebaran aplikasi dengan cara mengisolasi aplikasi beserta seluruh dependensinya ke dalam satu unit yang disebut *container*. Pendekatan ini memungkinkan aplikasi dijalankan secara konsisten di berbagai lingkungan komputasi tanpa dipengaruhi perbedaan konfigurasi sistem operasi atau infrastruktur. *Containerization* banyak digunakan dalam pengembangan sistem berbasis *microservices* karena mampu meningkatkan efisiensi penggunaan sumber daya, kemudahan *deployment*, serta skalabilitas sistem secara keseluruhan.

Berbeda dengan virtualisasi tradisional yang mengharuskan setiap aplikasi berjalan di atas sistem operasi tamu (*guest OS*), *Container* berbagi kernel sistem operasi *host* sehingga lebih ringan dan cepat dijalankan. Hal ini menjadikan *containerization* sangat sesuai untuk sistem informasi berbasis *web* yang membutuhkan keandalan, fleksibilitas, dan kemudahan pengelolaan layanan, seperti sistem pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak. Dalam konteks arsitektur *microservices*, *containerization* memungkinkan setiap layanan dijalankan secara terpisah, independen, dan mudah dikembangkan tanpa mengganggu layanan lainnya (Bentaleb dkk., 2022).

2.6.1 *Container*

Container merupakan teknologi virtualisasi tingkat aplikasi yang digunakan untuk mengemas aplikasi beserta seluruh dependensinya ke dalam satu lingkungan terisolasi. Dengan pendekatan ini, aplikasi dapat dijalankan secara konsisten pada berbagai lingkungan sistem tanpa dipengaruhi perbedaan konfigurasi sistem operasi. *Container* banyak dimanfaatkan dalam pengembangan arsitektur *microservices* karena mampu meningkatkan efisiensi penggunaan sumber daya dan mempermudah proses *deployment* layanan (Prasetyo dan Wijaya, 2021).

Container memiliki karakteristik utama seperti bersifat ringan (*lightweight*), waktu inisialisasi yang cepat, portabilitas tinggi, serta mendukung isolasi antar aplikasi. *Container* berjalan dengan memanfaatkan kernel sistem operasi *host*, sehingga tidak memerlukan sistem operasi tamu seperti pada *virtual machine*. Hal

ini menyebabkan *Container* membutuhkan sumber daya yang lebih kecil dan lebih efisien untuk menjalankan banyak layanan secara bersamaan (Riyanto, Safaat, dan Affandes, 2022).

Perbedaan mendasar antara *container* dan *virtual machine* terletak pada lapisan virtualisasi yang digunakan. *Virtual machine* memvirtualisasikan perangkat keras dan menjalankan sistem operasi secara penuh, sedangkan *container* hanya memvirtualisasikan lingkungan aplikasi. Akibatnya, *virtual machine* memiliki *overhead* yang lebih besar dibandingkan *container*. Dalam arsitektur *microservices*, *Container* dinilai lebih optimal karena mendukung pemisahan layanan secara independen dengan konsumsi sumber daya yang lebih efisien (Dirgantara, Kusumo, dan Utomo, 2024).

2.6.2 Docker

Docker adalah *platform open-source* yang mengotomatisasi penyebaran aplikasi menggunakan *lightweight containers*. Docker memungkinkan pengembang mengemas aplikasi beserta dependensinya ke dalam satu unit (*image*), yang menjamin konsistensi kinerja di berbagai lingkungan dari pengembangan hingga produksi. Ini adalah fondasi penting dalam arsitektur *microservices*, memfasilitasi operasi layanan yang independen, modular, dan efisien sumber daya (Gudelli, 2023).

Docker bekerja dengan memanfaatkan Docker Engine sebagai komponen utama yang menjalankan *Container* di atas sistem operasi *host*. Setiap *Container* dapat dijalankan secara independen dan saling berkomunikasi melalui jaringan internal yang disediakan oleh Docker. Dalam sistem informasi berbasis *web*, Docker mempermudah proses pengembangan dan *deployment* karena aplikasi dapat dijalankan dengan lingkungan yang konsisten pada tahap pengembangan, pengujian, hingga produksi (Wati dkk., 2025).

Keunggulan Docker antara lain kemudahan instalasi, konsistensi lingkungan aplikasi, efisiensi penggunaan sumber daya, serta dukungan terhadap pengembangan berkelanjutan. Docker juga memungkinkan setiap layanan dalam arsitektur *microservices* dijalankan dalam *container* terpisah, sehingga pembaruan

atau perbaikan pada satu layanan tidak mempengaruhi layanan lainnya. Hal ini menjadikan Docker sebagai solusi yang tepat untuk mendukung sistem pelayanan publik berbasis *web* yang membutuhkan keandalan dan skalabilitas tinggi (Riyanto, Safaat, dan Affandes, 2022).

2.7 Node JS

Node.js merupakan *runtime environment* berbasis JavaScript yang bersifat *open-source* dan *cross-platform*, memungkinkan pengembang untuk menjalankan kode JavaScript di sisi *server*. Node.js dibangun di atas *engine* V8 milik Google Chrome, sehingga mampu mengeksekusi kode JavaScript dengan kecepatan tinggi. Arsitektur Node.js mengadopsi model *event-driven* dan *non-blocking I/O*, yang menjadikannya sangat efisien dalam menangani banyak koneksi secara bersamaan dengan penggunaan sumber daya yang minimal. Pendekatan ini memungkinkan Node.js menjadi pilihan utama dalam pengembangan aplikasi *web* yang membutuhkan performa tinggi dan skalabilitas (Nurhayati dan Agussalim, 2023).

Keunggulan utama Node.js terletak pada kemampuannya menyatukan bahasa pemrograman di sisi *client* dan *server* dalam satu bahasa yaitu JavaScript, sehingga mempermudah proses pengembangan dan pemeliharaan sistem. Selain itu, Node.js dilengkapi dengan *Node Package Manager* (NPM) yang menyediakan ribuan *library* dan modul siap pakai untuk mempercepat proses pengembangan. Portabilitas Node.js juga menjadi nilai tambah karena dapat berjalan pada berbagai sistem operasi seperti Windows, Linux, dan macOS. Dalam konteks arsitektur *microservices*, Node.js sangat cocok digunakan karena mendukung pembuatan layanan-layanan kecil yang independen dan ringan, serta mampu menangani komunikasi antar-*service* melalui REST API secara efisien (Bratakusuma, Azmi, dan Ayuningtiyas, 2022).

2.8 JavaScript

JavaScript merupakan bahasa pemrograman yang digunakan untuk memungkinkan pengembang berinteraksi dengan berbagai fungsi yang disediakan oleh *web browser*. Secara khusus, JavaScript tergolong sebagai bahasa skrip, di mana kode sumbernya dijalankan melalui proses interpretasi pada saat runtime

tanpa melalui tahap kompilasi terlebih dahulu menjadi kode *byte*. Dalam penerapannya, JavaScript berfungsi untuk mengendalikan dan memodifikasi perilaku aplikasi lain yang umumnya dikembangkan menggunakan bahasa pemrograman berbeda, serta dieksekusi secara *real time* guna mendukung terciptanya aplikasi *web* yang dinamis dan interaktif (Galahartlambang, Khotiah, dan Jumain, 2021).

JavaScript merupakan bahasa scripting yang terintegrasi dengan HTML untuk membuat elemen *web* dinamis, seperti animasi, validasi *form*, dan pembaruan konten tanpa *reload* halaman penuh. Ia berbasis standar ECMAScript, mendukung pemrograman fungsional dan objek, serta diinterpretasikan langsung oleh *browser* tanpa kompilasi (Malavolta dkk.. 2023).

2.9 MongoDB

MongoDB merupakan sistem manajemen basis data yang termasuk dalam kategori NoSQL dengan model penyimpanan berbasis dokumen (*document-oriented*). Data dalam MongoDB tidak disimpan dalam bentuk tabel seperti basis data relasional, melainkan dalam bentuk koleksi dan dokumen menggunakan format BSON (*Binary JSON*) yang bersifat fleksibel. Dengan skema yang tidak kaku, MongoDB memungkinkan setiap dokumen dalam satu koleksi memiliki atribut yang berbeda-beda tanpa memerlukan perubahan struktur basis data secara menyeluruh, sehingga sangat sesuai untuk aplikasi yang memiliki kebutuhan data yang beragam dan berkembang secara dinamis (Maharani, Alifa, dan Febriga 2022; Yulius dan Susetyo, 2023).

Dalam pengembangan sistem informasi berbasis *web*, MongoDB banyak dipilih karena kemampuannya berintegrasi secara alami dengan ekosistem JavaScript, khususnya Node.js dan Express.js, sehingga format data yang digunakan konsisten di seluruh lapisan aplikasi. MongoDB juga menunjukkan performa yang unggul dalam operasi baca-tulis data dibandingkan basis data NoSQL lainnya, khususnya dalam menangani volume data yang besar secara bersamaan (Sinaga, Charibaldi, dan Cahyana, 2023). Dalam konteks arsitektur *microservices*, setiap layanan dapat memiliki instansi MongoDB-nya sendiri secara

independen, yang sejalan dengan prinsip *database per service* sehingga setiap *service* dapat berkembang tanpa bergantung pada layanan lainnya.

2.10 ExpressJS

Express.js merupakan *framework* aplikasi *web back-end* yang berjalan di atas *platform* Node.js dan dirilis sebagai perangkat lunak *open-source* di bawah lisensi MIT. *Framework* ini dirancang untuk membangun aplikasi *web* dan API dengan pendekatan yang minimalis namun fleksibel, sehingga memungkinkan pengembang untuk mengatur struktur aplikasi sesuai kebutuhan tanpa batasan arsitektur yang kaku. Express.js telah diakui sebagai standar *de facto framework server* untuk Node.js dan menjadi komponen *back-end* utama dalam *development stack* populer seperti MERN (MongoDB, Express, React, Node.js) (Pradigi, Harlina, dan Solehatin 2022).

Dalam pengembangan sistem informasi berbasis *web*, Express.js banyak dimanfaatkan untuk membangun RESTful API yang menghubungkan *frontend* dengan *backend*. *Framework* ini menyediakan fitur *routing* yang kuat, dukungan *middleware* yang luas, serta integrasi yang mudah dengan berbagai sistem basis data. Keunggulan Express.js meliputi arsitektur yang ringan, kecepatan eksekusi yang tinggi berkat mekanisme *non-blocking I/O* dari Node.js, serta ekosistem *middleware* dan *plugin* yang sangat luas melalui NPM. Pendekatan minimalis Express.js menjadikannya pilihan yang tepat untuk membangun layanan-layanan kecil dan independen dalam arsitektur *microservices*, di mana setiap layanan dapat dikembangkan dan di-*deploy* secara terpisah dengan efisien (Cahyono, Sucipto, dan Firliana 2023).

2.11 React JS

React.js merupakan *library* JavaScript yang digunakan untuk membangun antarmuka pengguna pada aplikasi *web*, khususnya aplikasi berbasis *single-page application* (SPA). *Single Page Application* (SPA) adalah teknologi yang bekerja di dalam *browser* yang tidak membutuhkan *reload page* saat digunakan, sehingga pengguna tidak akan berpindah halaman dengan melakukan *request* kepada *server*

setiap kali terjadi interaksi pada aplikasi. React.js menerapkan konsep pengembangan berbasis komponen, di mana antarmuka aplikasi dibagi ke dalam bagian-bagian kecil yang bersifat mandiri dan dapat digunakan kembali. Pendekatan ini memudahkan pengelolaan tampilan aplikasi serta meningkatkan efisiensi pengembangan antarmuka pengguna. Selain itu, React.js memanfaatkan mekanisme *Virtual DOM* untuk mengoptimalkan proses pembaruan tampilan, sehingga *browser* akan lebih cepat dalam melakukan *rendering* halaman secara menyeluruh tanpa harus *me-reload* keseluruhan halaman saat bernavigasi, melainkan hanya sebagian saja sesuai inputan dari pengguna (Santoso, 2021).

Dalam pengembangan sistem informasi berbasis *web*, React.js banyak digunakan untuk membangun antarmuka *admin* yang interaktif dan terstruktur. Penggunaan React.js memungkinkan pengembangan antarmuka dilakukan secara modular, sehingga mempermudah proses pemeliharaan serta pengembangan sistem secara berkelanjutan. React.js dapat dipadukan dengan *library* atau paket lain seperti *CSS Framework Bootstrap* agar tampilan dapat menyesuaikan di berbagai perangkat sekaligus, sehingga tidak hanya berfokus pada kecepatan melainkan juga tetap mengedepankan estetika desain *layout web*. Struktur komponen yang diterapkan pada React.js juga mendukung integrasi dengan layanan *backend* melalui API, sehingga mampu meningkatkan efisiensi waktu pengembangan aplikasi *web* dan mendukung terciptanya sistem informasi yang fleksibel serta mudah dikembangkan (Santoso, 2021).

2.12 RabbitMQ

RabbitMQ merupakan perangkat lunak *message broker* yang digunakan sebagai perantara pertukaran pesan antarlayanan pada sistem terdistribusi. RabbitMQ mendukung *Advanced Message Queuing Protocol (AMQP)* dengan pola *publisher* dan *consumer*. *Publisher* mengirimkan pesan ke *exchange*, kemudian *exchange* meneruskan pesan tersebut ke queue yang sesuai untuk diproses oleh *consumer*. Mekanisme ini memungkinkan layanan saling berkomunikasi secara asinkron tanpa harus menunggu proses pada layanan lain selesai (Ćatović dkk., 2022).

Dalam arsitektur *microservices*, penggunaan RabbitMQ membantu mengurangi ketergantungan langsung antarlayanan karena pesan ditempatkan pada antrian sebelum diterima dan diproses oleh layanan tujuan. Komunikasi melalui message broker membuat setiap layanan dapat menjalankan tugasnya secara independen sekaligus mendukung pengiriman pesan yang lebih andal ketika jumlah permintaan meningkat. (Penelitian Aprianto dkk, 2024) menunjukkan bahwa kinerja RabbitMQ dapat dievaluasi melalui *throughput* dan latensi berdasarkan jumlah serta ukuran pesan. Pada sistem yang dikembangkan, RabbitMQ digunakan untuk menyampaikan perubahan status laporan dari *case service* ke *report service* secara asinkron.

2.13 Unified Modeling Language (UML)

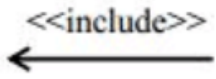
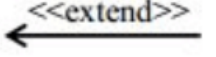
Unified Modeling Language (UML) merupakan seperangkat notasi dan *diagram* yang digunakan untuk merepresentasikan struktur serta perilaku suatu sistem dalam bentuk visual. Penggunaan UML memudahkan pengembang dalam memahami kebutuhan sistem, alur proses, dan hubungan antar komponen sebelum tahap implementasi, sehingga potensi kesalahan dalam perancangan dapat diminimalkan (Ramdany dkk., 2024).

Dalam konteks rekayasa perangkat lunak, UML berperan sebagai *blueprint* yang memfasilitasi komunikasi antara analis, pengembang, dan pemangku kepentingan non-teknis. Pemodelan sistem menggunakan UML turut mendukung proses pemeliharaan serta pengembangan jangka panjang, karena dokumentasi desain tersusun secara sistematis dan mudah dipahami. Adapun jenis-jenis *diagram* UML antara lain:

2.12.1 Use case Diagram

Use case diagram adalah *diagram* yang digunakan untuk merepresentasikan interaksi antara sistem dan aktor yang terlibat. *Diagram* ini menyajikan gambaran umum mengenai fungsi-fungsi utama sistem tanpa menjelaskan detail proses secara mendalam. Adapun simbol yang digunakan dalam *use case* dapat dilihat pada tabel 2.1 (Ramdany dkk., 2024) :

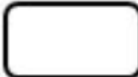
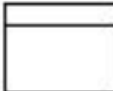
Tabel 2. 1 Simbol *Diagram Use case*

Simbol	Nama	Keterangan
	<i>Actor</i>	Mewakili peran orang, sistem yang lain, atau alat ketika berkomunikasi dengan <i>use case</i>
	<i>Use case</i>	Abstraksi dan interaksi antara sistem dan aktor
	<i>Association</i>	Abstraksi dari penghubung antara aktor dengan <i>use case</i>
	<i>Generalisasi</i>	Menunjukkan spesialisasi aktor untuk dapat berpartisipasi dengan <i>use case</i>
	<i>Include</i>	Menunjukkan bahwa suatu <i>use case</i> merupakan fungsionalitas dari <i>use case</i> lainnya
	<i>Exclude</i>	Menunjukkan bahwa suatu <i>use case</i> merupakan tambahan fungsional dari <i>use case</i>

2.12.2 *Activity Diagram*

Activity Diagram merupakan salah satu komponen penting dalam *Unified Modeling Language* (UML) yang digunakan untuk memvisualisasikan alur kerja, proses, atau aktivitas di dalam suatu sistem. *Diagram* ini menunjukkan keterkaitan antar tindakan serta bagaimana sistem berpindah dari satu keadaan ke keadaan lainnya. Dengan memberikan gambaran yang jelas mengenai alur kerja sederhana maupun kompleks, *Activity Diagram* membantu pengembang dan pemangku kepentingan memahami interaksi antar elemen dalam sistem. Adapun notasi yang digunakan *Activity Diagram* dapat dilihat pada tabel 2.2 (Ramdany dkk., 2024):

Tabel 2. 2 Notasi *Activity Diagram*

Simbol	Nama	Keterangan
	Status Awal	Sebuah <i>Diagram</i> aktivitas memiliki sebuah status awal.
	Aktivitas	Aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja.
	Percabangan / <i>Decision</i>	Percabangan di mana ada pilihan aktivitas yang lebih dari satu.
	Penggabungan / <i>Join</i>	Penggabungan di mana yang ada lebih dari satu aktivitas lalu digabungkan jadi satu.
	Status Akhir	Status akhir yang dilakukan sistem, sebuah <i>Activity Diagram</i> harus memiliki status akhir.
	<i>Swimlane</i>	<i>Swimlane</i> memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi.

2.12.3 *Class Diagram*

Class Diagram merupakan salah satu representasi dalam *Unified Modeling Language* (UML) yang berfungsi memodelkan struktur statis suatu sistem melalui pendefinisian kelas beserta atribut, operasi, dan relasinya. *Diagram* ini menyediakan deskripsi konseptual terhadap komponen-komponen sistem serta pola keterkaitan yang membentuk arsitektur perangkat lunak. Adapun notasi yang digunakan *Class Diagram* dapat dilihat pada tabel 2.3 (Arianti dkk., 2022) :

Tabel 2. 3 Simbol Class Diagram

Simbol	Nama	Keterangan
	<i>Association</i>	Relasi antar kelas
	<i>Class</i>	Merepresentasikan sebuah kelas yang ditulis dalam program.

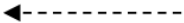
2.12.4 Sequence Diagram

Sequence Diagram berfokus pada representasi perilaku suatu sistem dengan menunjukkan bagaimana setiap objek saling berinteraksi. Di dalam *Sequence Diagram*, ditampilkan objek-objek beserta pesan yang dipertukarkan di antara objek tersebut. *Diagram* ini umumnya dimanfaatkan untuk memodelkan alur interaksi objek dalam suatu *use case*. Satu *use case* pada prinsipnya membutuhkan satu *Sequence Diagram*; namun apabila *use case* tersebut memiliki beberapa skenario, masing-masing skenario dapat digambarkan sebagai fragmen dalam *Sequence Diagram* yang sama (Arianti dkk., 2022). Adapun simbol dari *sequence* dapat dilihat pada tabel 2.4.

Tabel 2. 4 Simbol Sequence Diagram

Simbol	Nama	Keterangan
	<i>Actor</i>	<i>User</i> , orang, proses, atau sistem lain yang berinteraksi dengan sistem dan mendapatkan
	<i>Activation Box</i>	Mewakili periode di mana suatu elemen melakukan operasi
	<i>Lifeline</i>	Mendeskripsikan sebuah objek dalam sebuah sistem atau salah satu komponennya
	<i>Message</i>	Menggambarkan pesan atau interaksi antar objek

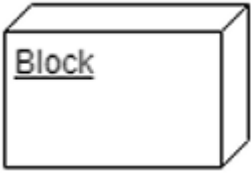
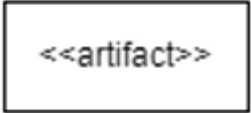
Tabel 2. 4 Simbol Sequence Diagram

Simbol	Nama	Keterangan
	<i>Message to self</i>	Menggambarkan pesan balikan atau reaksi dari objek sebelumnya

2.12.5 Deployment Diagram

Deployment Diagram merupakan salah satu jenis *diagram* dalam *Unified Modeling Language* (UML) yang digunakan untuk memvisualisasikan, menspesifikasikan, dan mendokumentasikan konfigurasi fisik dari sistem perangkat lunak yang akan dibangun. *Diagram* ini menggambarkan bagaimana komponen perangkat lunak (*artifact*) didistribusikan dan ditempatkan pada perangkat keras (*node*) dalam lingkungan eksekusi sistem, misalnya server web, server database, atau container. *Deployment Diagram* mewakili pandangan penyebaran sistem sehingga biasanya hanya akan terdapat satu *Deployment Diagram* untuk satu sistem. Setiap node pada *diagram* merepresentasikan satu unit komputasi yang berupa perangkat keras atau lingkungan eksekusi perangkat lunak, seperti server, *workstation*, atau *container* (Andriati, 2023) . Adapun simbol yang digunakan di *Deployment Diagram* dapat dilihat pada Tabel 2.5.

Tabel 2. 5 Simbol Deployment Diagram

Simbol	Nama	Keterangan
	<i>Node</i>	Merepresentasikan sumber daya komputasi tempat artifact dijalankan, seperti <i>server</i> , perangkat keras, atau <i>container</i> Docker.
	<i>Artifact</i>	Spesifikasi bentuk fisik informasi yang digunakan atau dihasilkan, seperti file konfigurasi atau <i>deployment specification</i> .

2.14 Rational Unified Process (RUP)

Rational Unified Process (RUP) merupakan kerangka kerja pengembangan perangkat lunak yang bersifat iteratif dan dikembangkan oleh *Rational Software*

Corporation, yang kini menjadi bagian dari IBM sejak tahun 2003. RUP tidak berupa metode tunggal dengan aturan kaku, melainkan berupa kerangka proses yang fleksibel dan dapat disesuaikan dengan kebutuhan tiap proyek, sehingga organisasi dan tim pengembang dapat memilih dan memodifikasi elemen-elemen proses sesuai karakteristik sistem yang dibangun (Hardianti Ramli dkk., 2026).

RUP terdiri dari empat fase utama, di mana setiap fase memiliki tujuan dan hasil spesifik (Irawan Chandra dkk., 2024). Berikut adalah penjelasan keempat fase tersebut:

1. Fase Insepsi

Fase ini adalah fase awal yang berfokus pada kajian mendalam untuk mengenali kebutuhan sistem di masa depan. Aktivitas yang dilakukan meliputi analisis tujuan bisnis, identifikasi peluang, penentuan ruang lingkup proyek, perumusan solusi yang akan digunakan, penetapan kriteria efektivitas, evaluasi risiko, serta perkiraan kebutuhan sumber daya. Pada fase ini juga disusun jadwal awal proyek sebagai acuan koordinasi pelaksanaan keempat fase pengembangan.

2. Fase Elaborasi

Pada fase ini, arsitektur sistem dikembangkan berdasarkan domain bisnis yang telah diidentifikasi sebelumnya, disertai analisis domain yang dilakukan lebih mendalam agar gambaran proyek menjadi semakin jelas. Fokus utama fase ini adalah mengurangi atau menangani risiko-risiko kritis yang telah ditemukan pada tahap sebelumnya melalui studi kasus bisnis dan penetapan target peningkatan kualitas arsitektur sistem.

3. Fase Konstruksi

Pada tahap ini, proyek berskala besar dibagi menjadi sejumlah subproyek, di mana setiap bagian mewakili satu iterasi dalam pengembangan berbasis RUP. Proses dikerjakan dengan pendekatan iteratif dan inkremental untuk menekan kemungkinan kegagalan sistem. Kegiatan yang dilakukan meliputi penyempurnaan kebutuhan yang sebelumnya belum terdefinisi, perancangan sistem secara rinci, proses implementasi, serta pengujian fungsi-fungsi sistem.

4. Fase Transisi

Tahap akhir ini menitikberatkan pada penyerahan sistem kepada pengguna akhir. Prosesnya mencakup serangkaian pengujian, perbaikan produk, peninjauan aspek nonfungsional, serta penanganan berbagai kendala yang muncul. Versi awal sistem yang sebelumnya diuji secara bertahap kemudian diperbarui dan digantikan dengan versi final setelah seluruh perbaikan diselesaikan berdasarkan hasil pengujian.

2.15 Load testing

Load testing atau pengujian beban merupakan jenis pengujian kinerja yang digunakan untuk mengukur respons sistem terhadap kondisi beban pengguna yang berbeda-beda. Pengujian ini dilakukan dengan menambahkan beban secara bertahap untuk melihat perubahan yang terjadi pada sistem ketika sejumlah pengguna mengaksesnya secara bersamaan (Tejaya dkk., 2023). Tujuan dari pengujian beban adalah untuk mengidentifikasi kapasitas maksimum sistem dalam menampung jumlah pengguna yang mengakses secara bersamaan, sehingga dapat diketahui batas kemampuan sistem sebelum mengalami penurunan kinerja atau kegagalan (Setiawan dan Enda, 2025).

Dalam pelaksanaannya, pengujian beban umumnya dibagi menjadi tiga kondisi skenario yang berbeda. Kondisi *baseline* menggambarkan situasi di mana sistem diakses pada saat layanan belum terlalu digunakan atau dalam kondisi penggunaan normal yang rendah. Kondisi *peak* menggambarkan situasi di mana sistem diakses pada saat layanan sedang digunakan secara aktif oleh pengguna. Kondisi *stress* menggambarkan situasi di mana sistem diakses dengan jumlah pengguna dua kali lebih banyak dari kondisi sebelumnya, untuk menguji batas kemampuan sistem dalam menghadapi beban yang berlebih (Setiawan dan Enda, 2025; Tejaya dkk., 2023).

Terdapat 3 aspek metrik utama yang diamati dalam pengujian *load testing*, yaitu *throughput*, *error rate*, dan *response time*. Penjelasan dari masing-masing metrik beserta rumus perhitungannya adalah sebagai berikut (Setiawan dan Enda, 2025) :

1. *Throughput*

Throughput merupakan jumlah permintaan pengguna yang berhasil diproses oleh sistem per satuan waktu. Metrik ini menunjukkan kapasitas sistem dalam menangani volume permintaan secara bersamaan. Semakin tinggi nilai *throughput*, maka semakin besar kemampuan sistem dalam memproses permintaan pengguna. Nilai *throughput* dihitung menggunakan rumus berikut:

$$T = \frac{N}{D} \quad (2.1)$$

Keterangan :

T = *throughput* (request/detik)

N = Jumlah seluruh *sample/request*

D = total/rentang waktu pengujian (detik)

2. *Error rate*

Error rate merupakan perbandingan antara jumlah permintaan yang gagal diproses dengan total seluruh permintaan yang dikirimkan oleh sistem. Metrik ini menjadi indikator penting *dalam* menilai keandalan sistem, di mana semakin rendah nilai *error rate* maka semakin stabil sistem dalam memproses permintaan pengguna. Sistem yang baik memiliki nilai *error rate* sebesar 0%, yang berarti seluruh permintaan berhasil diproses tanpa kegagalan. Nilai *error rate* dihitung menggunakan rumus berikut:

$$E = \left(\frac{F}{N} \right) \times 100\% \quad (2.2)$$

Keterangan:

E = *Error rate* (%)

F = Jumlah permintaan yang gagal

N = Jumlah keseluruhan permintaan yang dikirimkan

3. *Average Response Time*

Average Response time merupakan total waktu yang dibutuhkan sistem mulai dari saat klien mengirimkan permintaan hingga seluruh respons diterima kembali oleh klien. Waktu respons sebesar 0,1 detik membuat pengguna merasa bahwa sistem merespons secara instan tanpa memerlukan umpan balik khusus. Semakin

kecil nilai *response time*, maka semakin baik pengalaman pengguna dalam menggunakan sistem dihitung menggunakan rumus berikut:

$$RT = \frac{T_s}{N} \quad (2.3)$$

Keterangan:

RT = Rata-rata *response time* (ms)

T_s = total waktu *response* dari seluruh permintaan

N = jumlah keseluruhan *sample* yang dikirimkan

2.16 Fault Injection

Fault injection merupakan teknik pengujian yang dilakukan dengan memasukkan gangguan secara sengaja dan terkendali ke dalam sistem untuk mengamati perilaku sistem ketika menghadapi kondisi kegagalan. Gangguan dapat diberikan saat sistem berjalan dalam bentuk penambahan latensi, pembatasan bandwidth, gangguan lalu lintas HTTP atau basis data, maupun penghentian permintaan. Hasil pengujian kemudian dibandingkan dengan kondisi normal untuk menilai pengaruh gangguan terhadap waktu respons, skalabilitas, dan ketahanan layanan (Al-Said Ahmad dan Andras, 2022).

Pada sistem berbasis *microservices*, *fault injection* digunakan untuk memeriksa kemampuan isolasi kegagalan, mekanisme pemulihan, dan ketepatan penanganan kesalahan antarlayanan. Pengujian dilakukan melalui skenario kegagalan yang dapat diulang agar pengembang dapat mengetahui jalur penyebaran kesalahan dan komponen yang menjadi titik lemah toleransi kegagalan. (Wu dkk. 2023) menjelaskan bahwa pengujian *fault injection* dapat mengeksplorasi dan memvalidasi komponen redundan pada sistem layanan dengan mengidentifikasi kombinasi gangguan yang memutus jalur eksekusi. Dalam penelitian ini, skenario *fault injection* dapat dilakukan dengan menonaktifkan salah satu *service* atau menambahkan latensi untuk memeriksa apakah layanan lain tetap berjalan dan pesan RabbitMQ dapat diproses setelah *service* pulih.

2.17 Apache JMeter

Apache JMeter merupakan perangkat lunak *open-source* berbasis Java yang dirancang untuk menguji perilaku fungsional dan mengukur kinerja sistem. JMeter bekerja dengan mengirimkan sejumlah permintaan HTTP ke *endpoint* API yang diuji secara simultan melalui mekanisme *thread group*, kemudian mengumpulkan data statistik dari setiap respons yang diterima. JMeter mendukung berbagai protokol pengujian termasuk HTTP, HTTPS, JDBC, LDAP, JMS, dan protokol lainnya, sehingga sangat fleksibel untuk digunakan pada berbagai jenis sistem (Ginasari dkk., 2021).

Dalam konfigurasi pengujian JMeter, terdapat beberapa parameter utama yang perlu ditentukan. *Number of Threads* merupakan jumlah pengguna virtual (*concurrent users*) yang akan mengakses sistem secara bersamaan. *Ramp-Up Period* merupakan waktu yang dibutuhkan JMeter untuk memulai seluruh *thread*, di mana semakin kecil nilainya maka beban akan semakin berat karena seluruh pengguna dimulai dalam waktu yang singkat. *Loop Count* menentukan berapa kali setiap *thread* akan mengeksekusi skenario pengujian. Hasil pengujian JMeter disajikan dalam bentuk tabel ringkasan (*Summary Report*) dan grafik yang memuat metrik-metrik performa seperti *average response time*, *throughput*, *standard deviation*, dan *error rate* (Farchani dkk., 2025).

2.18 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) merupakan tahap pengujian yang dilakukan setelah proses pengembangan sistem selesai dengan melibatkan pengguna secara langsung dalam proses evaluasi. Dari pelaksanaan pengujian ini dihasilkan dokumen yang menjadi bukti bahwa aplikasi telah memenuhi kebutuhan pengguna dan dinilai layak untuk diterapkan. Tujuan utama UAT adalah memastikan setiap fitur dapat dijalankan dengan baik dan nyaman tanpa kendala. Selain mengecek kesesuaian fungsi dengan kebutuhan, UAT juga membantu menemukan bagian sistem yang masih perlu diperbaiki serta mengidentifikasi kesalahan yang mungkin belum terdeteksi pada pengujian teknis (Thabibi dkk., 2025).

2.19 Skala Likert

Skala Likert merupakan metode pengukuran yang digunakan untuk mengkuantifikasi sikap, pendapat, atau persepsi responden terhadap kegunaan sistem. Skala ini dipilih karena mampu memberikan gambaran objektif mengenai tingkat penerimaan pengguna secara sistematis (Huh dan Gim, 2025).

Dalam pengolahan hasil UAT biasanya digunakan skala Likert sebagai metode penilaian. Skala Likert merupakan skala pengukuran yang dipakai untuk menilai pendapat, sikap, dan persepsi seseorang terhadap suatu sistem atau pernyataan. Melalui skala ini, jawaban pengguna dikonversi menjadi nilai terukur sehingga tingkat penerimaan dan kepuasan terhadap sistem dapat dihitung secara kuantitatif, seperti yang ditunjukkan pada Tabel 2.6.

Tabel 2. 6 Tabel Skala Likert

No.	Bobot	Persentase	Skala Jawaban	Keterangan
1.	5	84,01% - 100%	SS	Sangat Setuju
2.	4	68,01% - 84,00%	S	Setuju
3.	3	52,01% - 68,00%	N	Netral
4.	2	36,01% - 52,00%	TS	Tidak Setuju
5.	1	20,00% - 36,00%	STS	SangatTidak Setuju

BAB III

METODOLOGI PENELITIAN

3.1. Metode Pengumpulan Data

Metode Pengumpulan data yang digunakan dalam penelitian ini adalah sebagai berikut :

1. Studi Literatur

Studi literatur dilakukan dengan mengumpulkan berbagai referensi yang berkaitan dengan sistem pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak, sistem informasi berbasis *web*, serta arsitektur *microservices*. Sumber referensi meliputi jurnal ilmiah, artikel konferensi, buku, peraturan pemerintah, dan sumber daring terpercaya. Studi ini bertujuan untuk memahami konsep, metode, serta praktik terbaik dalam pengembangan sistem pelayanan publik berbasis teknologi.

2. Wawancara

Wawancara dilakukan dengan pihak Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA) Kota Kendari untuk memperoleh informasi mengenai alur pelaporan yang berjalan saat ini, kendala yang dihadapi, serta kebutuhan sistem yang diharapkan. Data ini digunakan sebagai dasar dalam perancangan sistem yang sesuai dengan kebutuhan pengguna.

3. Observasi

Observasi dilakukan dengan mengamati proses pelaporan dan penanganan kasus kekerasan yang berlangsung secara manual di lingkungan DPPPA dan kelurahan. Hasil observasi digunakan untuk mengidentifikasi kelemahan sistem yang berjalan serta peluang penerapan sistem berbasis *web*

3.2. Metode Pengembangan Sistem

Metode pengembangan sistem yang digunakan dalam sistem ini adalah metode *Rational Unified Process* (RUP). Dalam metode ini, terdapat empat tahap pengembangan perangkat lunak, yaitu:

1. *Inception* (Permulaan)

Pada tahap ini dilakukan identifikasi masalah, penentuan ruang lingkup penelitian, serta pengumpulan kebutuhan fungsional dan nonfungsional sistem. Peneliti menganalisis kebutuhan sistem pelaporan kekerasan berbasis *web*, menentukan aktor sistem, serta menetapkan tujuan pengembangan sistem.

2. *Elaboration* (Perencanaan)

Tahap ini berfokus pada perancangan arsitektur sistem berbasis *microservices*, perancangan basis data, serta pemodelan sistem menggunakan UML. Risiko pengembangan juga dianalisis, seperti keamanan data dan integrasi antar *service*.

3. *Construction* (Konstruksi)

Tahap ini merupakan fase implementasi sistem menggunakan teknologi Express JS, MongoDB, dan REST API. Setiap layanan *microservices* dibangun secara terpisah, meliputi layanan pelaporan, verifikasi, dan manajemen kasus. Pengujian dilakukan secara bertahap.

4. *Transition* (Transisi)

Pada tahap ini dilakukan pengujian langsung oleh pengguna, penyempurnaan berdasarkan masukan, serta dokumentasi sistem. Sistem yang telah selesai dikembangkan kemudian dipersiapkan agar dapat digunakan sebagai sarana pelaporan dan manajemen kasus di DPPPA Kota Kendari.

3.3. Waktu dan Tempat Penelitian

3.3.1. Waktu Penelitian

Waktu pelaksanaan penelitian tugas akhir dilakukan mulai bulan Mei 2026 sampai dengan Agustus 2026. Adapun rincian kegiatan untuk waktu penelitian dapat dilihat pada Tabel 3.1 berikut:

Tabel 3. 1 Waktu Penelitian

No.	Uraian	Waktu Penelitian (2026)															
		Mei				Juni				Juli				Agustus			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	<i>inception</i>																
2	<i>Elaboration</i>																
3	<i>Construction</i>																
4	<i>Transition</i>																

3.3.2. Tempat Penelitian

Penelitian tugas akhir ini dilaksanakan di dua lokasi, yaitu Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA) Kota Kendari sebagai lokasi studi kasus dan pengambilan data, serta Laboratorium *Software Engineering* Jurusan Teknik Informatika, Fakultas Teknik, Universitas Halu Oleo sebagai lokasi pengembangan sistem.

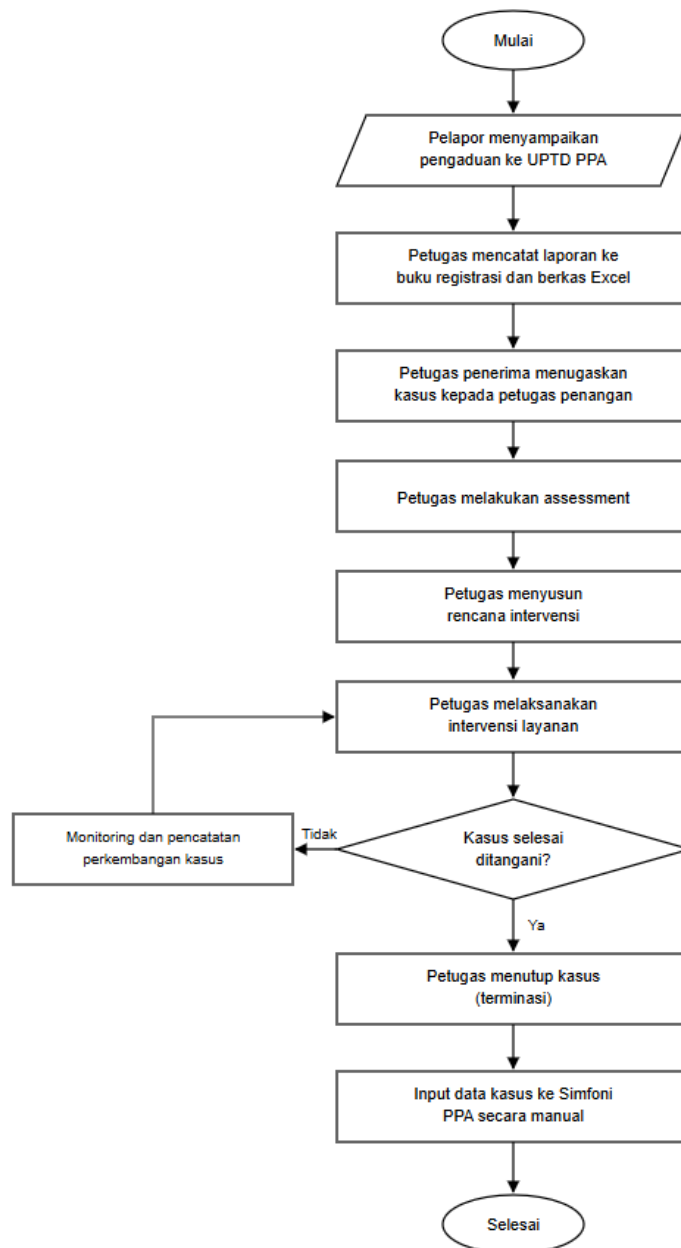
BAB IV

ANALISIS DAN PERANCANGAN

4.1. Analisis Sistem yang Sedang Berjalan

Penanganan kasus kekerasan terhadap perempuan dan anak di Kota Kendari dilaksanakan oleh Dinas Pemberdayaan Perempuan dan Perlindungan Anak (DPPPA) Kota Kendari. Berdasarkan hasil wawancara dengan pihak DPPPA Kota Kendari, petugas layanan yang menangani kasus berjumlah sembilan orang tanpa pembagian bidang khusus, sehingga seluruh petugas dapat menangani seluruh tahapan penanganan kasus. Laporan masuk melalui pengaduan langsung ke kantor, penjangkauan oleh petugas ke lokasi, maupun rujukan dari pihak kelurahan, dengan volume rata-rata sekitar sepuluh laporan setiap bulan. Setiap laporan yang diterima dicatat oleh petugas ke dalam buku registrasi dan berkas *Microsoft Excel*.

Setelah laporan tercatat, penanganan kasus dilanjutkan oleh petugas layanan. Pada praktiknya, petugas yang menerima laporan dapat langsung menangani kasus tersebut hingga selesai, namun sebagian besar kasus diserahkan kepada petugas lain untuk ditangani. Ketiadaan pembagian bidang khusus menyebabkan penugasan bersifat fleksibel dan tidak terikat pada struktur tertentu. Petugas yang menangani kemudian melakukan *assessment* terhadap kondisi dan kebutuhan korban, menyusun rencana intervensi, serta melaksanakan intervensi layanan berupa konseling, pendampingan, maupun rujukan ke layanan hukum dan kesehatan. Perkembangan penanganan dipantau melalui kegiatan monitoring, dan setiap perkembangan dicatat kembali secara manual pada buku dan berkas Excel. Setelah kasus dinyatakan selesai (terminasi), data kasus diinput ulang secara manual ke dalam sistem Simfoni PPA milik Kementerian Pemberdayaan Perempuan dan Perlindungan Anak. Alur penanganan kasus yang sedang berjalan ditunjukkan pada Gambar 4.1.



Gambar 4. 1 *Flowchart* Sistem yang Sedang Berjalan

Berdasarkan alur tersebut, teridentifikasi beberapa permasalahan. Pencatatan yang dilakukan pada dua media berbeda, yaitu buku registrasi dan berkas Excel, kemudian diinput ulang ke Simfoni PPA, menimbulkan pekerjaan ganda dan berpotensi menyebabkan data tidak sinkron antarmedia. Selain itu, pelapor tidak memiliki sarana untuk mengetahui perkembangannya secara mandiri, sehingga harus menghubungi atau mendatangi kantor secara langsung.

4.2. Analisis Kebutuhan Sistem

4.2.1. Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan layanan-layanan spesifik yang harus disediakan oleh sistem bagi pengguna, yaitu pelapor (masyarakat), *admin* petugas, dan *super admin*, serta bagaimana sistem merespons input dari masing-masing pengguna. Rincian kebutuhan fungsional disajikan pada Tabel 4.1 berikut

Tabel 4.1 Kebutuhan Fungsional Sistem

No.	Kebutuhan Fungsional	Deskripsi
1	Pelaporan Kasus oleh Masyarakat	Sistem harus menyediakan formulir pelaporan kasus kekerasan terhadap perempuan dan anak yang dapat diakses tanpa <i>login</i> , dengan isian data seperti identitas korban, jenis kekerasan, kronologis, lokasi, dan tanggal kejadian.
2	Pembuatan kode laporan otomatis	Sistem harus menghasilkan kode laporan secara otomatis setelah pelapor mengirimkan laporan, yang dapat digunakan untuk memantau status penanganan kasus.
3	Cek Status Laporan	Pelapor dapat memasukkan kode laporan untuk melihat status penanganan kasus secara <i>real-time</i> , meliputi tahapan yang sedang berjalan mulai dari menunggu registrasi, proses <i>assessment</i> , dalam penanganan, hingga selesai.
4	<i>Login admin</i> petugas dan <i>super admin</i>	Sistem menyediakan fitur autentikasi bagi <i>admin</i> petugas dan <i>super admin</i> menggunakan <i>username</i> dan <i>password</i> untuk membatasi akses sesuai dengan peran masing-masing pengguna.
5	Registrasi laporan oleh <i>admin</i> petugas	<i>Admin</i> petugas dapat meninjau kelengkapan laporan yang masuk dan melakukan registrasi terhadap laporan tersebut.

Tabel 4.1 Kebutuhan Fungsional Sistem (Lanjutan)

No.	Kebutuhan Fungsional	Deskripsi
6	Input <i>assessment</i> oleh <i>admin</i> petugas	<i>Admin</i> petugas dapat mengisi formulir <i>assessment</i> terhadap kasus yang telah diregistrasi, mencakup kategori kasus, tingkat keparahan, dan rekomendasi penanganan. Setelah <i>assessment</i> disimpan, sistem memperbarui status kasus menjadi dalam penanganan.
7	Manajemen akun admin oleh <i>Super admin</i>	<i>Super admin</i> dapat menambahkan, mengubah, dan menghapus akun <i>admin</i> petugas yang terdaftar dalam sistem sesuai dengan kebutuhan operasional.
8	Pengelolaan data master oleh <i>super admin</i>	<i>Super admin</i> dapat mengelola data master sistem melalui pusat kendali layanan, yang mencakup pengelolaan kategori kekerasan dan metode penanganan yang tersedia dalam sistem.
9	<i>Dashboard</i> dan monitoring kasus	Sistem menyediakan <i>dashboard</i> bagi <i>admin</i> petugas yang menampilkan ringkasan status kasus secara <i>real-time</i> , serta <i>dashboard</i> bagi <i>super admin</i> yang menampilkan statistik agregat, tren pelaporan per bulan, distribusi kategori kekerasan, dan rekapitulasi kinerja petugas.
10	Kelola kasus dalam penanganan	<i>Admin</i> petugas dapat memantau perkembangan kasus yang sedang dalam penanganan aktif, menambahkan catatan perkembangan melalui fitur <i>log</i> aktivitas, serta menyelesaikan kasus apabila penanganan telah tuntas.
11	Ekspor data pelaporan	<i>Super admin</i> dapat melakukan pemfilteran data laporan berdasarkan rentang tanggal dan wilayah, serta mengekspor hasil filter ke dalam format <i>Excel</i> untuk keperluan pelaporan dan evaluasi kebijakan.

4.2.2. Kebutuhan Non Fungsional

Analisis kebutuhan non fungsional merupakan langkah penting dalam pembangunan aplikasi, di mana dilakukan penilaian terhadap sumber daya yang diperlukan. Proses analisis ini terbagi menjadi dua tahap utama, yaitu analisis kebutuhan perangkat keras dan analisis kebutuhan perangkat lunak. Tujuan dari analisis kebutuhan non fungsional adalah untuk mengidentifikasi dan memahami semua kebutuhan sumber daya yang diperlukan dalam pembangunan aplikasi, baik dari segi perangkat keras maupun perangkat lunak.

1. Kebutuhan Perangkat Keras (*Hardware*)

Untuk menerapkan rancangan yang telah dijelaskan sebelumnya, dibutuhkan beberapa perangkat keras sebagai sarana untuk mengimplementasikan aplikasi yang dibangun. Kebutuhan perangkat keras ditunjukkan pada Tabel 3.3 berikut.

Tabel 4.2 Kebutuhan Perangkat Keras

No	Nama Perangkat	Spesifikasi
1	Laptop	ASUS TUF GAMING F15
2	Processor	Intel(R) Core(TM) i5-10300H CPU
3	Memori (RAM)	8,00 GB DDR4
4	Storage	SSD 512 GB

2. Kebutuhan Perangkat Lunak (*Software*)

Adapun perangkat lunak yang dibutuhkan dalam penelitian ini ditunjukkan pada Tabel 4.3 berikut.

Tabel 4.3 Kebutuhan Perangkat Lunak

No	Nama perangkat	Kategori	Spesifikasi
1	Windows	Sistem Operasi	Windows 11 <i>Home</i>
2	NodeJS	<i>Runtime Environment</i>	Versi 24
3	Express JS	<i>Framework Backend</i>	Versi 5.2.1
4	MongoDB	<i>Database</i>	Versi 7.0
5	React JS	<i>Frontend</i>	Versi 18

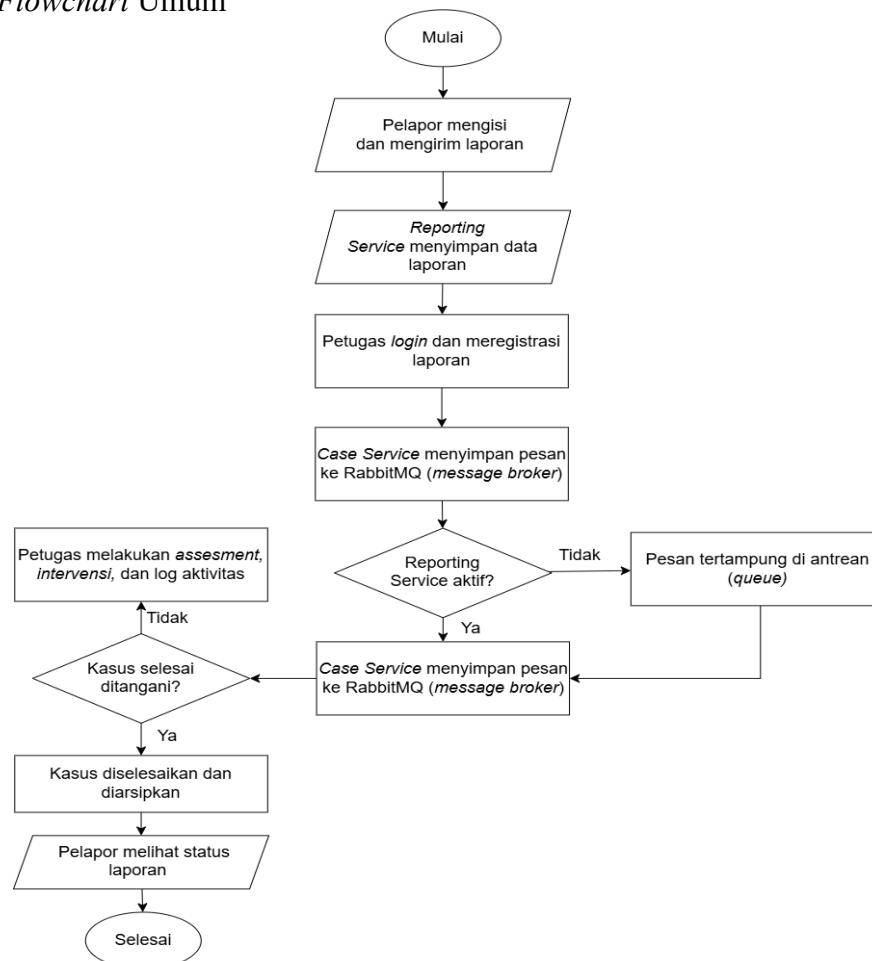
4.3. Analisis Perancangan Sistem

Perancangan sistem yang akan dibangun terdiri atas perancangan *Flowchart*, perancangan *Unified Modeling Language* (UML) dan perancangan *user interface*.

4.3.1 *Flowchart*

Flowchart merupakan *diagram* yang digunakan untuk menggambarkan alur langkah-langkah dalam menyelesaikan suatu masalah atau proses secara visual menggunakan simbol-simbol standar. Dalam penelitian ini, disusun tiga *Flowchart* sebagai acuan perancangan dan implementasi sistem, yaitu proses pelaporan, verifikasi laporan dan kelola metode penanganan.

1. *Flowchart* Umum



Gambar 4.2 *Flowchart* Umum

Flowchart pada Gambar 4.2 menggambarkan alur kerja proses pelaporan kasus dalam sistem yang dibangun dengan pendekatan berbasis layanan (*microservice*) dan komunikasi asinkron melalui *message broker*. Proses dimulai

ketika pelapor mengisi dan mengirimkan laporan melalui sistem. Data laporan tersebut kemudian disimpan oleh *Report service*.

Selanjutnya, petugas melakukan *login* untuk meregistrasi laporan yang masuk. Setelah laporan diregistrasi, *Case service* mengirimkan pesan ke *message broker* (RabbitMQ) sebagai mekanisme komunikasi antar layanan. Sistem kemudian memeriksa status ketersediaan *Report service*. Apabila *Report service* dalam kondisi tidak aktif (tidak), pesan akan tertampung sementara di dalam antrean (*queue*) hingga layanan kembali aktif dan pesan dapat diteruskan. Apabila *Report service* aktif (ya), pesan langsung diproses dan diteruskan ke *Case service* untuk disimpan kembali ke *message broker*.

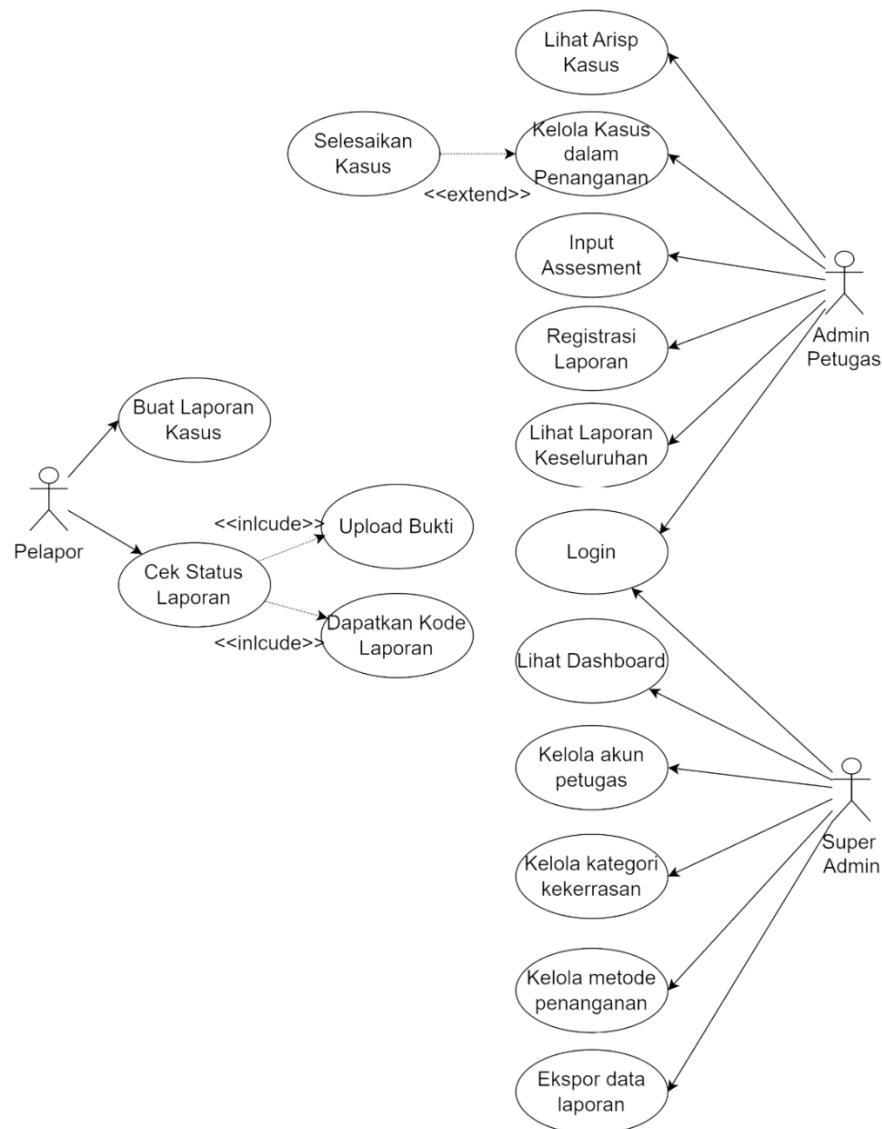
Pesan yang telah diproses tersebut kemudian masuk ke tahap penanganan kasus. Sistem memeriksa apakah kasus telah selesai ditangani. Apabila kasus belum selesai (tidak), petugas melakukan asesmen, intervensi, dan pencatatan (*log*) aktivitas penanganan, kemudian sistem kembali memeriksa status penyelesaian kasus tersebut secara berulang hingga kasus dinyatakan selesai. Apabila kasus telah selesai ditangani (ya), kasus akan diselesaikan dan diarsipkan oleh sistem.

Pada tahap akhir, pelapor dapat melihat status laporan yang telah diajukan. Alur ini dirancang untuk menjamin keandalan pertukaran pesan antar layanan melalui mekanisme antrean pada *message broker*, sehingga proses pelaporan tetap dapat berjalan meskipun salah satu layanan mengalami gangguan sementara, serta memastikan setiap kasus tertangani secara terstruktur hingga tuntas.

4.3.2 Analisis Perancangan *Unified Modeling Language* (UML)

Perancangan *Unified Modeling Language* (UML) dilakukan untuk memodelkan sistem secara visual menggunakan *diagram* yang terdiri dari *use case diagram*, *Class Diagram*, *Activity Diagram*, dan *Sequence Diagram*.

1. Use case Diagram



Gambar 4.3 Use Case Diagram

Use case diagram digunakan untuk memvisualisasikan interaksi antara sistem dengan aktor (pengguna). Di mana *diagram* ini membantu mengidentifikasi fungsional sistem tanpa memperhatikan hal detail secara teknis. Adapun *use case diagram* sistem ini ditunjukkan pada gambar 4.3.

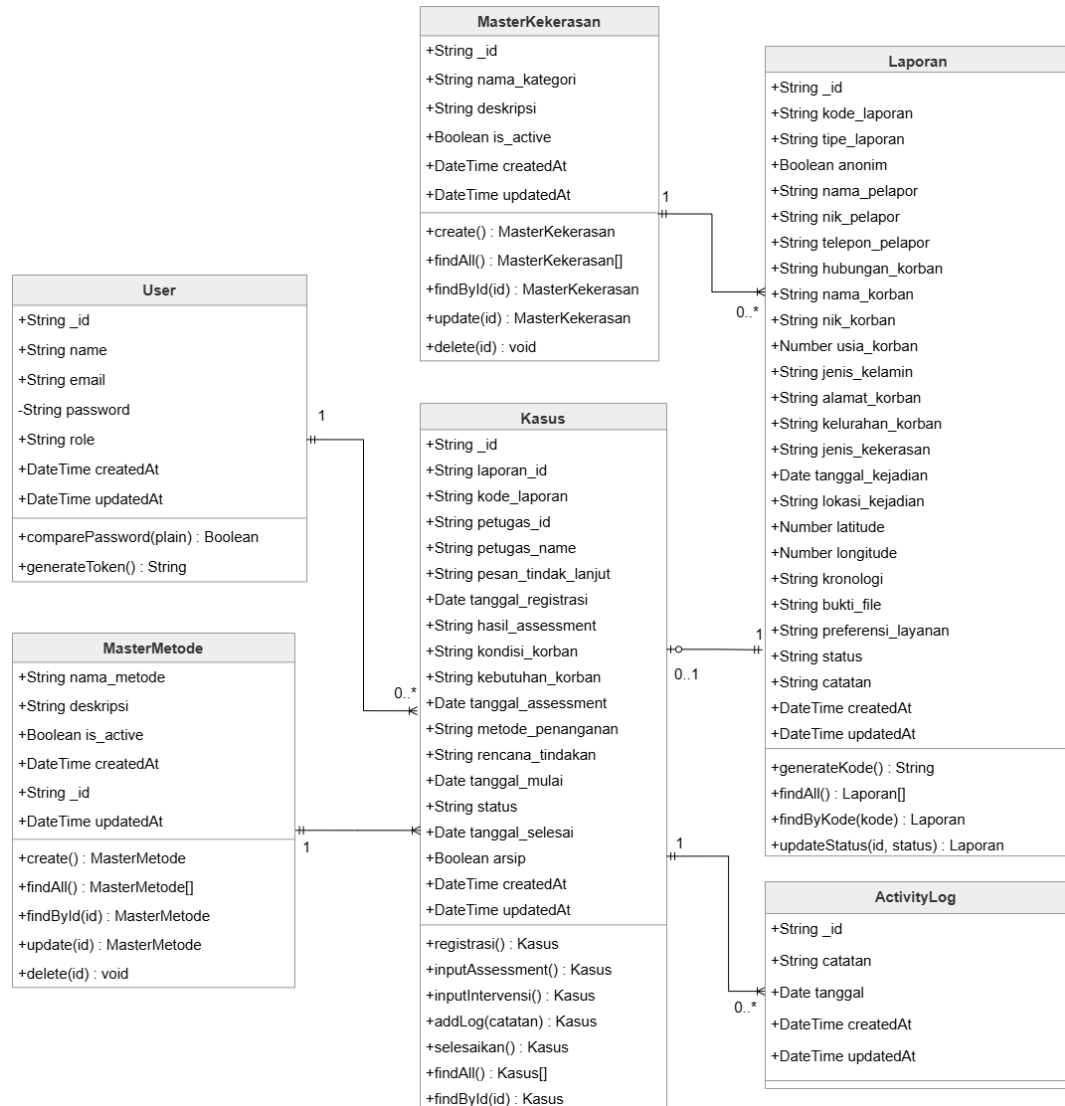
Aktor pertama adalah pelapor, yaitu masyarakat umum yang tidak memerlukan proses autentikasi untuk mengakses sistem. Pelapor memiliki dua *use case* utama. Pertama, buat laporan kasus, di mana pelapor mengisi dan mengirimkan formulir laporan kasus kekerasan melalui sistem. *Use case* ini

memiliki relasi «*include*» terhadap *use case* upload bukti, yang berarti setiap pembuatan laporan kasus selalu menyertakan proses pengunggahan bukti pendukung sebagai bagian yang tidak terpisahkan. Kedua, cek status laporan, di mana pelapor dapat memantau perkembangan penanganan laporan yang telah dikirimkan. *Use case* ini memiliki relasi «*include*» terhadap *use case* dapatkan kode laporan, yang berarti fitur mendapatkan kode laporan bersifat wajib

Aktor kedua adalah *admin* petugas, yaitu *admin* petugas yang memiliki hak akses operasional terhadap penanganan kasus. *Admin* petugas diwajibkan melakukan *login* terlebih dahulu sebelum dapat mengakses seluruh fitur sistem. Setelah berhasil masuk, *admin* petugas dapat mengakses lima *use case*, yaitu lihat laporan keseluruhan untuk memantau seluruh laporan yang masuk ke dalam sistem, registrasi laporan untuk melakukan registrasi terhadap laporan yang menunggu tindak lanjut, input *assessment* untuk mengisi data *assessment* terhadap kasus yang telah diregistrasi, kelola kasus dalam penanganan untuk memantau dan mengelola kasus yang sedang dalam penanganan aktif, serta lihat arsip kasus untuk mengakses data kasus yang telah selesai ditangani. Terdapat relasi «*extend*» antara *use case* selesaikan kasus terhadap *use case* kelola kasus dalam penanganan, yang berarti fitur penyelesaian kasus merupakan ekstensi opsional yang hanya dapat dijalankan apabila kondisi penanganan kasus telah memenuhi syarat untuk diselesaikan.

Aktor ketiga adalah *super admin*, yaitu pengguna dengan hak akses tertinggi yang bertanggung jawab atas administrasi dan pengawasan sistem secara menyeluruh. *Super admin* diwajibkan melakukan *login* terlebih dahulu sebelum mengakses seluruh fitur administrasi. Setelah berhasil masuk, *super admin* dapat mengakses lima *use case*, yaitu lihat *dashboard* untuk memantau statistik dan aktivitas sistem secara agregat, kelola akun petugas untuk mengelola data akun *admin* petugas yang terdaftar dalam sistem, kelola kategori kekerasan untuk mengonfigurasi daftar kategori jenis kekerasan yang tersedia, kelola metode penanganan untuk mengelola daftar metode penanganan yang aktif dalam sistem, serta ekspor data laporan untuk mengunduh rekapitulasi data laporan ke dalam format *Excel* berdasarkan filter tanggal dan wilayah yang ditentukan.

2. Class Diagram



Gambar 4. 4 Class Diagram

Class Diagram pada Gambar 4.4 menggambarkan struktur statis sistem serta hubungan antarkelas yang terlibat dalam proses pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak. Terdapat enam kelas utama yang diidentifikasi, yaitu *User*, *Laporan*, *Kasus*, *ActivityLog*, *MasterKekerasan*, dan *MasterMetode*.

Class User merepresentasikan entitas pengguna yang memiliki hak akses khusus ke dalam sistem, yakni *Admin* Petugas dan *Super admin*. Kelas *Laporan* berfungsi untuk menyimpan seluruh entri aduan awal yang diinputkan oleh

masyarakat (pelapor) dari portal publik. Laporan yang dinyatakan valid akan direlasikan ke kelas Kasus, yang menjadi pusat manajemen operasional (seperti pencatatan *assessment* dan rencana intervensi) oleh *Admin* Petugas.

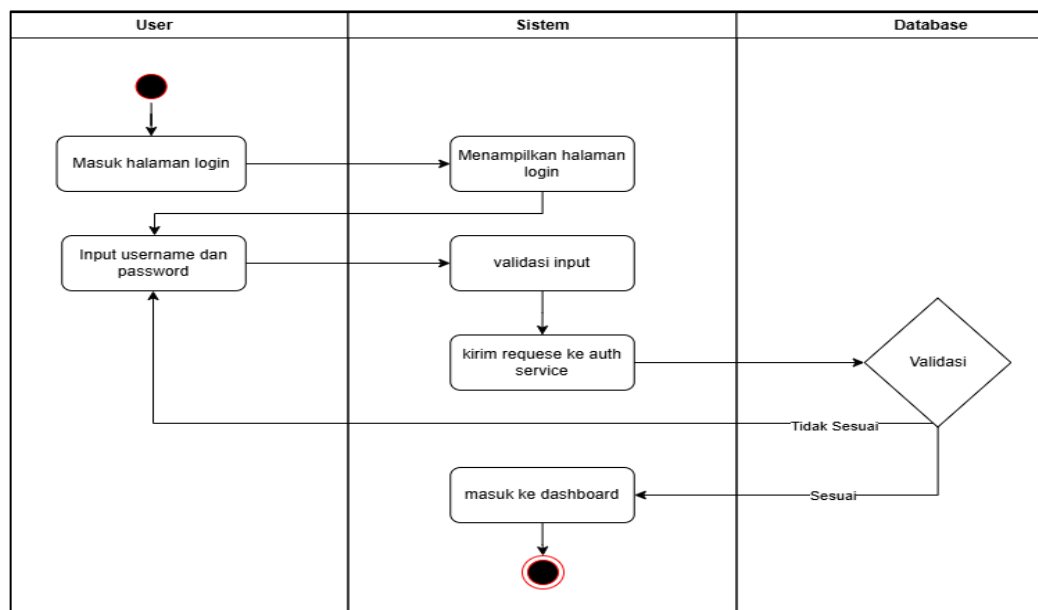
Untuk merekam setiap jejak layanan pendampingan, sistem menggunakan kelas *ActivityLog* yang berelasi dengan kelas Kasus. Sementara itu, kelas *MasterKekerasan* dan *MasterMetode* merupakan tabel data referensi (data master) yang dikelola secara eksklusif oleh *Super admin* untuk menentukan kategori kekerasan dan pilihan layanan penanganan yang dapat digunakan oleh petugas.

3. Activity Diagram

Activity Diagram adalah *diagram* yang menjelaskan alur kerja dalam sistem. Setelah melakukan analisis sistem, didapatkan *Activity* sebagai berikut:

a. Activity Diagram Login

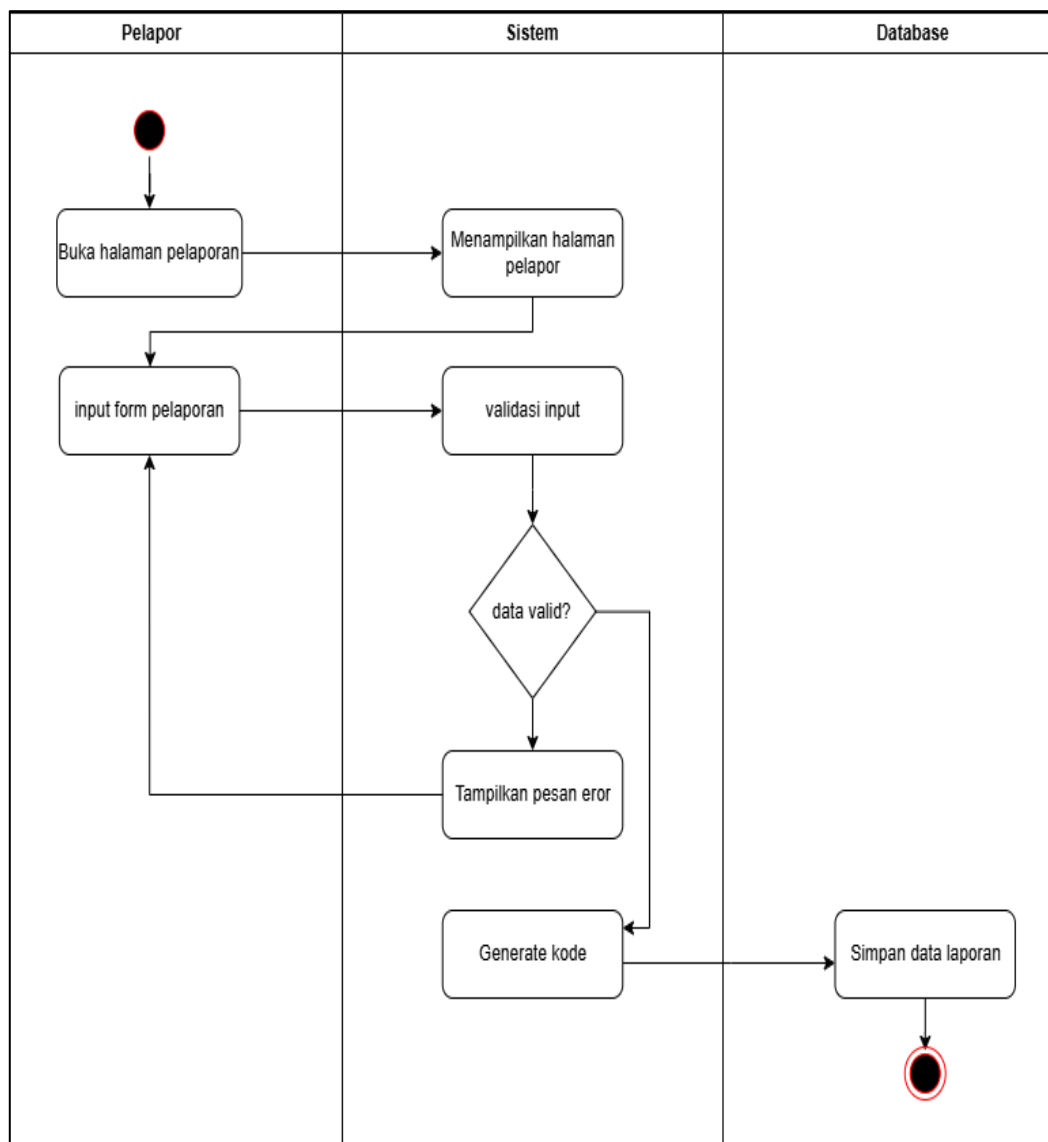
Activity Diagram ini menunjukkan aktivitas yang dilakukan pengguna (*admin* petugas atau *super admin*) untuk melakukan proses autentikasi. Pengguna mengakses halaman *login*, lalu memasukan *email* dan *password*. Apabila kredensial tidak sesuai, sistem akan memunculkan pesan kesalahan. Jika sesuai, sistem akan menerbitkan *token* dan mengarahkan pengguna ke *dashboard* masing-masing sesuai dengan *role* (hak akses) mereka.pengguna.



Gambar 4. 5 Activity Login

b. *Activity Diagram* Buat Laporan

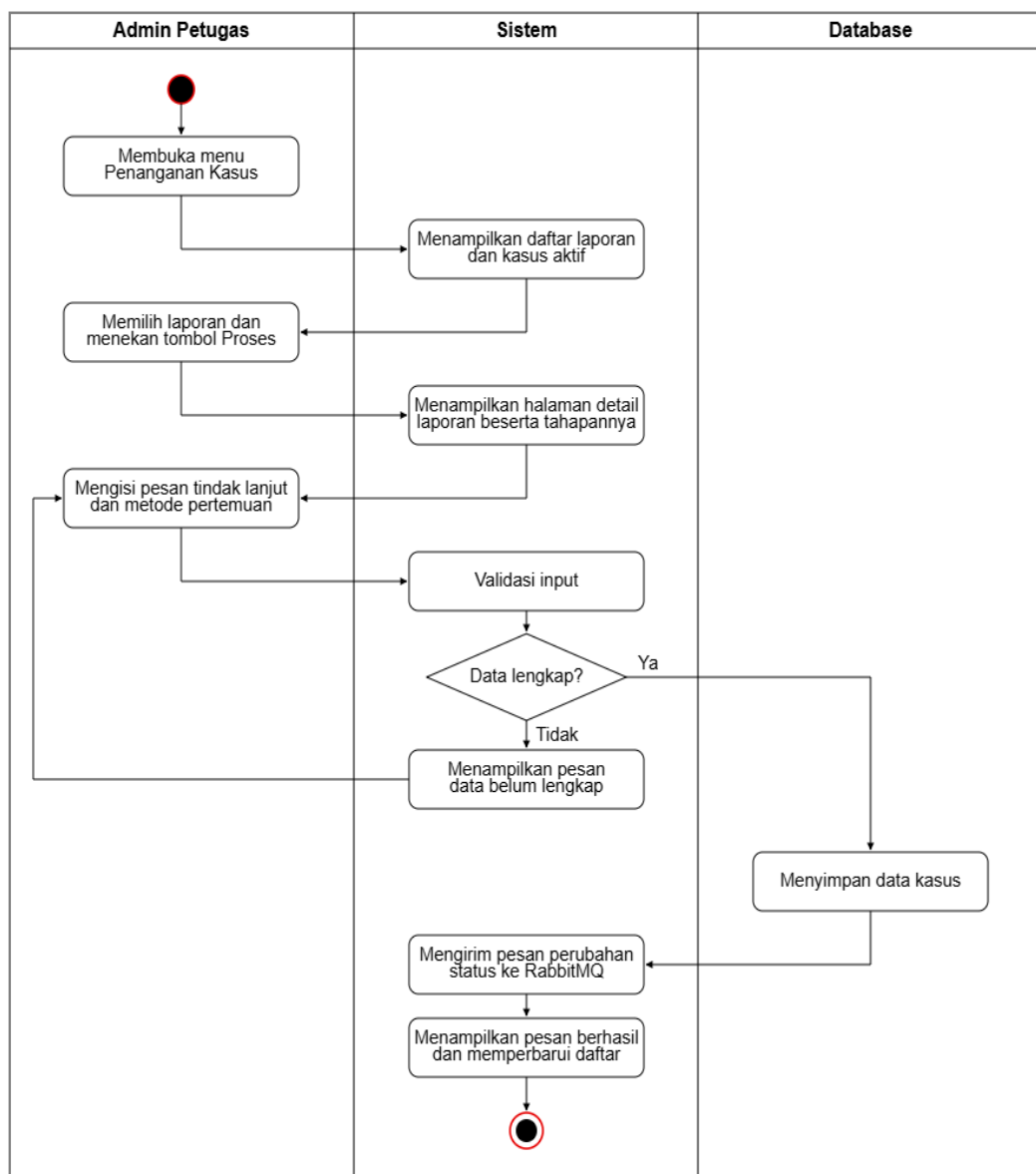
Activity diagram buat laporan dapat dilihat pada gambar 4.6 *Diagram* ini menunjukkan alur aktivitas pengguna (pelapor) saat membuat sebuah laporan baru. Proses dimulai dari pelapor mengakses halaman pelaporan, lalu menginputkan data ke dalam *form* yang tersedia. Selanjutnya, sistem akan memvalidasi data masukan tersebut. Jika data valid, sistem akan menghasilkan kode laporan secara otomatis dan menyimpannya ke dalam *database*. Namun, jika data tidak valid, sistem akan menampilkan pesan error dan meminta pelapor untuk memperbaiki inputannya kembali.



Gambar 4. 6 Activity Buat Laporan

c. *Activity Diagram* Registrasi Laporan

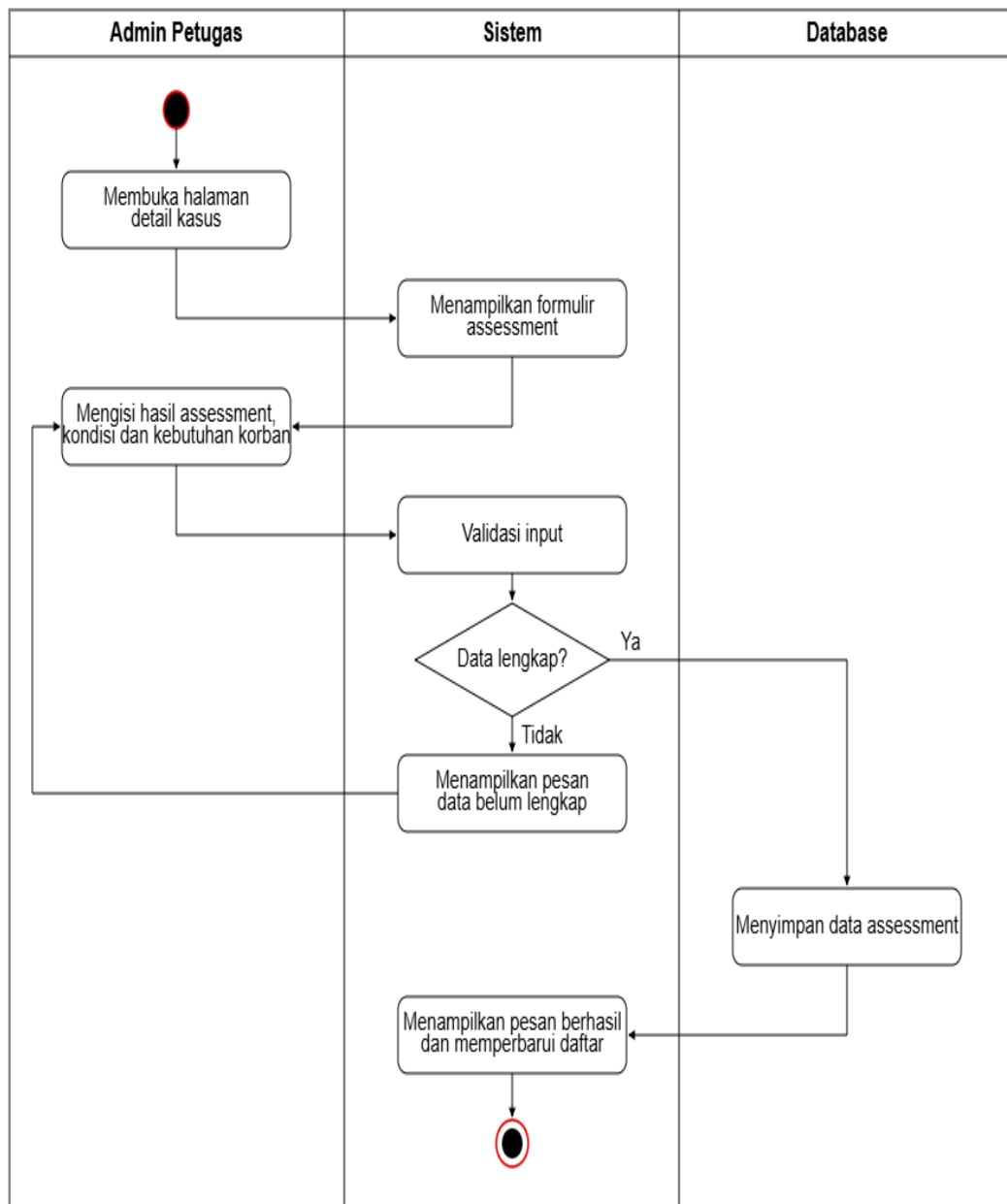
Activity diagram registrasi laporan dapat dilihat pada Gambar 4.7. Petugas membuka menu Penanganan Kasus, memilih laporan yang menunggu registrasi, lalu mengisi pesan tindak lanjut dan metode pertemuan pada halaman detail yang ditampilkan sistem. Setelah validasi berhasil, *Case service* menyimpan data kasus dan mempublikasikan pesan perubahan status ke RabbitMQ agar *Report service* dapat menyinkronkan status laporan, kemudian sistem menampilkan pesan berhasil.



Gambar 4. 7 Activity Registrasi Laporan

d. *Activity Diagram Input Assessment*

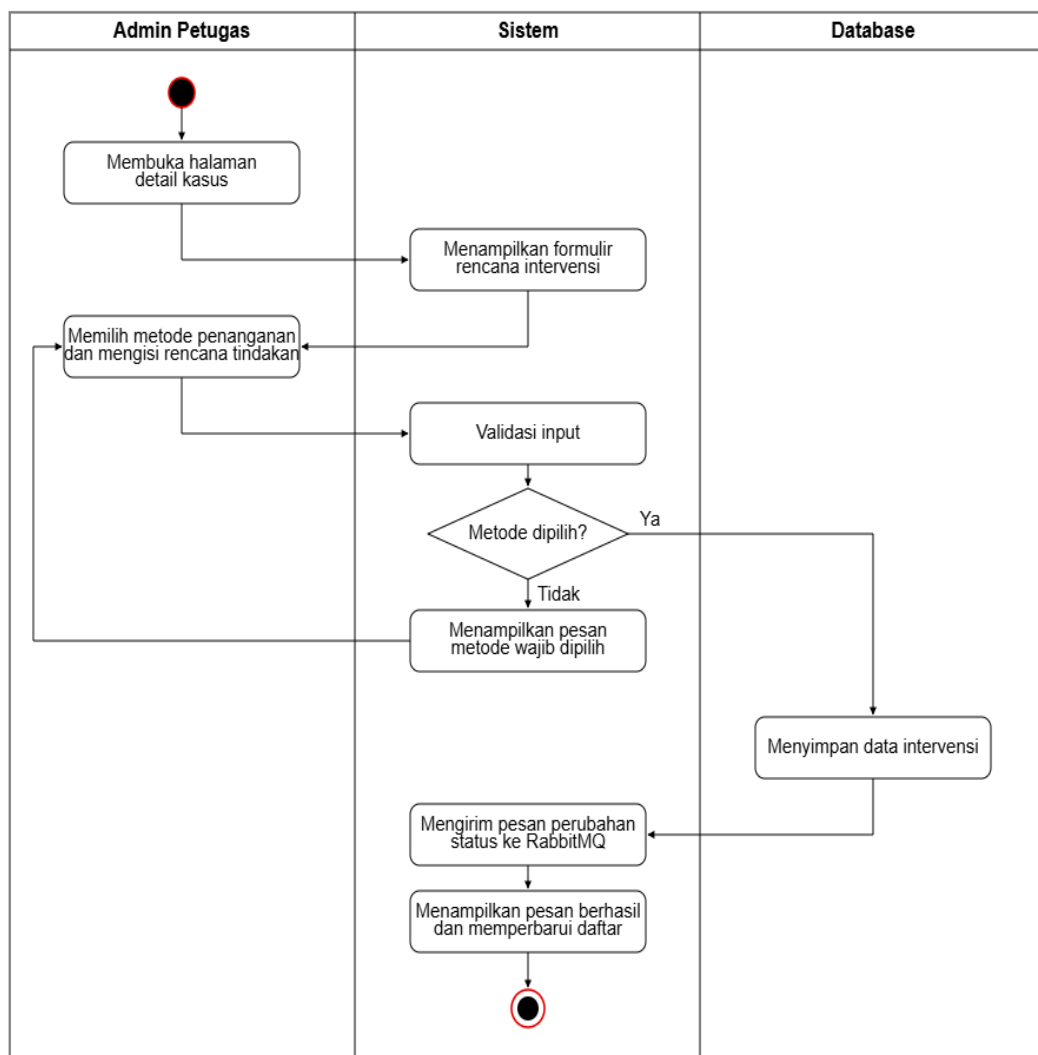
Activity diagram input assessment dapat dilihat pada Gambar 4.8. Petugas membuka halaman detail kasus yang telah diregistrasi, kemudian mengisi hasil assessment, kondisi korban, dan kebutuhan korban pada formulir yang ditampilkan sistem. Setelah validasi berhasil, data disimpan langsung ke basis data *Case service* tanpa mempublikasikan pesan ke RabbitMQ, karena tahap ini tidak mengubah status laporan yang ditampilkan kepada pelapor.



Gambar 4. 8 Activity Input Aessment

e. *Activity Diagram Rencana Intervensi*

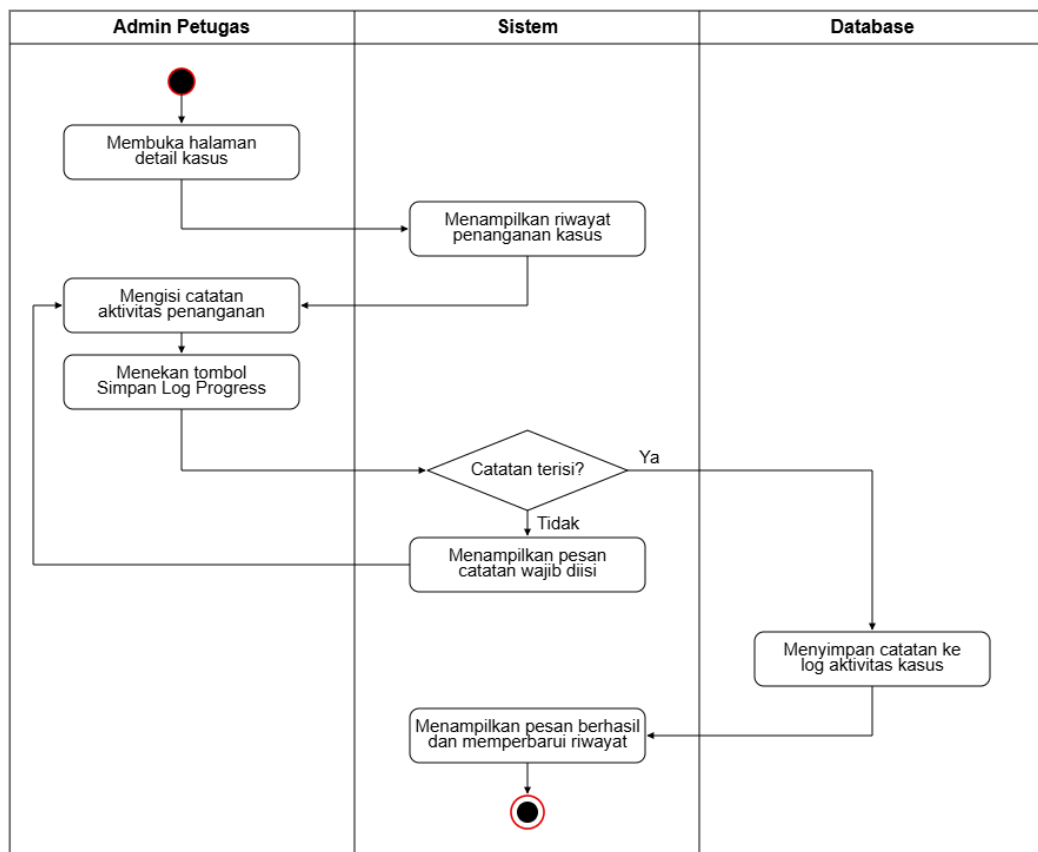
Activity diagram rencana intervensi dapat dilihat pada Gambar 4.9. Petugas membuka halaman detail kasus yang telah melalui tahap *assessment*, memilih metode penanganan, dan mengisi rencana tindakan. Setelah validasi berhasil, *Case service* menyimpan data intervensi dan mempublikasikan pesan perubahan status ke RabbitMQ agar status laporan tersinkronisasi menjadi dalam penanganan.



Gambar 4. 9 Activity Renca Intervensi

f. *Activity Diagram* Kelola Kasus dalam Penanganan

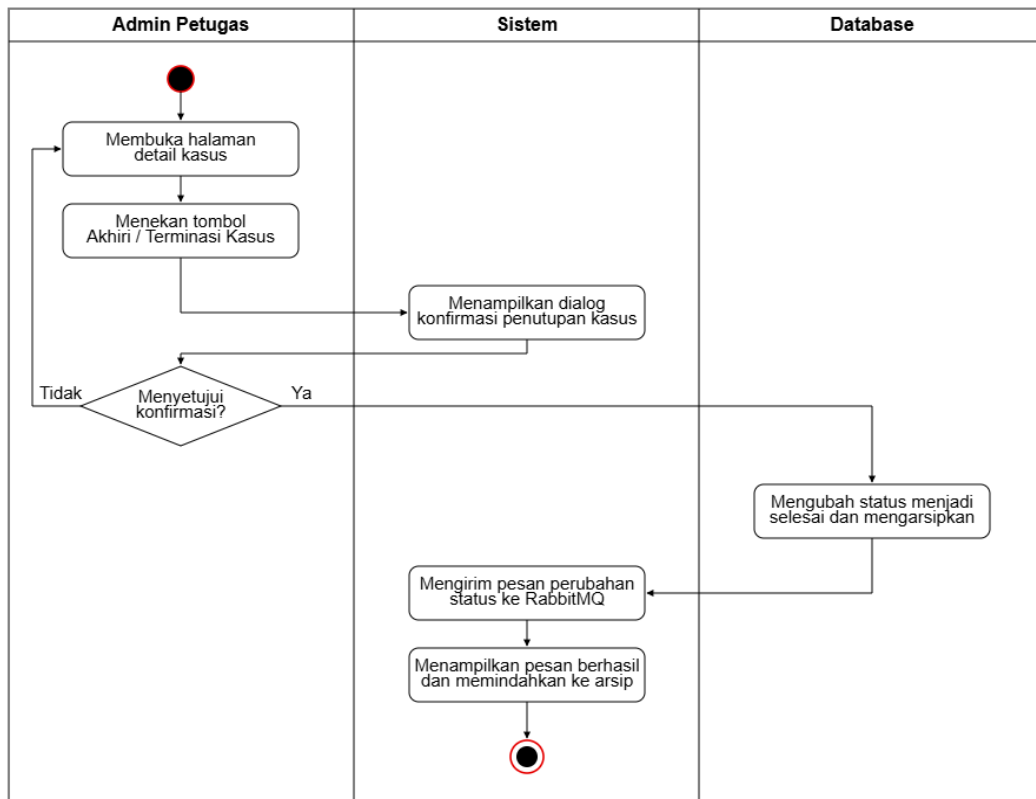
Activity diagram kelola kasus dalam penanganan dapat dilihat pada Gambar 4.10. Petugas membuka halaman detail kasus dan mengisi catatan aktivitas penanganan pada riwayat yang ditampilkan sistem. Setelah validasi berhasil, sistem menambahkan entri baru ke dalam log aktivitas kasus tanpa mengubah status utama kasus, sehingga tahap ini tidak mempublikasikan pesan ke RabbitMQ.



Gambar 4. 10 Activity Kelola Kasus

g. *Activity Diagram* Selesaikan Kasus

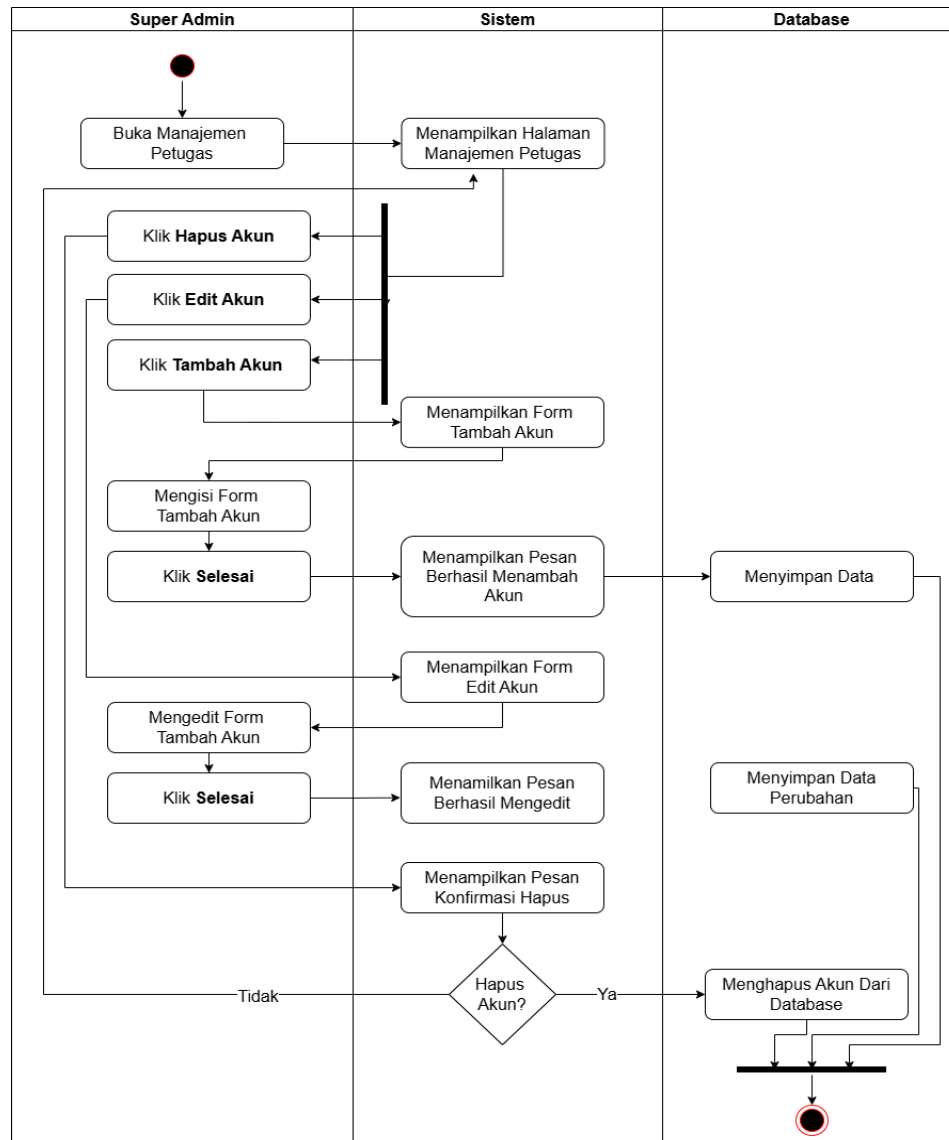
Activity diagram selesaikan kasus dapat dilihat pada Gambar 4.11. Petugas membuka halaman detail kasus dan menekan tombol Akhiri/Terminasi Kasus, kemudian menyetujui dialog konfirmasi yang ditampilkan sistem. Setelah disetujui, *Case service* mengubah status kasus menjadi selesai dan mengarsipkannya, lalu mempublikasikan pesan perubahan status ke RabbitMQ agar status laporan tersinkronisasi menjadi selesai.



Gambar 4. 11 Activity Selesaikan Kasus

h. *Activity Diagram Menu Manajemen Petugas*

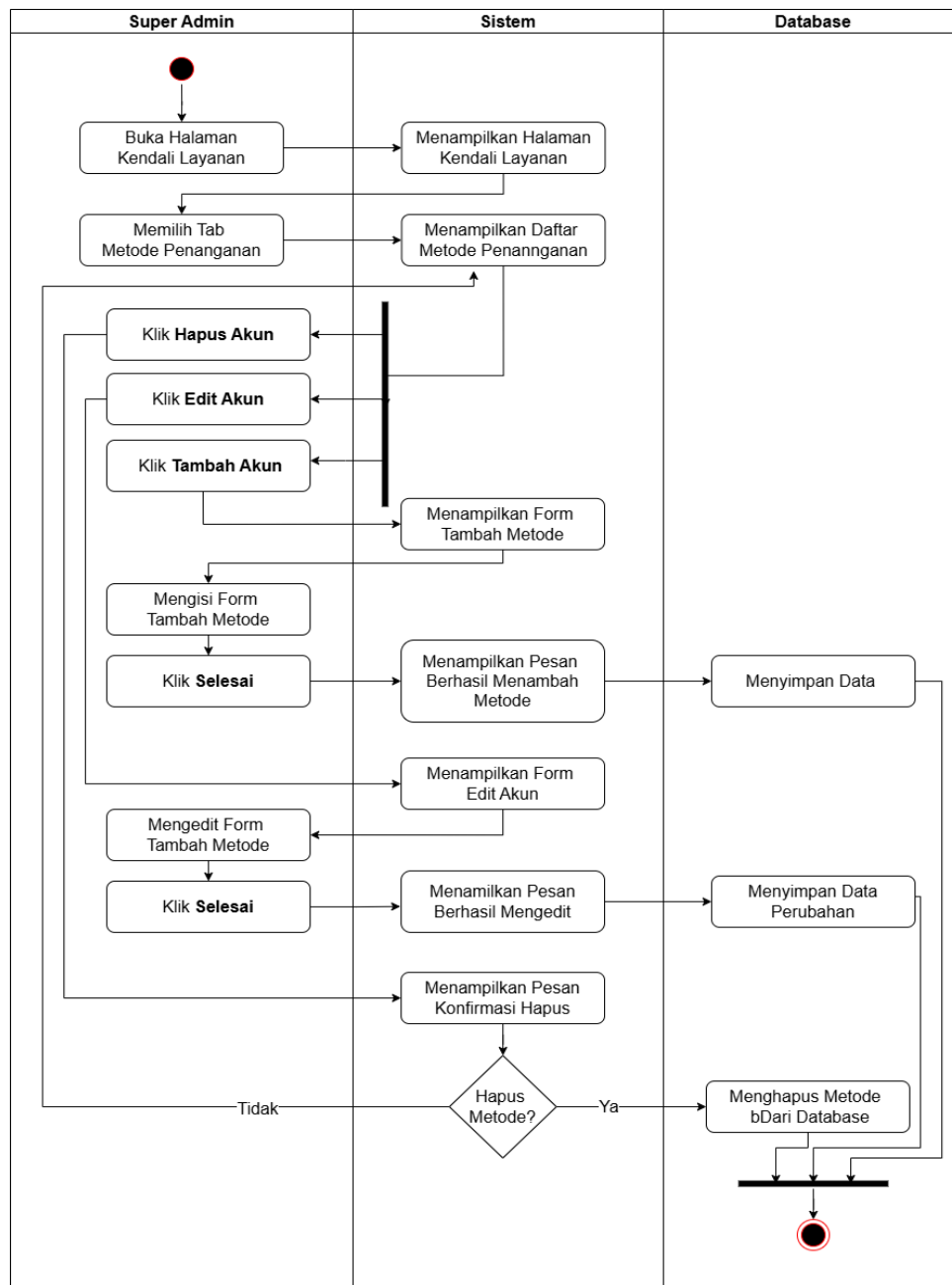
Activity diagram menu manajemen petugas menggambarkan alur aktivitas *super admin* dalam mengelola data akun petugas di dalam sistem. Proses diawali saat *super admin* mengakses menu manajemen petugas, kemudian sistem menampilkan daftar seluruh akun petugas yang terdaftar. *Super admin* dapat menambahkan akun petugas baru dengan mengisi formulir data yang tersedia, memperbarui informasi akun petugas yang sudah ada, atau menonaktifkan akun petugas yang tidak lagi aktif bertugas. Setiap perubahan yang dilakukan akan langsung tersimpan ke dalam database dan tercermin pada daftar petugas yang ditampilkan sistem.



Gambar 4. 12 Activity Menu Manajemen Petugas

i. *Activity Diagram Kelola Metode Penanganan*

Activity diagram kelola metode penanganan menggambarkan alur aktivitas *super admin* dalam mengelola data metode penanganan yang tersedia dalam sistem. Proses diawali ketika *super admin* mengakses halaman pusat kendali layanan dan memilih sub-modul metode penanganan. Sistem menampilkan daftar metode penanganan yang terdaftar beserta status aktif masing-masing. *Super admin* dapat menambahkan metode penanganan baru, mengubah data metode yang telah ada, atau menghapus metode yang tidak lagi diperlukan.

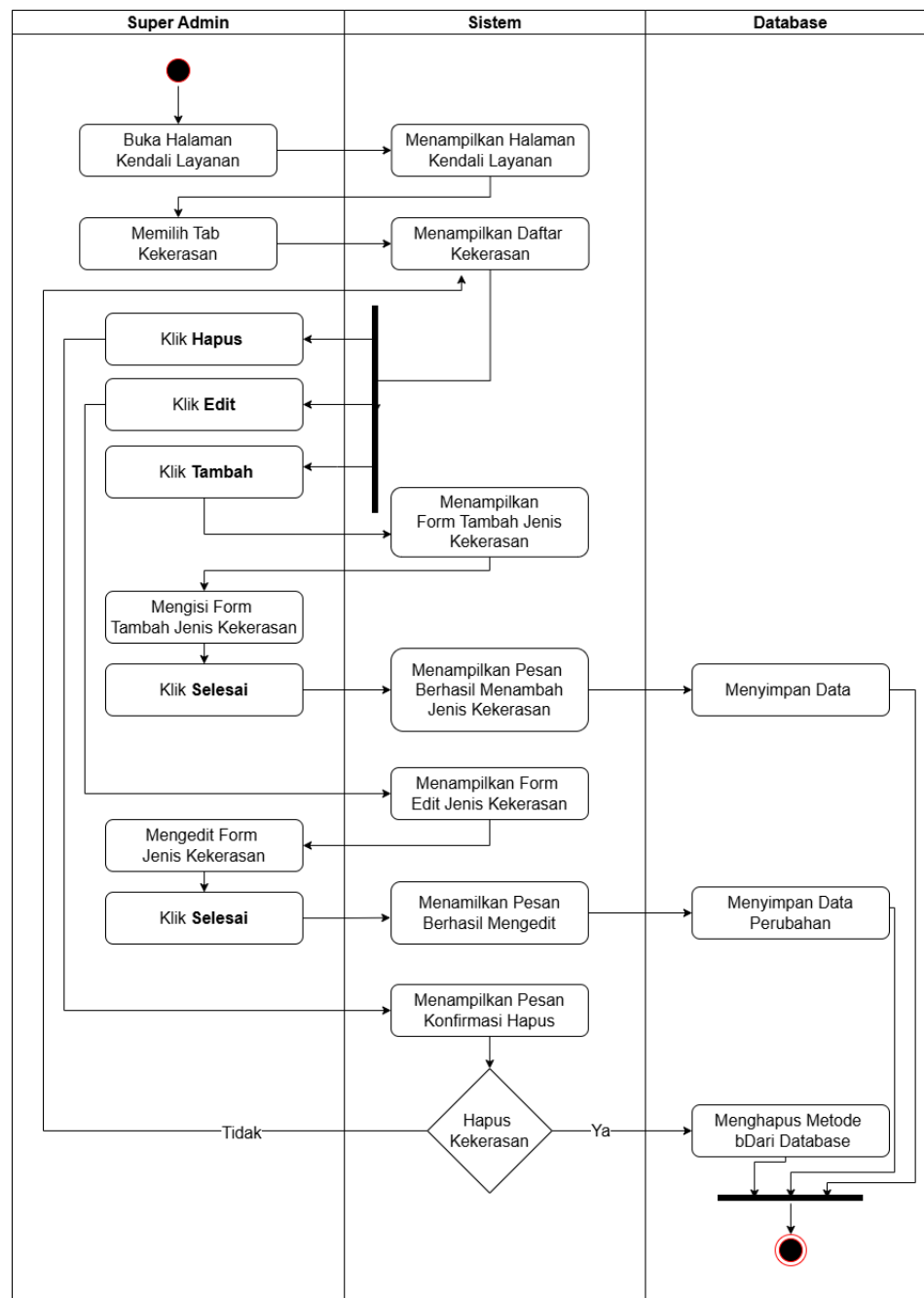


Gambar 4. 13 Activity Kelola Metode Penanganan

j. *Activity Diagram Kelola Jenis Kekerasan*

Activity diagram kelola jenis kekerasan menggambarkan alur aktivitas *super admin* dalam mengelola data kategori kekerasan yang tersedia dalam sistem. Proses diawali ketika *super admin* mengakses halaman pusat kendali layanan dan memilih sub-modul kategori kekerasan. Sistem menampilkan daftar kategori kekerasan yang telah terdaftar. *Super admin* dapat menambahkan kategori kekerasan baru,

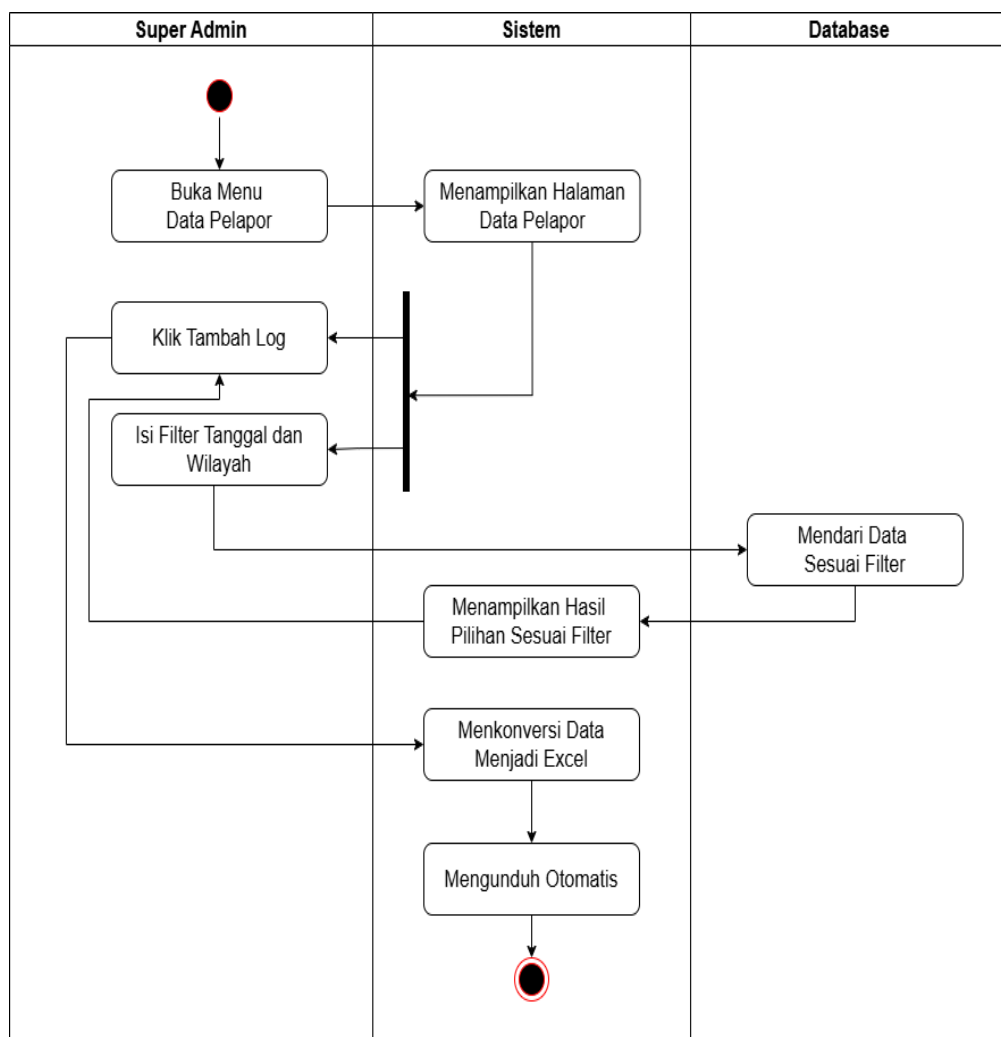
mengubah data kategori yang telah ada, atau menghapus kategori yang tidak lagi diperlukan. Setiap perubahan yang dilakukan akan divalidasi oleh sistem sebelum disimpan ke dalam basis data, dan daftar kategori kekerasan akan diperbarui secara otomatis pada seluruh antarmuka sistem.



Gambar 4. 14 Activity Kelola Jenis Kekerasan

k. *Activity Diagram* Ekspor Data Laporan

Activity diagram ekspor data laporan menggambarkan alur aktivitas *super admin* dalam mengelola dan mengekspor data laporan yang tersimpan di dalam sistem. Proses diawali ketika *super admin* mengakses menu data pelaporan, lalu sistem menampilkan seluruh data laporan yang telah masuk. *Super admin* dapat melakukan pemfilteran data berdasarkan rentang tanggal kejadian dan wilayah atau kelurahan tertentu untuk memperoleh rekapitulasi data yang spesifik. Setelah filter diterapkan, sistem menampilkan data hasil filter secara dinamis. *Super admin* selanjutnya dapat mengekspor seluruh data yang ditampilkan ke dalam format Excel melalui tombol ekspor data yang tersedia, untuk keperluan pelaporan dan evaluasi kebijakan.



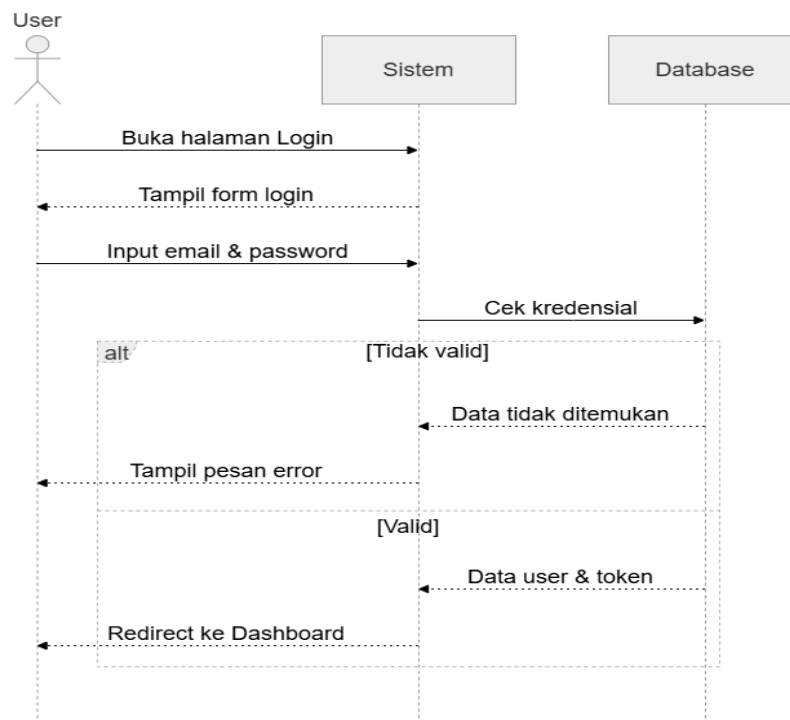
Gambar 4. 15 *Activity Export Data Laporan*

4. *Sequence Diagram*

Sequence diagram adalah *diagram* yang menjelaskan interaksi dan pertukaran pesan antar objek di dalam sistem berdasarkan urutan waktu. Setelah melakukan analisis sistem, didapatkan *Sequence Diagram* sebagai berikut:

a. *Sequence Diagram Login*

Sequence diagram login (masuk) dapat dilihat pada Gambar 3.15. *Diagram* ini menggambarkan alur autentikasi pengguna untuk dapat mengakses sistem. Proses dimulai saat pengguna memasukkan *email* dan kata sandi pada halaman antarmuka sistem. Selanjutnya, data kredensial tersebut akan dikirimkan ke layanan autentikasi untuk dicocokkan dan diverifikasi dengan data yang tersimpan di dalam database. Setelah verifikasi berhasil dan sistem mengeluarkan token akses, pengguna akan langsung diarahkan untuk masuk ke halaman *dashboard* utama.

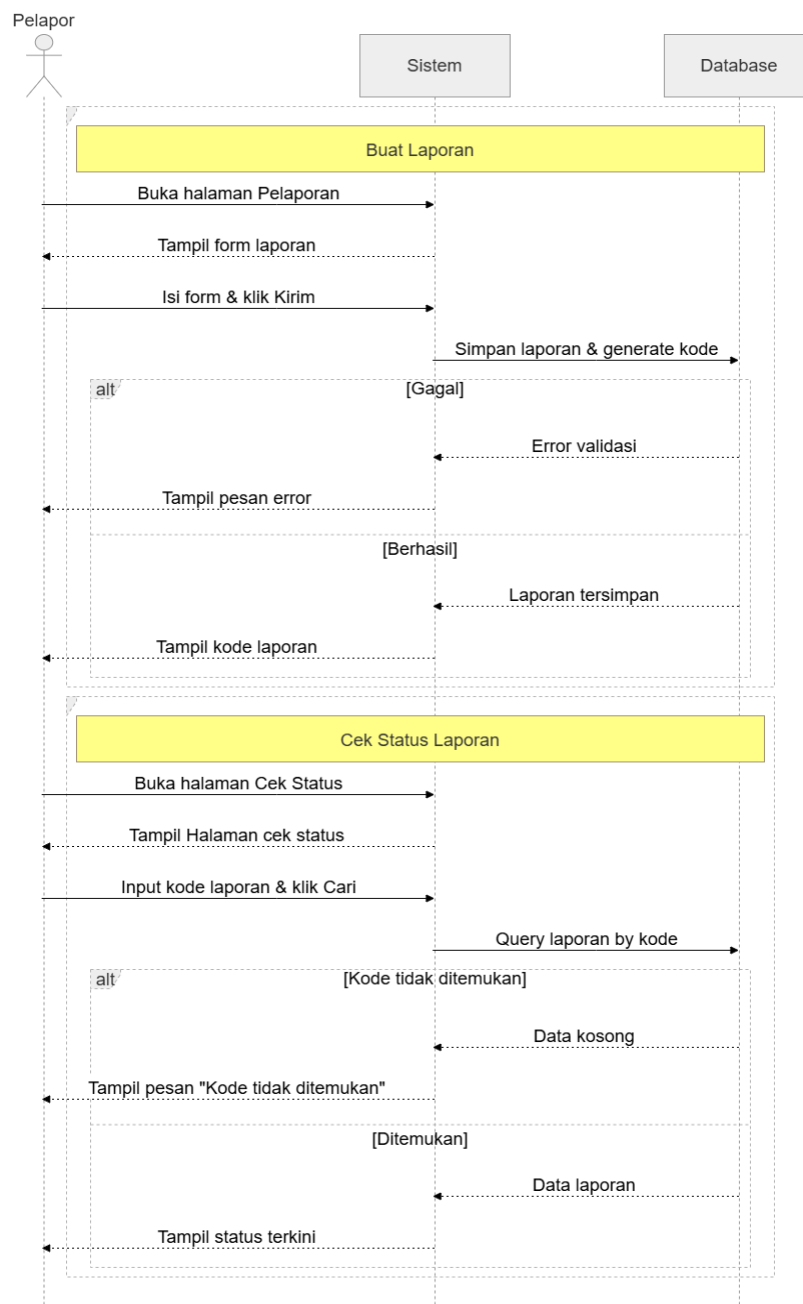


Gambar 4. 16 *Sequence Login*

b. *Sequence Diagram input dan cek laporan*

Sequence diagram input dan cek laporan dapat dilihat pada Gambar 4.17. *Sequence diagram* ini menunjukkan Pelapor melakukan interaksi dengan sistem untuk membuat laporan dan mengecek status. Proses pembuatan laporan dimulai dari pelapor mengisi *form* pada halaman laporan, kemudian sistem akan memproses

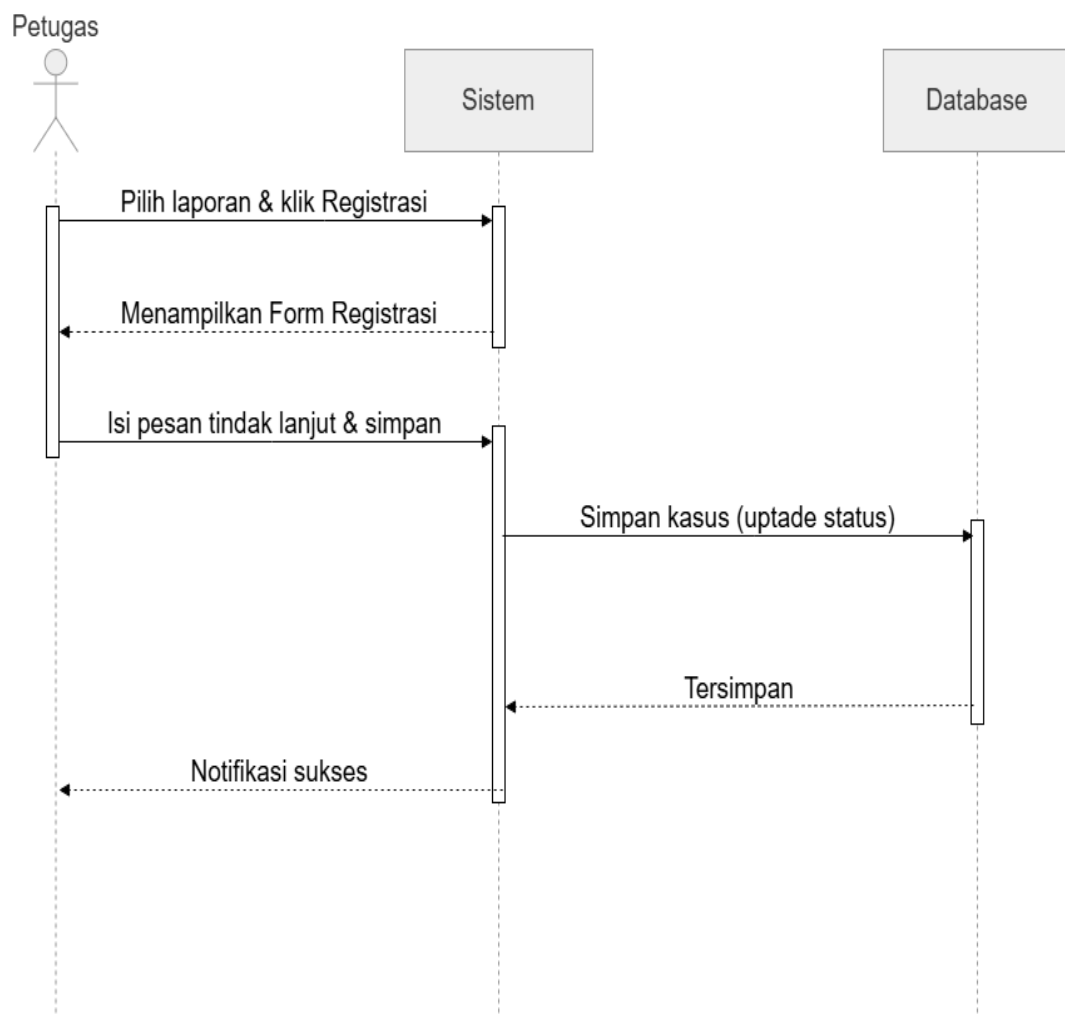
dan menyimpannya ke dalam database. Jika penyimpanan berhasil, sistem akan langsung menampilkan kode laporan kepada pelapor. Selanjutnya, untuk proses cek status, pelapor cukup memasukkan kode laporan yang telah didapat, kemudian sistem akan meminta dan mencocokkan data tersebut ke database. Setelah data sesuai dan ditemukan, sistem akan menampilkan status laporan terkini kepada pelapor.



Gambar 4. 17 Sequence Input dan Cek Lapora

c. *Sequence Diagram* Registrasi Laporan

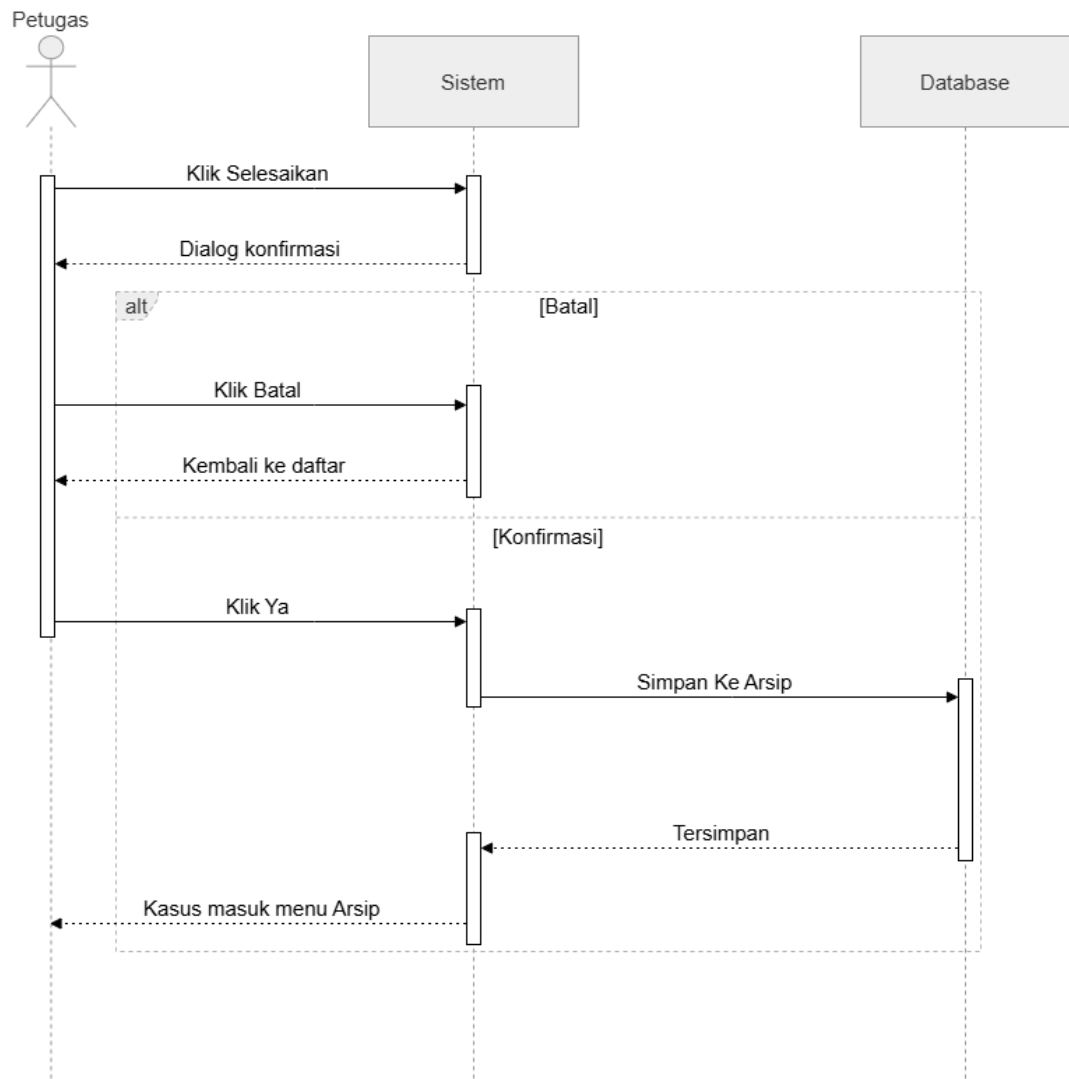
Sequence diagram registrasi laporan dapat dilihat pada Gambar 4.18. *Diagram* ini menunjukkan interaksi *admin* petugas saat memproses laporan baru. Proses diawali saat petugas meminta sistem menampilkan form registrasi untuk laporan tertentu. Petugas kemudian menginputkan pesan tindak lanjut dan menyimpannya. Sistem (*Case service*) memproses instruksi tersebut, mengubah status laporan, menyimpannya ke *database*, dan memberikan notifikasi sukses ke antarmuka petugas.



Gambar 4. 18 *Sequence Diagram* Registrasi Laporan

d. *Sequence Diagram* Selesaikan Kasus

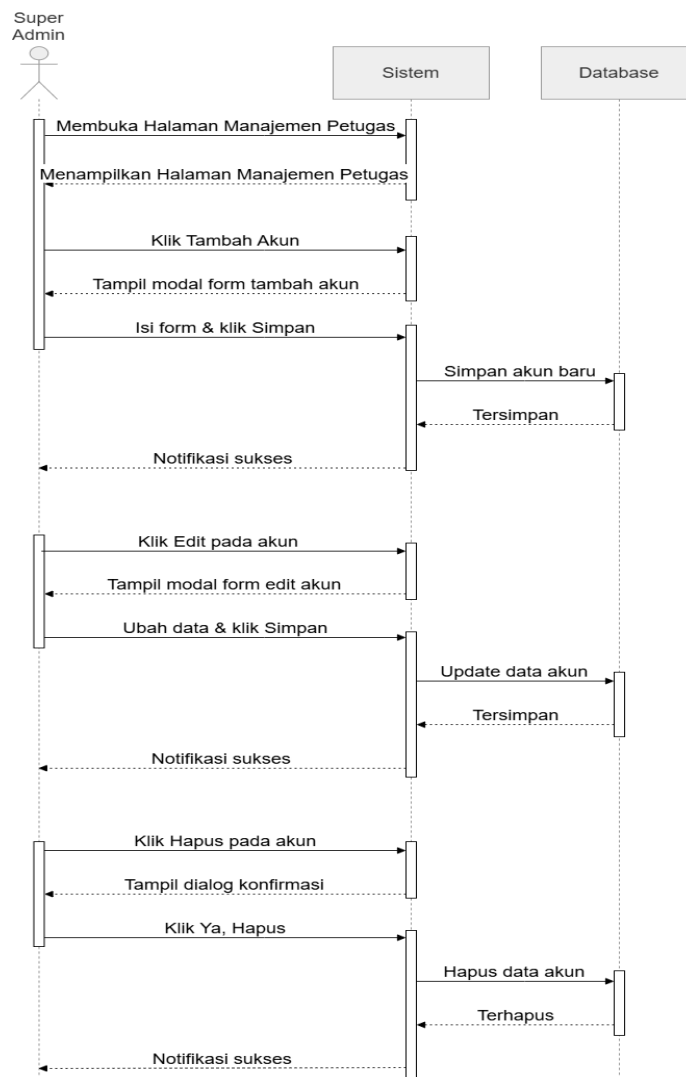
Sequence diagram selesaikan kasus dapat dilihat pada Gambar 4.19. *Diagram* ini menggambarkan alur interaksi *admin* petugas dengan sistem dalam menyelesaikan dan menutup kasus yang telah selesai ditangani. Proses dimulai saat petugas mengakses menu dalam penanganan dan memilih kasus yang telah tuntas ditangani, lalu memilih opsi untuk menyelesaikan kasus. Sistem meminta konfirmasi dari petugas, kemudian mengubah status kasus menjadi selesai dan menyimpan catatan penyelesaian ke dalam basis data. Sistem juga mengarsipkan seluruh riwayat penanganan kasus secara otomatis dan menampilkan notifikasi keberhasilan kepada petugas pada halaman antarmuka.



Gambar 4. 19 *Sequence* Selesaikan Kasus

e. *Sequence Diagram* Manajemen Petugas

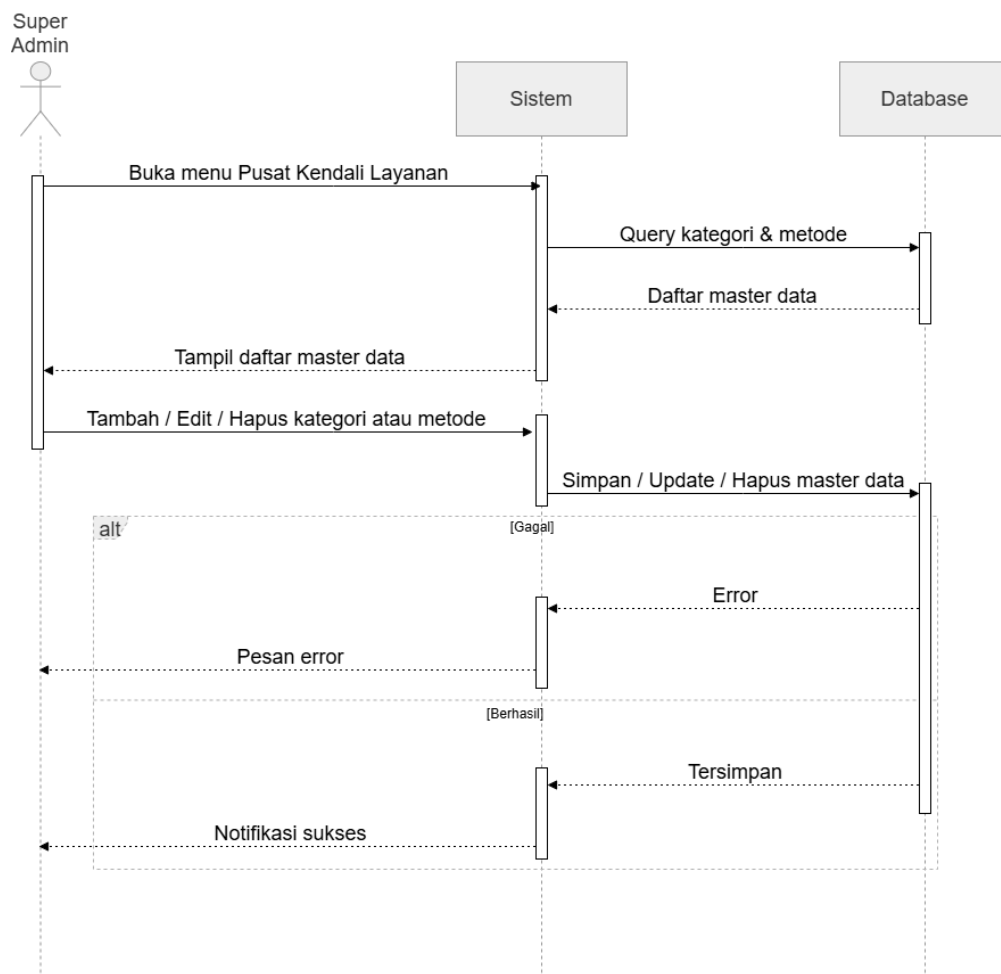
Sequence diagram manajemen petugas dapat dilihat di Gambar 4.20. *Diagram* ini menggambarkan alur interaksi *super admin* sistem dengan layanan *backend* dalam mengelola data akun petugas yang terdaftar. *Diagram* ini mencakup tiga alur utama, yaitu penambahan akun petugas baru, pembaruan data petugas yang sudah ada, dan hapus akun petugas. Pada proses penambahan, *super admin* menginputkan data petugas baru yang kemudian disimpan ke dalam database. Pada proses pembaruan, *super admin* mengubah informasi petugas yang ada dan sistem menyimpan perubahan tersebut. Sedangkan pada hapus akun petugas, *super admin* mengkonfirmasi tindakan dan sistem menghapus status akun petugas di database.



Gambar 4. 20 *Sequence* Manajemen Petugas

f. *Sequence Diagram* Pusat Kendali Layanan

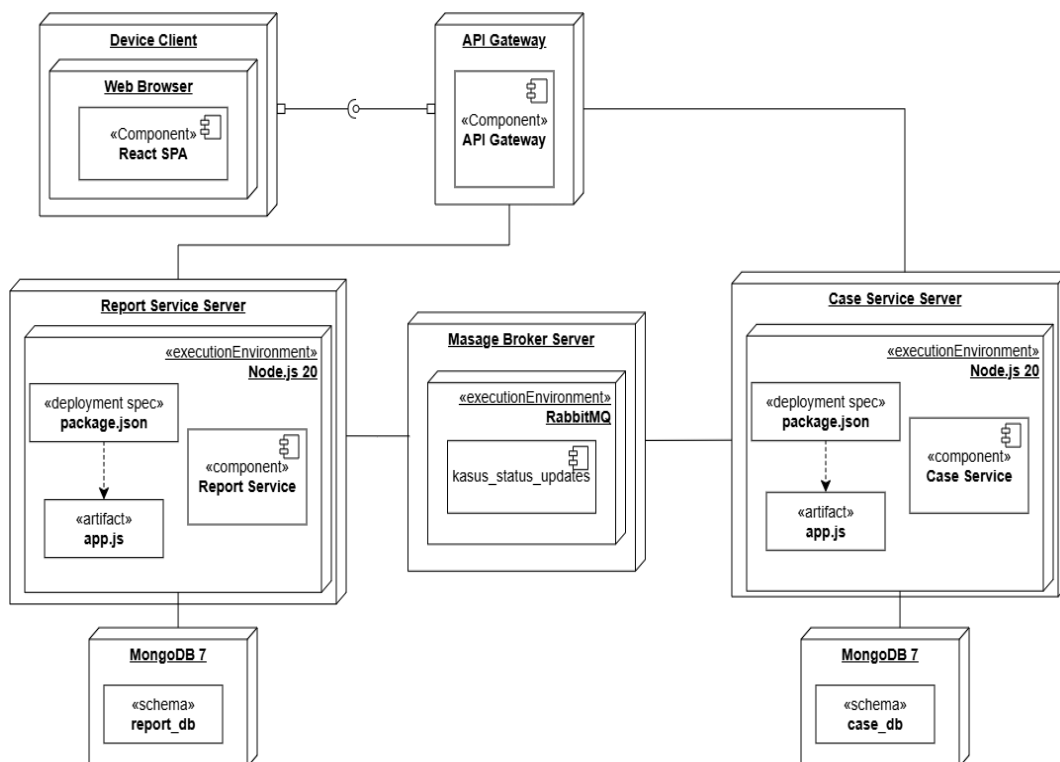
Sequence diagram pusat kendali layanan menggambarkan alur interaksi *super admin* dengan sistem dalam memantau dan mengendalikan seluruh layanan yang sedang berjalan melalui dashboard pusat kendali. Proses dimulai saat petugas mengakses halaman pusat kendali, kemudian sistem menampilkan ringkasan data real-time yang mencakup jumlah laporan masuk, status penanganan kasus, dan statistik layanan terkini. Petugas dapat melakukan pemfilteran dan pencarian data berdasarkan kriteria tertentu, serta mengekspor laporan rekapitulasi ke dalam format yang dapat diunduh. Setiap permintaan data diteruskan oleh sistem ke layanan statistik, yang kemudian mengembalikan hasil pengolahan data kepada antarmuka pengguna secara dinamis.



Gambar 4. 21 *Sequence* Pusat Kendali Layanan

5. Deployment Diagram Arsitektur Microservices

Deployment diagram menggambarkan arsitektur fisik dari sistem yang dikembangkan, meliputi bagaimana setiap komponen sistem didistribusikan dalam lingkungan *container* Docker. *Arsitektur microservices* pada sistem ini terdiri dari beberapa komponen utama yang berjalan secara independen dalam *container* terpisah dan saling berkomunikasi melalui jaringan internal Docker (*bridge network*).



Gambar 4. 22 Deployment Diagram arsitektur microservices

4.3.3 Perancangan Antarmuka (User Interface)

Perancangan antarmuka (*user interface*) merupakan suatu tahapan sistematis yang melibatkan proses perencanaan, pengembangan, dan pengaturan komponen visual serta fungsional dalam sebuah sistem. Proses ini dilakukan dengan tujuan untuk memastikan keterpaduan antara aspek estetika, fungsionalitas, dan kemudahan penggunaan (*usability*) bagi pengguna. Melalui perancangan yang baik, diharapkan interaksi antara pengguna dengan sistem dapat berjalan secara intuitif, efisien, dan tepat sasaran.

Perancangan antarmuka sistem dibagi berdasarkan tiga kelompok pengguna, yaitu pelapor, *admin* petugas, dan *super admin*. Berikut adalah penjelasan perancangan antarmuka dari masing-masing kelompok pengguna.

1. Antarmuka Pelapor

Antarmuka pelapor dirancang untuk diakses secara publik tanpa memerlukan proses autentikasi. Tampilan dibuat sesederhana mungkin agar masyarakat umum dapat menggunakannya secara mandiri tanpa pelatihan khusus.

a. Halaman Pelaporan

SIPEKA — PORTAL PELAPORAN PUBLIK LOGIN ADMIN

1. BUAT LAPORAN **2. CEK STATUS**

FORMULIR LAPORAN KEKERASAN
Layanan resmi DP3A Kota Kendari. Privasi anda adalah prioritas kami.

☐ **LAPOR SECARA ANONIM (IDENTITAS XXX)**
Jika dicentang, nama anda tidak akan dicatat dalam database publik.

01. IDENTITAS PELAPOR

NAMA LENGKAP **NIK PELAPOR**
*baru

NO. TELEPON **HUBUNGAN DENGAN KORBAN**
*baru — orang tua / saudara / saksi / diri sendiri

02. DATA KORBAN & KEJADIAN

NAMA KORBAN **USIA** **JENIS KELAMIN**
*baru

KELURAHAN KEJADIAN **JENIS KEKERASAN**
*baru — fisik / psikis / seksual / penelantaran

ALAMAT KEJADIAN **TANGGAL KEJADIAN**
*baru

KRONOLOGI KEJADIAN

UPLOAD FILE BUKTI
 Tarik & lepas atau klik untuk pilih file
Format: jpg, png, pdf, mp4 — maks. 10 Mili. Bisa lebih dari 1 file.
*baru — foto luka, screenshot chat, surat dll. (opsional)

KIRIM LAPORAN SEKARANG

Gambar 4. 23 Halaman Pelaporan

Halaman ini berfungsi sebagai fasilitas utama bagi masyarakat untuk melaporkan tindak kekerasan secara daring. Antarmuka memuat formulir

terstruktur yang mencakup opsi pelaporan secara anonim, pengisian identitas pelapor, rincian data korban, kategori kekerasan, wilayah kejadian, serta kronologi kejadian. Setelah formulir dikirimkan, sistem secara otomatis menghasilkan kode laporan unik yang ditampilkan kepada pelapor sebagai bukti pengiriman sekaligus sarana pelacakan status kasus.

b. Halaman Cek Status Laporan

SIPEKA — PORTAL PELAPORAN PUBLIK LOGIN ADMIN

1. BUAT LAPORAN **2. CEK STATUS**

KODE LAPORAN
LP-2026-289A CARI ULANG

HASIL CEK STATUS
LP-2026-289A • SEDANG DI INTERVENSI

Progress Bar:

- 1. Asesment (Completed)
- 2. Intervensi (Current)
- 3. Layanan Intervensi
- 4. Terminasi

RIWAYAT PENANGANAN

- Sedang Asesment — selesai dd/mm
- Sedang di Intervensi — sekarang
- Sedang Dilakukan Layanan Intervensi — belum
- Terminasi — belum

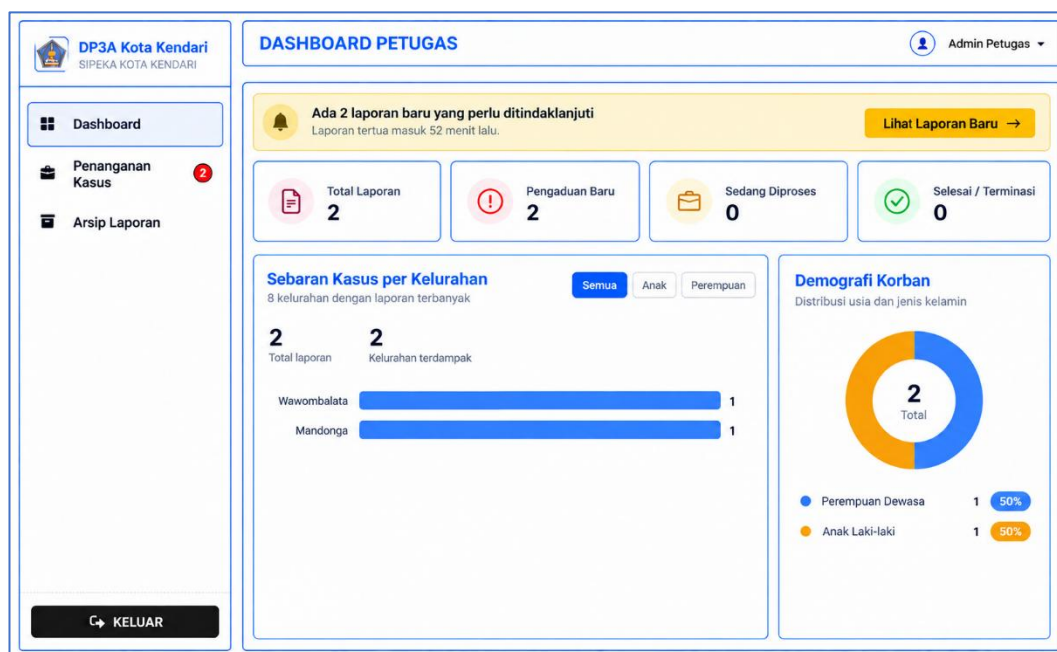
Gambar 4. 24 Halaman Cek Status Laporan

Halaman ini berfungsi sebagai fasilitas pelacakan (*tracking*) bagi masyarakat untuk memantau perkembangan penanganan laporan yang telah dikirimkan. Antarmuka dirancang secara minimalis dan terfokus, dengan menyediakan kolom *input* tempat pelapor memasukkan kode laporan unik yang telah diterima sebelumnya. Setelah kode diverifikasi oleh sistem, status terkini dari laporan ditampilkan secara transparan kepada pelapor, meliputi tahapan penanganan yang sedang berjalan.

2. Antarmuka *Admin* Petugas

Antarmuka *admin* petugas hanya dapat diakses setelah proses autentikasi berhasil dilakukan. Seluruh halaman pada kelompok ini dirancang untuk mendukung alur kerja operasional penanganan kasus secara sistematis dan efisien.

a. Halaman *Dashboard*



Gambar 4. 25 Halaman *Dashboard Admin* Petugas

Halaman beranda berfungsi sebagai pusat informasi utama bagi *admin* petugas. Antarmuka menampilkan empat panel kartu metrik yang merangkum status kasus secara *real-time*, yaitu jumlah laporan yang menunggu registrasi, jumlah kasus yang perlu *assessment*, jumlah kasus yang sedang dalam penanganan aktif, dan jumlah kasus yang telah selesai. Di bagian bawah halaman, sistem menampilkan daftar laporan yang berstatus menunggu registrasi secara eksplisit, memuat informasi kode laporan, nama korban, usia, jenis kelamin, wilayah, dan kategori kekerasan, disertai tombol registrasi dan tombol detail untuk setiap laporan. Melalui halaman ini, petugas dapat memperoleh gambaran situasi terkini secara menyeluruh sebelum memulai aktivitas penanganan.

b. Halaman Penanganan Kasus

ID LAPORAN	PELAPOR (INISIAL)	KORBAN (INISIAL)	KATEGORI	TAHAP SAAT INI	AKSI
LP-2026-6Z5MZ 27 Agu 2026 BARU	A.	F. (P)	Perempuan	Registrasi	Proses
LA-2026-UFPZ 27 Agu 2026 BARU	F.	F. (L)	Anak	Registrasi	Proses

Gambar 4. 26 Halaman Penanganan Kasus

Halaman ini menampilkan rekap seluruh laporan yang telah masuk ke dalam sistem tanpa memandang status penanganan. Data disajikan dalam format tabel yang memuat kolom kode laporan, tanggal melapor, nama dan data demografis korban, wilayah, serta status kasus saat ini. Status kasus ditampilkan dalam bentuk *badge* berwarna yang membedakan antara status proses *assessment*, dalam penanganan, dan selesai, sehingga petugas dapat mengidentifikasi kondisi setiap kasus secara cepat. Setiap baris laporan dilengkapi tombol detail untuk mengakses informasi lengkap terkait kasus yang bersangkutan.

c. Halaman Arsip Laporan

ID LAPORAN	TGL SELESAI	KORBAN	KATEGORI	METODE PENANGANAN	AKSI
LA-2026-1XXSF	28/8/2026	R.	Anak	Psikososial	Detail

Gambar 4. 27 Halaman Arsip Laporan

Halaman ini berfungsi sebagai repositori kasus-kasus yang telah selesai ditangani. Data disajikan dalam format tabel yang memuat kolom kode laporan, tanggal selesai, nama dan data demografis korban, wilayah, serta metode penanganan yang diterapkan. Setiap entri dilengkapi dengan tombol detail untuk mengakses catatan lengkap penanganan kasus. Halaman ini dirancang untuk menjaga transparansi, akuntabilitas, dan keteraturan pendataan kasus, sehingga memudahkan proses evaluasi dan pelaporan di kemudian hari.

3. Antarmuka *Super admin*

Antarmuka *super admin* hanya dapat diakses setelah proses autentikasi berhasil dilakukan. Seluruh halaman pada kelompok ini dirancang untuk mendukung fungsi *administrasi* dan pengawasan sistem secara menyeluruh.

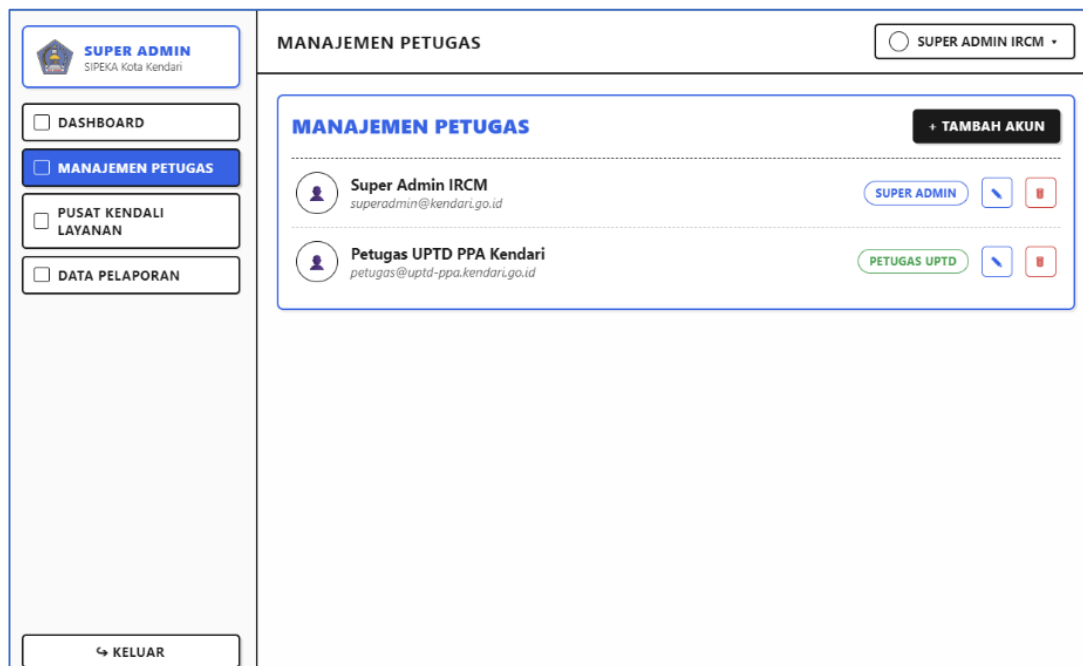
a. Halaman *Dashboard*



Gambar 4. 28 Halaman *Dashboard* untuk *Super admin*

Halaman *dashboard* berfungsi sebagai pusat kendali utama bagi *super admin* untuk memantau kinerja dan aktivitas sistem secara agregat. Antarmuka menampilkan empat panel kartu statistik yang mencakup total laporan yang masuk, jumlah laporan menunggu verifikasi, jumlah kasus sedang diproses, dan jumlah kasus selesai. Di bagian tengah halaman, sistem menampilkan grafik batang tren pelaporan kasus sepanjang tahun berjalan yang disusun per bulan, serta *diagram* distribusi kategori kekerasan yang menunjukkan persentase setiap kategori dari total kasus. Di bagian bawah, halaman menampilkan tabel rekapitulasi kinerja petugas yang memuat nama petugas, jumlah total kasus yang ditangani, jumlah kasus selesai, dan skor kinerja, serta panel *log* aktivitas penanganan terbaru yang menampilkan rekam jejak aktivitas sistem secara *real-time*.

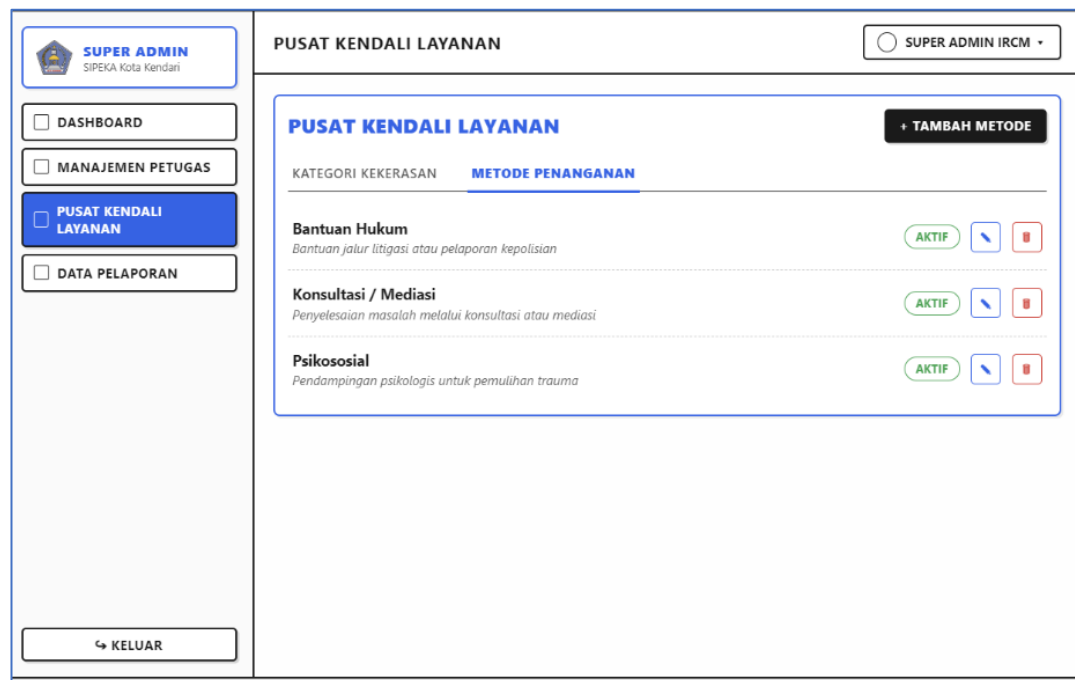
b. Halaman Manajemen Petugas



Gambar 4. 29 Halaman Manajemen Petugas

Halaman ini berfungsi sebagai modul pengelolaan akun pengguna sistem. Antarmuka menampilkan daftar seluruh akun yang terdaftar beserta informasi nama, alamat surel, dan peran masing-masing akun (*super admin* atau admin petugas). Setiap entri akun dilengkapi dengan ikon edit untuk mengubah data akun dan ikon delete untuk menghapus akun yang tidak lagi diperlukan. *Super admin* juga dapat menambahkan akun petugas baru melalui tombol tambah akun yang tersedia di sudut kanan atas halaman.

c. Halaman Pusat Kendali Layanan



Gambar 4. 30 Halaman Pusat Kendali Layanan

Halaman ini berfungsi sebagai modul konfigurasi data master sistem yang terbagi ke dalam dua sub-modul, yaitu kategori kekerasan dan metode penanganan. Pada sub-modul kategori kekerasan, *super admin* dapat menambahkan, mengubah, atau menghapus kategori jenis kekerasan yang tersedia dalam sistem. Pada sub-modul metode penanganan, *super admin* dapat mengelola daftar metode penanganan yang aktif beserta deskripsi masing-masing metode, seperti bantuan hukum, konsultasi/mediasi, dan psikososial. Setiap metode yang aktif ditandai dengan *badge* berwarna hijau, serta dilengkapi ikon edit dan delete untuk operasi pengelolaan data. *Super admin* dapat menambahkan metode baru melalui tombol tambah metode yang tersedia di sudut kanan atas halaman.

d. Halaman Data Pelaporan

SUPER ADMIN
SIPEKA Kota Kendari

☐ DASHBOARD

☐ MANAJEMEN PETUGAS

☐ PUSAT KENDALI LAYANAN

☒ DATA PELAPORAN

[KELUAR](#)

DATA PELAPORAN ○ SUPER ADMIN IRCM

DATA PELAPORAN ↓ EXPORT EXCEL

DARI TANGGAL: dd/mm/yyyy SAMPAI TANGGAL: dd/mm/yyyy WILAYAH / KELURAHAN: Semua Wilayah RESET FILTER

KODE	TGL KEJADIAN	NAMA KORBAN	WILAYAH	KEKERASAN	STATUS
LA-2026-PK7DR	4/9/2026	falah 13 Thn - Laki-laki	Labibia	Kekerasan Fisik	SELESAI

Gambar 4. 31 Halaman Data Pelaporan

Halaman ini berfungsi sebagai modul rekapitulasi dan ekspor data laporan yang telah masuk ke dalam sistem. Antarmuka menyediakan panel filter yang memungkinkan *super admin* untuk menyaring data berdasarkan rentang tanggal kejadian serta wilayah atau kelurahan tertentu. Data hasil filter ditampilkan dalam format tabel yang memuat kolom kode laporan, tanggal kejadian, nama dan data demografis korban, wilayah, kategori kekerasan, dan status kasus. *Super admin* dapat mengunduh seluruh data yang ditampilkan ke dalam format *Excel* melalui tombol ekspor data yang tersedia di sudut kanan atas halaman untuk keperluan pelaporan dan evaluasi kebijakan.

4.4. Pengujian Load Testing

Pengujian *load testing* dirancang untuk membandingkan kinerja sistem pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak pada arsitektur *microservices* dan monolitik. Pengujian menggunakan Apache JMeter 5.6.3 dalam mode non-*GUI* dengan tiga variasi beban, yaitu 100, 500, dan 1.000 pengguna virtual (*virtual user/VU*), *ramp-up period* selama 5 detik, serta *Loop*

Count 1. Setiap pengguna virtual menjalankan empat *endpoint* secara berurutan, yaitu Login, Submit Laporan, Get Penanganan, dan Master Kekerasan. Dengan konfigurasi tersebut, jumlah sampel yang direncanakan pada setiap *endpoint* sama dengan jumlah pengguna virtual pada masing-masing skenario.

Metrik yang diukur dalam pengujian ini meliputi *average response time* (rata-rata waktu respons sistem dalam satuan milidetik), *throughput* (jumlah permintaan yang berhasil diproses per detik), dan *error rate* (persentase permintaan yang gagal diproses). Pengujian dilakukan terhadap empat *endpoint* API utama yang merepresentasikan alur kerja inti sistem, yaitu:

- 1 Login (POST /api/auth/login), untuk mengukur waktu respons proses autentikasi pengguna.
- 2 Submit Laporan (POST /api/laporan), untuk mengukur kinerja proses pengiriman dan penyimpanan laporan masyarakat.
- 3 Get Penanganan (GET /api/penanganan), untuk mengukur kinerja pengambilan daftar data penanganan kasus.
- 4 Master Kekerasan (GET /api/masterkekerasan), untuk mengukur kinerja pengambilan data master jenis kekerasan.

Keempat *endpoint* tersebut mewakili proses autentikasi, penulisan data laporan, pembacaan data penanganan, dan pembacaan data master. Microservices diakses melalui *gateway* pada port 8080, sedangkan monolitik melalui *gateway* pada port 8090. Berikut adalah rancangan skenario pengujian performa yang akan dilakukan:

Tabel 4.4 Skenario Pengujian Performa

No.	Enpoint API	Jumlah <i>Virtual user</i>	Matrik Evaluasi		
			<i>response time</i>	<i>throughput</i>	<i>error rate</i>
1	POST /api/auth/login	100			
		500			
		1000			

Tabel 4.4 Skenario Pengujian Performa (Lanjutan)

No	Endpoint API	Jumlah <i>Virtual user</i>	Matrik Evaluasi		
			<i>response time</i>	<i>throughput</i>	<i>error rate</i>
2	<i>POST /api/laporan</i>	100			
		500			
		1000			
3	<i>GET /api/penanganan</i>	100			
		500			
		1000			
4	<i>GET /api/masterkekerasan</i>	100			
		500			
		1000			

Hasil pengujian dari masing-masing skenario pada Tabel 4.4 akan dianalisis dan dibandingkan untuk mengevaluasi konsistensi performa sistem. Mengacu pada (Tejaya dkk., 2023), sistem dinilai memiliki performa yang baik apabila *average response time* berada pada nilai yang stabil dan tidak mengalami peningkatan signifikan seiring bertambahnya jumlah *virtual user*, *throughput* menunjukkan nilai yang konsisten, serta *error rate* bernilai 0% atau mendekati 0%.

4.5. Pengujian *Fault Injection*

Pengujian *fault injection* dirancang untuk mengamati penampungan pesan perubahan status laporan pada RabbitMQ ketika *report-service* dihentikan sementara serta perubahan kondisi antrean setelah layanan diaktifkan kembali. Selama pengujian, *case-service*, RabbitMQ, basis data, dan *API gateway* akan tetap dijalankan. Rancangan pengujian disajikan pada tabel berikut.

Tabel 4.5 Rancangan pengujian fault injection

No .	Tahap	Tindakan pengujian	Pengamatan yang direncanakan
1	Persiapan	Menyiapkan dua laporan yang belum diregistrasi dan melakukan Login melalui Apache JMeter untuk memperoleh token autentikasi.	Kesiapan data laporan, keberhasilan autentikasi, serta kondisi awal antrian kasus_status_updates.
2	Pemberian gangguan	Menghentikan <i>report-service</i> melalui Docker Desktop, kemudian mengirim dua permintaan POST /api/penanganan/registrasi menggunakan laporan yang berbeda.	Respons registrasi serta nilai <i>Ready</i> , <i>Unacked</i> , dan <i>Total</i> pada antrian melalui RabbitMQ Management. Pesan diharapkan tetap tertampung selama layanan penerima dihentikan.
3	Pemulihan	Mengaktifkan kembali <i>report-service</i> dan mengamati antrian melalui RabbitMQ Management.	Perubahan nilai <i>Ready</i> , <i>Unacked</i> , dan <i>Total</i> setelah layanan diaktifkan kembali. Jumlah pesan dalam antrian diharapkan berkurang.

Hasil pengujian disajikan sebagai perbandingan kondisi antrian ketika *report-service* dihentikan dan setelah diaktifkan kembali. Pengamatan ini digunakan untuk menilai apakah pesan tertampung selama gangguan serta apakah pesan tidak lagi berada dalam antrian setelah pemulihan. Antrian yang kosong tidak digunakan sebagai satu-satunya bukti keberhasilan pembaruan status di basis data; pengujian ini dibatasi pada perilaku antrian pesan.

4.6. Pengujian User Acceptance Testing

User Acceptance Testing (UAT) digunakan untuk mengevaluasi sistem kekerasan terhadap perempuan dan anak berbasis *web* yang dikembangkan menggunakan arsitektur *microservices* untuk memfasilitasi pelaporan, verifikasi, dan pengelolaan kasus di lingkungan DPPPA kota Kendari. Pengujian melibatkan responden yang diminta untuk mencoba sistem secara langsung, kemudian memberikan penilaian melalui pengisian kuesioner. Kuesioner disusun menggunakan skala likert lima tingkat, yaitu sangat tidak setuju (STS), tidak setuju (TS), netral (N), setuju (S), dan sangat setuju (SS).

4.6.1 Instrumen *User Acceptance Testing* pelapor

Instrumen *User Acceptance Testing* (UAT) pelapor digunakan untuk memperoleh penilaian pengguna terhadap fitur dan kemudahan penggunaan sistem pelaporan kekerasan. Pernyataan mencakup aspek fungsionalitas, kemudahan penggunaan, keandalan sistem berdasarkan pengalaman pengguna, serta kelayakan implementasi. Penilaian akan diberikan menggunakan skala Likert lima tingkat setelah responden mencoba fitur pelaporan, dengan rincian pernyataan disajikan pada Tabel 4.6.

Tabel 4.6 Instrumen *User Acceptance Testing* pelapor

No.	Pernyataan	SS	S	N	TS	STS
A. Fungsionalitas						
1	Proses pengiriman laporan kekerasan (termasuk opsi lapor anonim) berjalan lancar dan sesuai yang saya harapkan.					
2	Sebagian formulir terisi otomatis berdasarkan cerita yang saya tulis, sehingga saya tidak perlu mengetik ulang semua informasi					
3	Fitur cek status laporan memberikan informasi yang cukup jelas mengenai perkembangan penanganan kasus saya.					
B. Kemudahan penggunaan (usability)						
4	Instruksi dan langkah-langkah pengisian formulir pelaporan mudah dipahami tanpa membingungkan.					

Tabel 4.6 Instrumen *User Acceptance Testing* pelapor

No.	Pernyataan	SS	S	N	TS	STS
5	Navigasi antar menu dan halaman dalam sistem berjalan dengan lancar dan mudah diikuti oleh pengguna.					
6	Sistem menampilkan notifikasi atau konfirmasi yang jelas setiap kali operasi berhasil atau gagal dilakukan.					
C. Keandalan sistem berdasarkan penilaian pengguna						
7	Sistem dapat saya gunakan selama proses pelaporan tanpa mengalami gangguan yang menghambat.					
8	Informasi laporan yang ditampilkan setelah pengiriman sesuai dengan data yang saya masukkan.					
9	Fitur cek status laporan dapat diakses kembali ketika saya membutuhkannya.					
D. Kelayakan implementasi						
10	Saya merasa lebih yakin melaporkan kekerasan lewat sistem ini dibandingkan cara pelaporan manual/datang langsung.					
11	Saya bersedia menggunakan sistem ini apabila perlu menyampaikan laporan kekerasan.					
12	Sistem ini sesuai dengan kebutuhan saya untuk menyampaikan laporan kekerasan.					

4.6.2 Instrumen *User Acceptance Testing* admin

Instrumen *User Acceptance Testing* (UAT) admin digunakan untuk memperoleh penilaian pengguna terhadap dukungan sistem dalam pengelolaan laporan dan penanganan kasus sesuai kewenangannya. Pernyataan mencakup aspek fungsionalitas, kemudahan penggunaan, keandalan sistem berdasarkan pengalaman pengguna, serta kelayakan implementasi. Penilaian akan diberikan menggunakan skala Likert lima tingkat setelah responden mencoba fitur pengelolaan sistem, dengan rincian pernyataan disajikan pada Tabel 4.7.

Tabel 4.7 Instrumen *User Acceptance Testing* admin

No.	Pernyataan	SS	S	N	TS	STS
A. Fungsionalitas						
1	Fitur registrasi laporan oleh petugas berfungsi sesuai dengan alur kerja yang seharusnya.					
2	Fitur input hasil assessment dan rencana intervensi tersimpan dengan benar ke dalam data kasus.					
3	Fitur pengelolaan akun petugas dan data master (kategori kekerasan, metode penanganan) berfungsi dengan benar.					
4	Fitur filter dan ekspor data laporan ke format Excel berfungsi sesuai kebutuhan.					
5	Fitur pengelolaan data master (kategori kekerasan dan metode penanganan) pada pusat kendali layanan berjalan dengan benar.					
B. Kemudahan penggunaan (usability)						
6	Antarmuka sistem, termasuk navigasi antar menu, mudah dipahami dan digunakan tanpa memerlukan pelatihan khusus.					

Tabel 4.7 Instrumen *User Acceptance Testing* admin

No.	Pernyataan	SS	S	N	TS	STS
7	Tampilan data laporan dan kasus tersaji secara terstruktur dan mudah dibaca.					
8	Navigasi antar menu dan halaman dalam sistem berjalan dengan lancar dan mudah diikuti oleh pengguna.					
9	Sistem menampilkan notifikasi atau konfirmasi yang jelas setiap kali operasi berhasil atau gagal dilakukan.					
C. Keandalan sistem berdasarkan penilaian pengguna						
10	Data yang ditampilkan pada setiap halaman konsisten dan sesuai dengan data yang telah diinputkan ke dalam sistem.					
11	Sistem mampu menangani perpindahan status kasus secara otomatis tanpa kesalahan.					
D. Kelayakan implementasi						
12	Sistem ini membantu mempercepat dan meningkatkan efektivitas proses penanganan kasus.					
13	Alur kerja yang diimplementasikan (pelaporan, registrasi, assessment, intervensi, penyelesaian) sesuai dengan prosedur operasional yang berlaku.					

BAB V

IMPLEMENTASI DAN PENGUJIAN SISTEM

5.1. Implementasi Sistem

Tahap implementasi merupakan penerapan hasil rancangan pada BAB IV ke dalam bentuk perangkat lunak yang dapat dioperasikan. Sistem Pelaporan dan Manajemen Kasus Kekerasan terhadap Perempuan dan Anak ini dikembangkan menggunakan bahasa pemrograman JavaScript, dengan React.js sebagai *framework* antarmuka (*frontend*), Express.js yang berjalan di atas Node.js sebagai kerangka kerja layanan (*backend*), dan MongoDB sebagai basis data, dibangun menggunakan Visual Studio Code sebagai *code editor*. Sistem menerapkan arsitektur *microservices* yang terdiri atas dua layanan independen, yaitu *Case service* dan *Report service*, dengan basis data terpisah sesuai prinsip *database per service*. komunikasi antarlayanan diimplementasikan menggunakan RabbitMQ sebagai *message broker* agar pembaruan status laporan tetap tersampaikan meskipun salah satu layanan tidak aktif, sedangkan seluruh permintaan dari antarmuka diteruskan melalui *API Gateway* berbasis Nginx dan setiap komponen dijalankan di dalam *container* Docker.

5.2. Implementasi Interface

Antarmuka sistem diimplementasikan berdasarkan rancangan *user interface* pada subbab 4.2. Antarmuka dibagi menjadi tiga kelompok pengguna sesuai hak akses masing-masing, yaitu antarmuka pelapor, antarmuka admin petugas, dan antarmuka super admin.

5.2.1 Halaman Login

Halaman Login adalah halaman dimana pengguna diminta untuk mengisi username dan password terlebih dahulu untuk mendapat hak akses masuk ke halaman utama sistem. Tampilan halaman Login dapat dilihat pada Gambar 5.1.

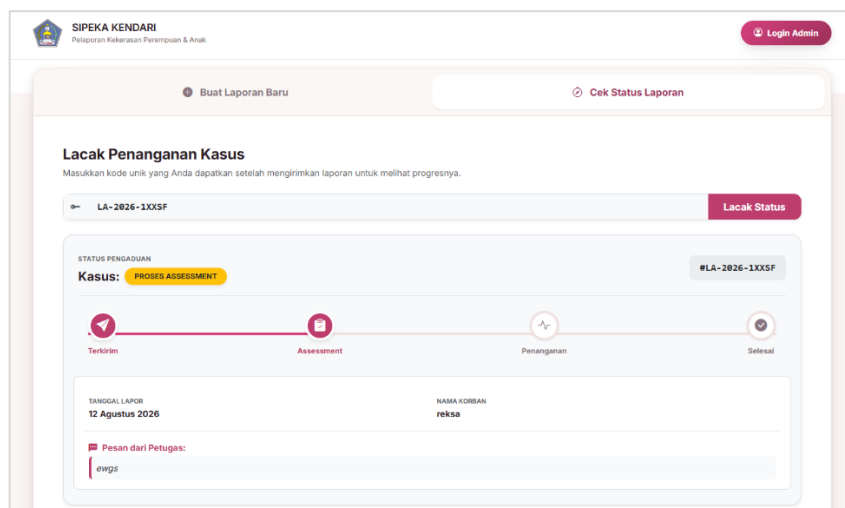
Gambar 5.1 Halaman Login

5.2.2 Antarmuka Pelapor

Antarmuka pelapor dapat diakses secara publik tanpa proses autentikasi. Halaman ini menyediakan formulir pelaporan yang memungkinkan masyarakat menyampaikan laporan kekerasan, baik dengan identitas maupun secara anonim. Tampilan halaman pelaporan ditunjukkan pada Gambar 5.1.

Gambar 5.2 Halaman pelaporan

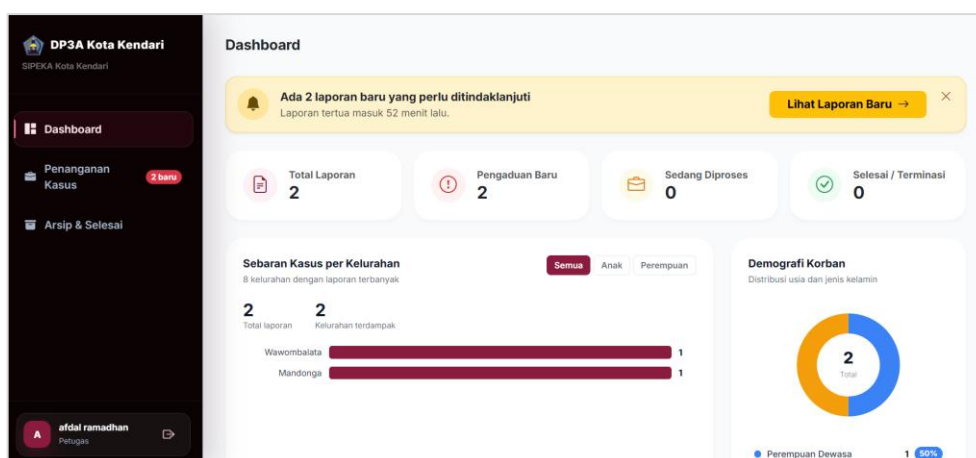
Selain dapat melihat halaman buat laporan di tab buat laporan baru, pengguna juga dapat mengecek status dari laporan yang sudah diajukan di sistem. Dengan cara melihat di tab cek status laporan yang dapat dilihat pada Gambar 5.2



Gambar 5. 3 Halaman pelaporan

5.2.3 Halaman Dashboard Petugas

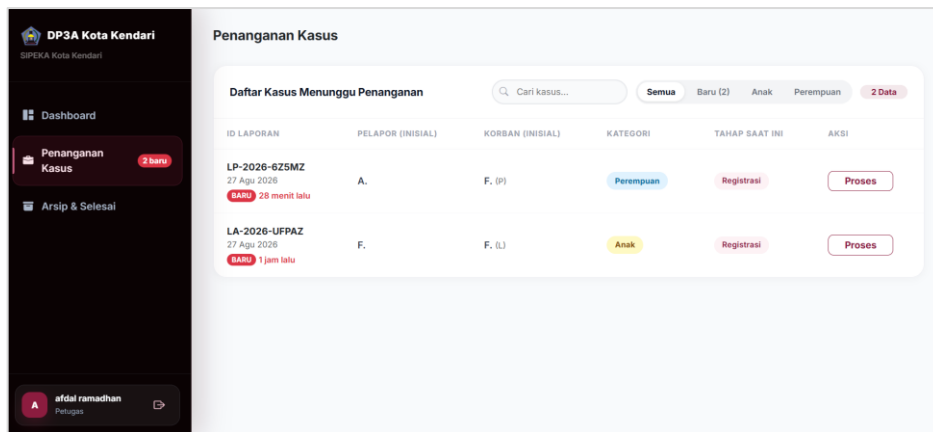
Halman admin petugas hanya dapat diakses setelah proses *login* berhasil dilakukan. Halaman pertama yang terbuka saat pertama kali masuk adalah halaman dashboard, dimana halaman dashboard berisi informasi total laporan, jumlah pengaduan baru, jumlah kasus yang diproses, dan jumlah kasus yang telah selesai. Juga terdapat grafik yang menunjukkan demografi korban dan sebaran kasus per kelurahan. Untuk tampilan halaman dashboard petugas dapat dilihat pada Gambar 5.4.



Gambar 5. 4 Halaman Dashboard Petugas

5.2.4 Halaman Penanganan Kasus

Halaman penanganan kasus menampilkan seluruh laporan yang telah masuk ke dalam sistem. Data disajikan dalam format tabel yang memuat kolom kode laporan, tanggal melapor, nama dan data demografis korban, wilayah, serta status kasus saat ini. Status kasus ditampilkan dalam bentuk *badge* berwarna yang membedakan antara status. Di halaman ini lah petugas memproses laporan dari pelapor. Tampilan halaman penanganan kasus dapat dilihat pada Gambar 5.5.



ID LAPORAN	PELAPOR (INISIAL)	KORBAN (INISIAL)	KATEGORI	TAHAP SAAT INI	AKSI
LP-2026-6Z5MZ 27 Agu 2026 BARU 28 menit lalu	A.	F. (P)	Perempuan	Registrasi	Proses
LA-2026-UFPZ 27 Agu 2026 BARU 1 jam lalu	F.	F. (L)	Anak	Registrasi	Proses

Gambar 5. 5 Halaman Penanganan Kasus

Untuk memproses setiap kasus yang masuk, user tingkat petugas harus menekan tombol proses. Nanti petugas akan dibawa ke halaman detail laporan dimana petugas dapat memproses kasus dari registrasi sampai selesai. Untuk halaman detail laporan dapat dilihat pada Gambar 5.6.

Detail Laporan

[← Kembali ke Daftar](#)

Detail Laporan LA-2026-UFAZ Status: Registrasi

Data sensitif (Nama Lengkap/Alamat Pelapor) disamarkan sesuai protokol keamanan.

1 Pengaduan
2 Registrasi
3 Assessment
4 Rencana Intervensi
5 Monitoring
6 Terminasi

Identitas Subjek

Inisial Pelapor	Inisial Korban
F.	F.
Gender Korban	Usia Korban
Laki-laki	15 Tahun
Pekerjaan Korban	Status Perkawinan
Data tidak tersedia	Data tidak tersedia

Kontak Pelapor (Untuk Tindak Lanjut)

081343751667 [Hubungi via WhatsApp](#)

Pastikan kondisi pelapor aman sebelum membahas laporan.

Preferensi Pertemuan (Dipilih Pelapor)

[Datang ke UPTD](#)

Informasi Kejadian

Jenis Kasus

Kekerasan Fisik

Alamat Kejadian / Domisili Korban

rumah saya, Mandonga

Bukti Terlampir

Tidak ada lampiran bukti

Detail Singkat Kasus

Laporan masuk terkait dugaan kasus Kekerasan Fisik yang terjadi di wilayah Mandonga.

Kronologi (Dari Pelapor)

saya berumur 15 tahun, nama saya fajar panaungi. saya melihat adek teman saya dipukuli bapaknya beberapa hari ini

Formulir Aksi: Tahap Registrasi

Lengkapi data berikut untuk melanjutkan ke tahap selanjutnya.

Metode Pertemuan yang Dikonfirmasi

-- Pilih Metode Pertemuan --

Pilihan awal pelapor: **Datang ke UPTD**. Ubah hanya jika pelapor menyepakati metode lain.

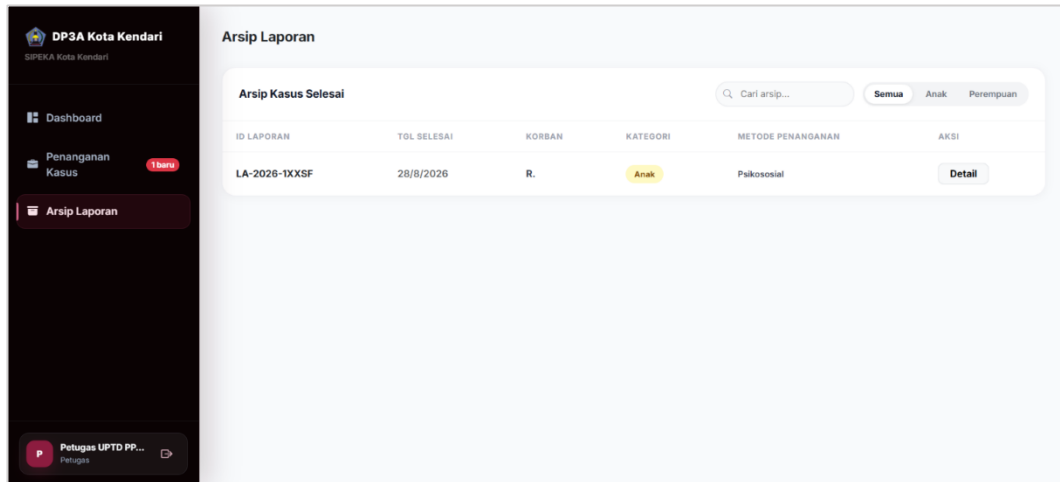
Pesan Tindak Lanjut untuk Pelapor

Contoh: Laporan tervalidasi. Siapkan jadwal assessment...

Gambar 5. 6 Halaman Detail Laporan

5.2.5 Halaman Arsip Laporan

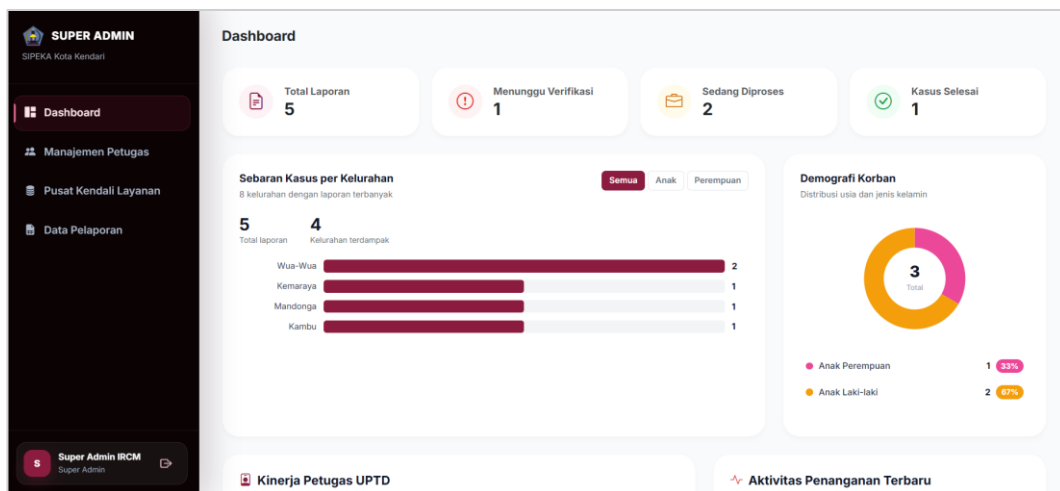
Halaman ini menampilkan daftar kasus yang telah selesai ditangani dan diarsipkan oleh sistem. Setiap baris menyajikan kode laporan, tanggal penyelesaian, inisial korban, kategori (anak atau perempuan), dan metode penanganan yang digunakan, dilengkapi fitur pencarian dan penyaringan berdasarkan kategori serta tombol Detail untuk melihat riwayat penanganan secara lengkap. Tampilan halaman arsip laporan ditunjukkan pada Gambar 5.6.



Gambar 5. 7 Halaman Arsip Laporan

5.2.6 Halaman *dashboard* Super Admin

Antarmuka super admin menyediakan fungsi pengelolaan akun petugas serta pengelolaan data master, yaitu kategori kekerasan dan metode penanganan. Antarmuka ini juga menampilkan rekapitulasi data laporan dan kasus dalam bentuk ringkasan statistik, dimana itu dapat dilihat di halaman *dashboard* super admin. Tampilan halaman *dashboard* super admin ditunjukkan pada Gambar 5.8.

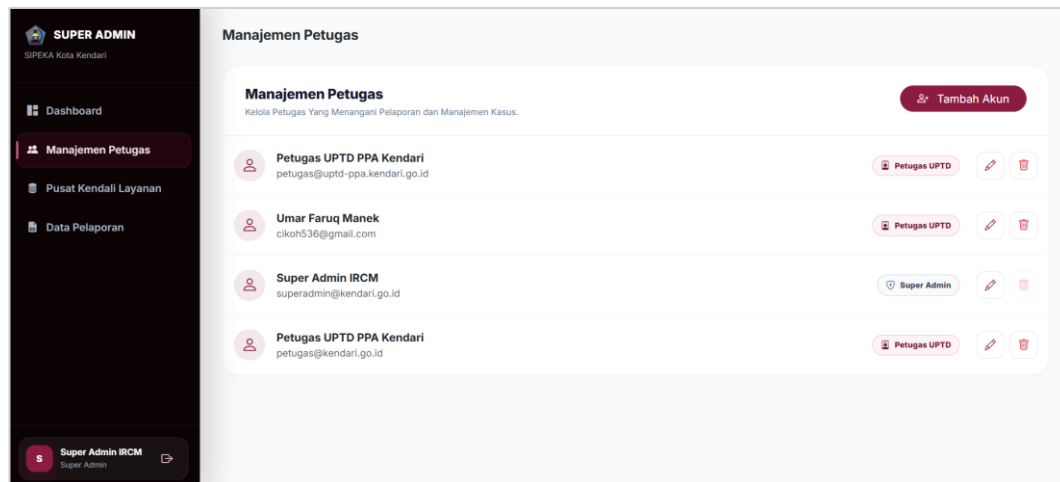


Gambar 5. 8 Halaman *Dashboard* Admin

5.2.7 Halaman Manajemen Petugas

Halaman ini menyediakan fungsi pengelolaan akun petugas oleh super admin, meliputi penambahan, pengubahan, dan penghapusan data akun. Halaman menampilkan daftar petugas yang terdaftar beserta perannya, dan menyediakan

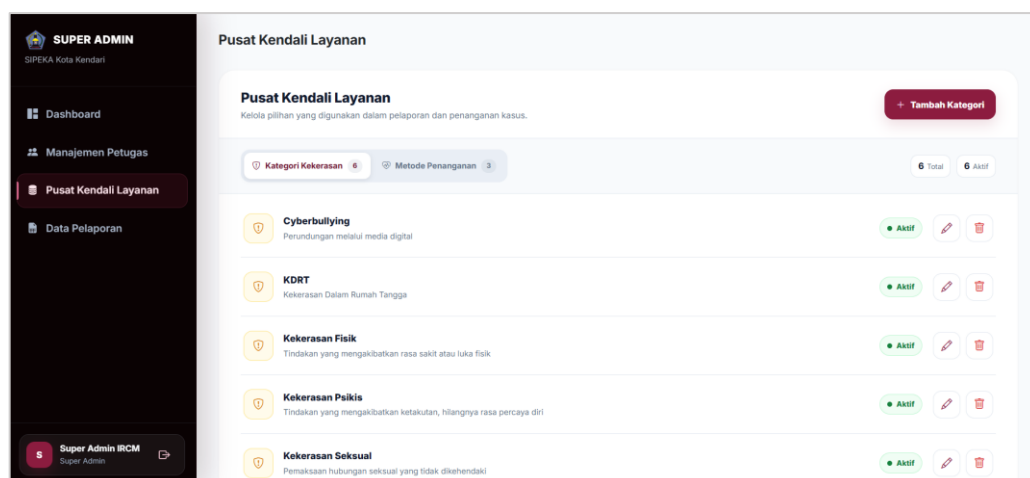
formulir untuk menambahkan atau memperbarui data akun petugas. Tampilan halaman manajemen petugas ditunjukkan pada Gambar 5.9.



Gambar 5. 9 Halaman Manajemen Petugas

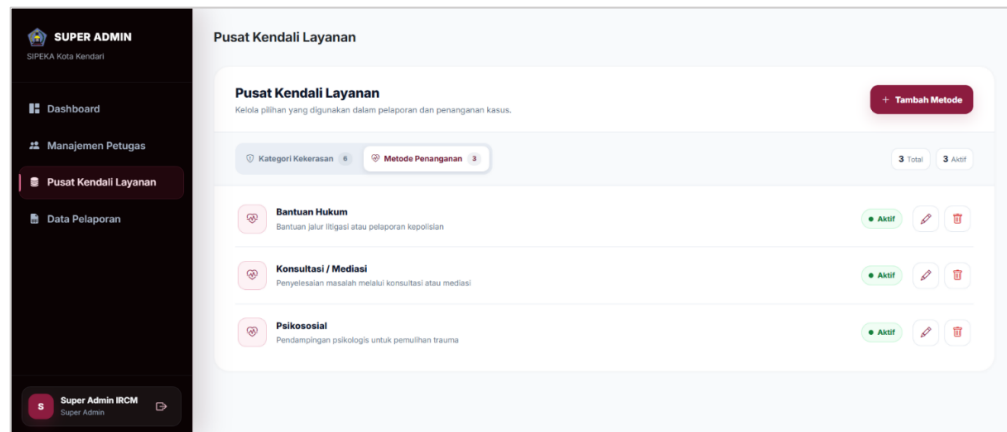
5.2.8 Halaman Pusat Kendali Layanan

Halaman ini menyediakan fungsi pengelolaan data master yang digunakan dalam proses pelaporan dan penanganan kasus, terbagi menjadi dua tab. Tab Kategori Kekerasan menampilkan daftar jenis kekerasan yang menjadi pilihan pada formulir pelaporan, seperti Kekerasan Fisik, Kekerasan Psikis, Kekerasan Seksual, KDRT, dan Cyberbullying, lengkap dengan nama, deskripsi, dan status aktif setiap kategori. Super admin dapat menambahkan, mengubah, atau menghapus kategori melalui tombol yang tersedia pada setiap baris data. Tampilan tab kategori kekerasan ditunjukkan pada Gambar 5.10.



Gambar 5. 10 Tab Kategori Kekerasan

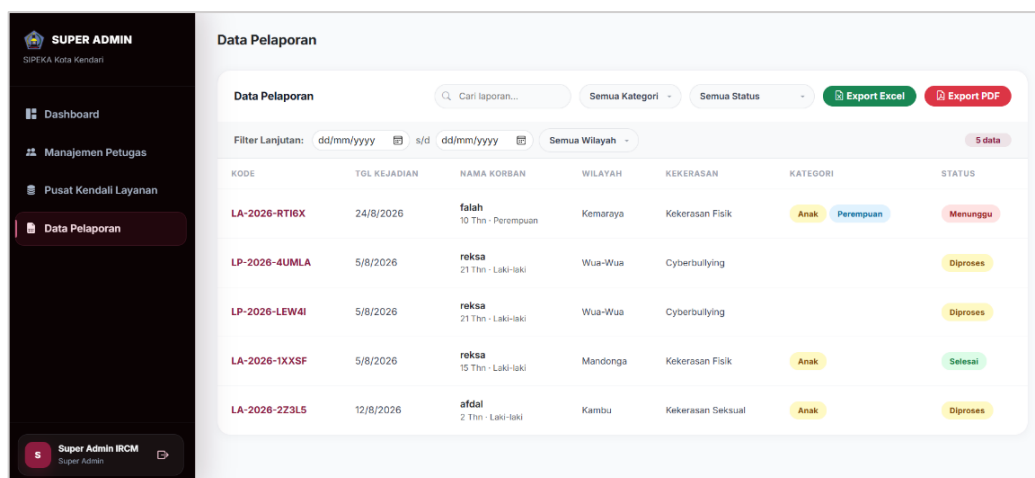
Tab Metode Penanganan menampilkan daftar metode yang menjadi pilihan petugas saat menyusun rencana intervensi kasus, dengan struktur tampilan dan fungsi pengelolaan yang sama seperti tab Kategori Kekerasan, yaitu penambahan, pengubahan, dan penghapusan data. Tampilan tab metode penanganan ditunjukkan pada Gambar 5.11.



Gambar 5. 11 Tab Metode Penanganan

5.2.9 Halaman Data Pelaporan

Halaman ini menampilkan seluruh data laporan yang masuk ke sistem, dilengkapi kolom kode laporan, tanggal kejadian, nama serta usia dan jenis kelamin korban, wilayah, jenis kekerasan, kategori, dan status penanganan. Super admin dapat mencari dan menyaring data berdasarkan kategori, status, rentang tanggal, maupun wilayah, serta mengekspor data yang ditampilkan ke dalam format Excel atau PDF. Tampilan halaman data pelaporan ditunjukkan pada Gambar 5.12.



Gambar 5. 12 Halaman Data Pelaporan

5.3. Implementasi *Message Broker*

Sinkronisasi status kasus antara *Case service* dan *Report service* diimplementasikan menggunakan RabbitMQ sebagai *message broker*, menggantikan pemanggilan HTTP langsung yang sebelumnya menyebabkan pembaruan status hilang permanen jika *Report service* sedang tidak aktif. *Case service* berperan sebagai *producer* dan *Report service* sebagai *consumer*, berkomunikasi melalui antrean *durable* bernama *kasus_status_updates* dengan pesan bersifat *persistent*. Publikasi pesan dilakukan pada tiga proses perubahan status kasus, sebagaimana ditunjukkan pada Tabel 5.1.

Tabel 5. 1 Titik publikasi pesan perubahan status kasus

No	Proses pada <i>Case service</i>	Status yang dipublikasikan
1	Registrasi laporan	proses_assessment
2	Rencana intervensi	dalam_penanganan
3	Penyelesaian kasus	selesai

Publikasi pesan dilakukan setelah perubahan data kasus berhasil disimpan ke basis data *Case service*. Urutan tersebut dipilih agar kegagalan publikasi tidak menggagalkan operasi utama, karena data kasus sebagai sumber kebenaran (*source of truth*) telah tersimpan terlebih dahulu. Potongan kode program pengiriman pesan pada sisi *producer* ditunjukkan pada Tabel 5.2

Tabel 5. 2 Kode program pengiriman pesan pada *Case service*

Kode
<pre>const amqp = require('amqplib'); const RABBITMQ_URL = process.env.RABBITMQ_URL 'amqp://localhost:5672'; const QUEUE_NAME = 'kasus_status_updates'; let channel = null; async function connect() { if (channel) return channel; const connection = await amqp.connect(RABBITMQ_URL); connection.on('close', () => { channel = null; });</pre>

```

    connection.on('error', () => { channel = null; });
    channel = await connection.createChannel();
    await channel.assertQueue(QueueName, { durable: true });
    return channel;
  }
  async function publishStatusUpdate({ laporan_id, status,
  catatan }) {
    const ch = await connect();
    const payload = Buffer.from(
      JSON.stringify({ laporan_id, status, catatan })
    );
    ch.sendToQueue(QueueName, payload, { persistent: true });
  }

```

Pada sisi *consumer*, *Report service* terhubung ke antrean secara otomatis pada saat layanan dijalankan, kemudian membaca pesan satu per satu dengan pengaturan *prefetch* bernilai satu. Pesan yang berhasil diproses ditandai selesai melalui mekanisme *acknowledgement*, sedangkan pesan yang gagal diproses dikembalikan ke antrean untuk diproses ulang. Potongan kode program penerimaan pesan ditunjukkan pada Tabel 5.3

Tabel 5. 3 Kode program pengiriman pesan pada *Case service*

Kode
<pre> async function startConsumer() { let connection; try { connection = await amqp.connect(RABBITMQ_URL); } catch (err) { await sleep(RETRY_DELAY_MS); return startConsumer(); } connection.on('close', () => { setTimeout(startConsumer, RETRY_DELAY_MS); }); const channel = await connection.createChannel(); await channel.assertQueue(QueueName, { durable: true }); channel.prefetch(1); </pre>

```

channel.consume(QueueName, async (msg) => {
  if (!msg) return;
  const { laporan_id, status, catatan } =
    JSON.parse(msg.content.toString());
  try {
    await Laporan.findByIdAndUpdate(laporan_id, { status,
catatan });
    channel.ack(msg);
  } catch (dbErr) {
    channel.nack(msg, false, true);
  }
});
}

```

5.4. Implementasi Docker

Dalam melakukan penerapan sistem, penulis menggunakan Docker untuk menjalankan komponen dalam container. Konfigurasi masing-masing komponen dikelola melalui Docker Compose, meliputi layanan backend, basis data, message broker, API Gateway, dan antarmuka pengguna. Setiap container backend, baik pada arsitektur *microservices* maupun monolitik, dikonfigurasi dengan batas sumber daya sebesar 1 CPU dan memori 512 MB. Batas tersebut berlaku per container, sehingga total alokasi kedua layanan *microservices* berbeda dari satu layanan monolitik. Berikut potongan konfigurasi utama yang digunakan pada sistem.

1. Arsitektur Monolitik

Konfigurasi sistem menggunakan arsitektur monolitik dijalankan dalam satu container dengan database `monolith_db`. Layanan ini digunakan sebagai pembanding pada pengujian performa dan dapat diakses melalui port 8090. Konfigurasinya ditunjukkan pada kode berikut.

Tabel 5. 4 Konfigurasi Arsitektur Monolitik

Kode
<pre> monolith-service: build: context: ./backend/monolith-service dockerfile: Dockerfile container_name: ircm-monolith-service restart: unless-stopped deploy: resources: limits: cpus: '1' memory: 512M environment: MONGODB_URI: mongodb://mongodb:27017/monolith_db PORT: 8000 REPORTING_SERVICE_URL: http://localhost:8000 extra_hosts: - "host.docker.internal:host-gateway" ports: - "8090:8000" volumes: - ./backend/monolith-service:/usr/src/app - /usr/src/app/node_modules depends_on: - mongodb networks: - ircm-network </pre>

2. Arsitektur *Microservice*

Pada arsitektur *microservices*, sistem dibagi menjadi *Report service* dan *Case service* yang dikembangkan menggunakan Node.js dan Express.js. *Report service* menggunakan database *report_db* serta terhubung ke RabbitMQ untuk menerima pesan perubahan status laporan. Konfigurasi *Report service* ditunjukkan pada kode berikut.

Tabel 5. 5 Konfigurasi *Report service*

Kode
<pre> report-service: build: context: ../backend/reporting-service dockerfile: Dockerfile container_name: ircm-report-service restart: unless-stopped deploy: resources: limits: cpus: '1' memory: 512M environment: MONGODB_URI: mongodb://mongodb:27017/report_db RABBITMQ_URL: amqp://rabbitmq:5672 PORT: 8000 extra_hosts: - "host.docker.internal:host-gateway" volumes: - ../backend/reporting-service:/usr/src/app - /usr/src/app/node_modules depends_on: - mongodb - rabbitmq networks: - ircm-network </pre>

Case service dijalankan dalam container terpisah untuk mengelola data penanganan kasus. Layanan ini menggunakan database *case_db* dan terhubung ke RabbitMQ untuk mengirimkan pesan perubahan status kepada *Report service*. Konfigurasi *Case service* ditunjukkan pada kode berikut.

Tabel 5. 6 Konfigurasi *Case service*

Kode
<pre> case-service: build: context: ./backend/case-service dockerfile: Dockerfile container_name: ircm-case-service restart: unless-stopped deploy: resources: limits: cpus: '1' memory: 512M environment: MONGODB_URI: mongodb://mongodb:27017/case_db RABBITMQ_URL: amqp://rabbitmq:5672 PORT: 8000 REPORTING_SERVICE_URL: http://report-service:8000 volumes: - ./backend/case-service:/usr/src/app - /usr/src/app/node_modules depends_on: - mongodb - rabbitmq networks: - ircm-network </pre>

3. Database MongoDB

Konfigurasi MongoDB digunakan untuk menyediakan penyimpanan data sistem. Database `report_db`, `case_db`, dan `monolith_db` ditempatkan dalam satu container MongoDB, sehingga pemisahan dilakukan pada tingkat database. *Volume* MongoDB data digunakan untuk menyimpan data di luar lapisan tulis container. Konfigurasi MongoDB ditunjukkan pada kode berikut.

Tabel 5. 7 Konfigurasi Database

Kode
<pre> mongodb: image: mongo:latest container_name: ircm-mongodb restart: unless-stopped ports: - "27017:27017" volumes: - mongodb-data:/data/db networks: - ircm-network </pre>

4. Message Broker RabbitMQ

Konfigurasi RabbitMQ sebagai message broker menggunakan image yang dilengkapi antarmuka pengelolaan. Port 5672 digunakan untuk komunikasi pesan, sedangkan port 15672 digunakan untuk mengakses halaman pengelolaan RabbitMQ. Penyimpanan data broker menggunakan volume rabbitmq-data, sebagaimana ditunjukkan pada kode berikut.

Tabel 5. 8 Konfigurasi *Message Broker*

Kode
<pre> rabbitmq: image: rabbitmq:3-management-alpine container_name: ircm-rabbitmq restart: unless-stopped ports: - "5672:5672" - "15672:15672" volumes: - rabbitmq-data:/var/lib/rabbitmq networks: - ircm-network </pre>

5.5. Pengujian *Load Testing*

Pengujian *load testing* dilakukan untuk mengukur kinerja sistem pada arsitektur *microservices* dan monolitik ketika menerima permintaan pengguna. Metrik yang diamati meliputi rata-rata waktu respons (*average response time*), *throughput*, dan *error rate*. Pengujian menggunakan 100, 500, dan 1.000 pengguna virtual untuk mengamati perubahan kinerja seiring peningkatan beban.

Pengujian dijalankan menggunakan Apache JMeter 5.6.3 dalam mode non-GUI dengan ramp-up 5 detik dan *Loop Count* 1. Setiap pengguna virtual menjalankan Login, Submit Laporan, Get Penanganan, dan Master Kekerasan secara berurutan, sehingga setiap endpoint menghasilkan jumlah sampel yang sama dengan jumlah pengguna virtual. *Microservices* diakses melalui gateway port 8080 dan monolitik melalui gateway port 8090. Kedua arsitektur menggunakan data awal dan indeks basis data yang setara serta batas sumber daya 1 CPU dan memori 512 MB pada setiap layanan backend. Setiap skenario dijalankan satu kali secara terpisah.

Data dasar dan hasil pengujian pada 100, 500, dan 1.000 pengguna virtual disajikan pada Tabel 5.9 sampai dengan Tabel 5.14. Rata-rata waktu respons dihitung dari T_s dibagi N , *throughput* dari N dibagi D , dan *error rate* dari F dibagi N kemudian dikalikan 100%.

Jumlah pengguna virtual berbeda dari *throughput* karena *throughput* menyatakan laju permintaan per detik. Rentang waktu aktual dihitung sejak sampel pertama dimulai hingga sampel terakhir selesai. Perhitungan memasukkan sampel berhasil dan gagal agar nilainya konsisten dengan hasil JMeter.

a. Pengujian 100 pengguna virtual

Pengujian pada 100 pengguna virtual dilakukan untuk mengamati waktu respons, *throughput*, dan *error rate* pada beban tersebut. Data dasar perhitungan disajikan pada Tabel 5.9, sedangkan hasil perbandingan kedua arsitektur disajikan pada Tabel 5.10. Data dasar berikut digunakan untuk menghitung rata-rata waktu respons, *throughput*, dan *error rate*. N adalah jumlah sampel. T_s adalah jumlah waktu respons seluruh sampel (ms). D adalah rentang waktu dari awal sampel

pertama hingga akhir sampel terakhir pada endpoint yang sama (detik). F adalah jumlah sampel gagal.

Tabel 5.9 Data dasar perhitungan pada 100 pengguna virtual

No	Endpoint	N	Microservices			Monolitik		
			Ts (ms)	D (detik)	F	Ts (ms)	D (detik)	F
1	Login	100	297.015	9,820	0	369.501	11,457	0
2	Submit Laporan	100	1.079	9,572	0	28.814	11,281	0
3	Get Penanganan	100	26.960	9,456	0	22.730	11,054	0
4	Master Kekerasan	100	977	9,081	0	24.481	10,820	0

Contoh substitusi rumus berikut menggunakan endpoint Login

1. Microservice

$$RT = \frac{T_s}{N} = \frac{297.015}{100} = 2.970,15 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{100}{9,820} = 10,18 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{100} \times 100\% = 0,00\%$$

2. Monolitik

$$RT = \frac{T_s}{N} = \frac{369.501}{100} = 3.695,01 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{100}{11,457} = 8,73 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{100} \times 100\% = 0,00$$

Perhitungan rata-rata waktu respons, throughput, dan error rate dilakukan pada seluruh endpoint menggunakan data dasar pada Tabel 5.9. Hasil perhitungan kemudian disajikan pada Tabel 5.10 untuk membandingkan kinerja arsitektur microservices dan monolitik pada beban 100 pengguna virtual.

Tabel 5.10 Perbandingan hasil pengujian pada 100 pengguna virtual

No	Endpoint	Microservices			Monolitik		
		Average (ms)	Throughput (req/s)	Error (%)	Average (ms)	Throughput (req/s)	Error (%)
1	Login	2.970,2	10,18	0,00	3.695,0	8,73	0,00
2	Submit Laporan	10,8	10,45	0,00	288,1	8,86	0,00
3	Get Penanganan	269,6	10,58	0,00	227,3	9,05	0,00
4	Master Kekerasan	9,8	11,01	0,00	244,8	9,24	0,00

Berdasarkan Tabel 5.12 *microservices* mencatat rata-rata waktu respons lebih rendah daripada monolitik pada Login sebesar 19,62%, Submit Laporan sebesar 96,26%, dan Master Kekerasan sebesar 96,01%. Pada Get Penanganan, monolitik memiliki rata-rata waktu respons lebih rendah sebesar 15,69%. *Throughput* *microservices* berada pada rentang 10,18-11,01 req/s, sedangkan monolitik berada pada rentang 8,73-9,24 req/s. Seluruh permintaan pada kedua arsitektur berhasil diproses dengan *error rate* 0%.

b. Pengujian 500 pengguna virtual

Pengujian pada 500 pengguna virtual dilakukan untuk mengamati waktu respons, *throughput*, dan *error rate* pada beban tersebut. Data dasar perhitungan disajikan pada Tabel 5.11, sedangkan hasil perbandingan kedua arsitektur disajikan pada Tabel 5.12. Data dasar berikut digunakan untuk menghitung rata-rata waktu respons, *throughput*, dan *error rate*. N adalah jumlah sampel. Ts adalah jumlah waktu respons seluruh sampel (ms). D adalah rentang waktu dari awal sampel pertama hingga akhir sampel terakhir pada endpoint yang sama (detik). F adalah jumlah sampel gagal.

Tabel 5.11 Data dasar perhitungan pada 500 pengguna virtual

No	Endpoint	N	Microservices			Monolitik		
			Ts (ms)	D (detik)	F	Ts (ms)	D (detik)	F
1	Login	500	11.331.001	48,659	0	13.386.457	56,306	0
2	Submit Laporan	500	3.496	48,323	0	158.794	55,670	0
3	Get Penanganan	500	177.804	48,304	0	143.824	55,387	0
4	Master Kekerasan	500	3.312	47,986	0	112.872	54,888	0

Contoh substitusi rumus berikut menggunakan endpoint Login

1. Microservice

$$RT = \frac{T_s}{N} = \frac{11.331.001}{500} = 22.662,00 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{500}{48,659} = 10,28 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{500} \times 100\% = 0,00\%$$

2. Monolitik

$$RT = \frac{T_s}{N} = \frac{13.386.457}{500} = 26.772,91 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{500}{56,306} = 8,88 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{500} \times 100\% = 0,00\%$$

Perhitungan rata-rata waktu respons, throughput, dan error rate dilakukan pada seluruh endpoint menggunakan data dasar pada Tabel 5.11. Hasil perhitungan kemudian disajikan pada Tabel 5.12 untuk membandingkan kinerja arsitektur microservices dan monolitik pada beban 500 pengguna virtual.

Tabel 5.12 Perbandingan hasil pengujian pada 500 pengguna virtual

No	Endpoint	Microservices			Monolitik		
		Average (ms)	Throughput (req/s)	Error (%)	Average (ms)	Throughput (req/s)	Error (%)
1	Login	22.662,0	10,28	0,00	26.772,9	8,88	0,00
2	Submit Laporan	7,0	10,35	0,00	317,6	8,98	0,00
3	Get Penanganan	355,6	10,35	0,00	287,6	9,03	0,00
4	Master Kekerasan	6,6	10,42	0,00	225,7	9,11	0,00

Berdasarkan Tabel 5.12, *microservices* memiliki rata-rata waktu respons lebih rendah dibandingkan monolitik pada Login sebesar 15,35%, Submit Laporan sebesar 97,80%, dan Master Kekerasan sebesar 97,07%. Sebaliknya, monolitik memiliki rata-rata waktu respons lebih rendah pada Get Penanganan sebesar 19,11%. *Throughput microservices* berada pada rentang 10,28 – 10,42 req/s, sedangkan monolitik berada pada rentang 8,88 – 9,11 req/s. Seluruh endpoint pada kedua arsitektur mencatat error rate sebesar 0%, sehingga tidak terdapat sampel yang dinyatakan gagal oleh JMeter pada skenario ini.

c. Pengujian 1.000 pengguna virtual

Pengujian pada 1.000 pengguna virtual dilakukan untuk mengamati waktu respons, *throughput*, dan *error rate* pada beban tersebut. Data dasar perhitungan disajikan pada Tabel 5.13, sedangkan hasil perbandingan kedua arsitektur disajikan pada Tabel 5.14. Data dasar berikut digunakan untuk menghitung rata-rata waktu respons, *throughput*, dan *error rate*. N adalah jumlah sampel. Ts adalah jumlah waktu respons seluruh sampel (ms). D adalah rentang waktu dari awal sampel pertama hingga akhir sampel terakhir pada endpoint yang sama (detik). F adalah jumlah sampel gagal.

Tabel 5.13 Data dasar perhitungan pada 1.000 pengguna virtual

No	Endpoint	N	Microservices			Monolitik		
			Ts (ms)	D (detik)	F	Ts (ms)	D (detik)	F
1	Login	1000	47.812.973	97,130	0	48.554.622	97,398	0
2	Submit Laporan	1000	7.855	96,345	0	3.392.249	98,321	1
3	Get Penanganan	1000	903.893	96,308	0	344.572	98,040	0
4	Master Kekerasan	1000	6.750	95,794	0	255.322	97,675	0

Contoh substitusi rumus berikut menggunakan endpoint Login

1. Microservice

$$RT = \frac{T_s}{N} = \frac{47.812.973}{1000} = 47.812,973 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{1000}{97,130} = 10,30 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{1000} \times 100\% = 0,00\%$$

2. Monolitik

$$RT = \frac{T_s}{N} = \frac{48.554.622}{1000} = 48.554,62 \text{ ms}$$

$$Throughput = \frac{N}{D} = \frac{1000}{97,398} = 10,27 \text{ req/s}$$

$$Error \text{ rate} = \frac{F}{N} \times 100\% = \frac{0}{1000} \times 100\% = 0,00\%$$

Perhitungan rata-rata waktu respons, throughput, dan error rate dilakukan pada seluruh endpoint menggunakan data dasar pada Tabel 5.13. Hasil perhitungan kemudian disajikan pada Tabel 5.14 untuk membandingkan kinerja arsitektur microservices dan monolitik pada beban 1000 pengguna virtual.

Tabel 5.14 Perbandingan hasil pengujian pada 1.000 pengguna virtual

No	Endpoint	Microservices			Monolitik		
		Average (ms)	Throughput (req/s)	Error (%)	Average (ms)	Throughput (req/s)	Error (%)
1	Login	47.813,0	10,30	0,00	48.554,6	10,27	0,00
2	Submit Laporan	7,9	10,38	0,00	3.392,2	10,17	0,10
3	Get Penanganan	903,9	10,38	0,00	344,6	10,20	0,00
4	Master Kekerasan	6,8	10,44	0,00	255,3	10,24	0,00

Berdasarkan Tabel 5.14 microservices mencatat rata-rata waktu respons lebih rendah daripada monolitik pada Login sebesar 1,53%, Submit Laporan sebesar 99,77%, dan Master Kekerasan sebesar 97,36%. Pada Get Penanganan, monolitik memiliki rata-rata waktu respons lebih rendah sebesar 61,88%. *Throughput* kedua arsitektur relatif berdekatan, yaitu 10,30-10,44 req/s pada microservices dan 10,17-10,27 req/s pada monolitik. Microservices menghasilkan *error rate* 0% pada seluruh endpoint. Pada monolitik, satu dari 1.000 permintaan Submit Laporan menerima kode 502 sehingga *error rate* sebesar 0,10%. Log gateway menunjukkan koneksi upstream terputus saat header respons dibaca, sedangkan container monolitik tetap berjalan dan tidak mengalami kehabisan memori.

Secara keseluruhan, peningkatan beban paling memengaruhi Login pada kedua arsitektur. *Microservices* memiliki waktu respons lebih rendah pada Login, Submit Laporan, dan Master Kekerasan, sedangkan monolitik lebih rendah pada Get Penanganan. *Microservices* tidak menghasilkan error hingga 1.000 pengguna virtual; monolitik mengalami satu error 502 pada Submit Laporan di beban 1.000 pengguna virtual tanpa mengalami crash. Perbandingan ini menggunakan batas sumber daya yang sama per container, sehingga total sumber daya kedua arsitektur tidak sama. Setiap skenario juga hanya dijalankan satu kali. Oleh karena itu, hasil

bersifat deskriptif untuk konfigurasi yang diuji dan belum mengisolasi pengaruh arsitektur dari perbedaan total sumber daya maupun variasi antar-run.

5.6. Pengujian *Fault Injection*

Pengujian *fault injection* dilakukan dengan menghentikan sementara report-service sebagai layanan penerima pesan pembaruan status laporan. Pengujian ini bertujuan untuk mengamati penampungan pesan selama layanan penerima mengalami gangguan dan perubahan antrean setelah layanan diaktifkan kembali. Selama skenario berlangsung, *case service*, RabbitMQ, *database*, dan API gateway tetap dijalankan. Dengan demikian, gangguan yang diuji terbatas pada berhentinya layanan penerima.

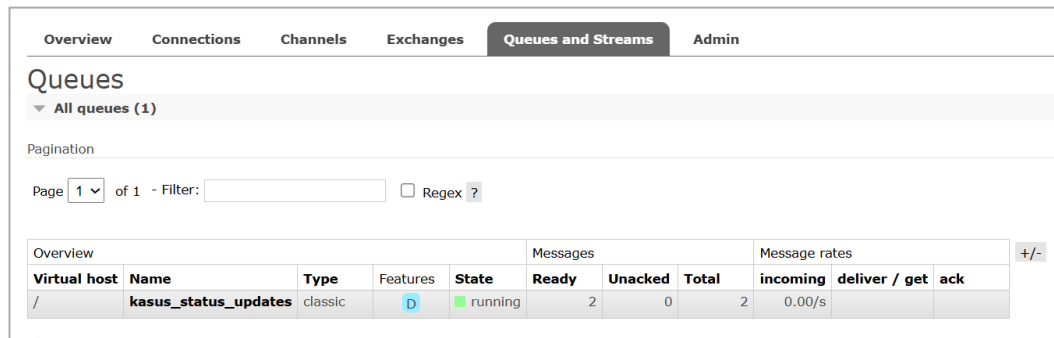
Permintaan registrasi kasus dikirim menggunakan Apache JMeter melalui endpoint POST `/api/penanganan/registrasi`. Pada alur ini, *case-service* menyimpan kasus dan mengirim pesan pembaruan status ke antrean *kasus_status_updates*. Pesan tersebut digunakan oleh *report service* untuk memperbarui status laporan menjadi *proses_assessment*. Login digunakan untuk memperoleh token autentikasi sebelum permintaan registrasi dikirim.

Pelaksanaan pengujian dilakukan dengan menyiapkan laporan yang belum diregistrasi, kemudian menghentikan report-service melalui Docker Desktop. Setelah itu, permintaan registrasi dikirim melalui JMeter. Berdasarkan pengamatan setelah dua pengiriman, antrean memuat dua pesan pada kolom *Ready*. *Report service* kemudian diaktifkan kembali dan kondisi antrean diamati melalui RabbitMQ *Management*.

Tabel 5. 15 Hasil Pengamatan Antrean pada Pengujian *Fault injection*

No	Kondisi layanan penerima	Ready	Unacked	Total
1	Report-service dihentikan, setelah dua pengiriman	2	0	2
2	Report-service diaktifkan kembali	0	0	0

Pada kondisi *report service* dihentikan, nilai *Ready* sebesar 2 menunjukkan adanya dua pesan yang menunggu untuk dikirim kepada konsumen. Nilai *Unacked* sebesar 0 menunjukkan tidak ada pesan yang sedang menunggu *acknowledgment* dari konsumen pada saat pengamatan. Total pesan yang tercatat adalah 2. Kondisi tersebut ditunjukkan pada Gambar 5.13.

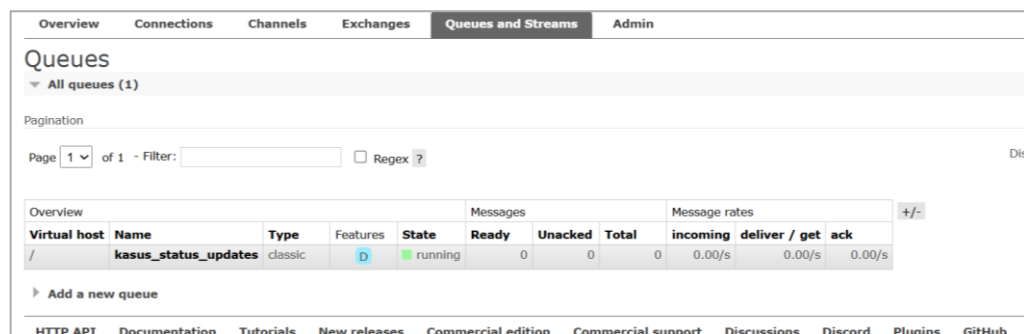


The screenshot shows the RabbitMQ management interface for a queue named 'kasus_status_updates'. The 'Ready' column shows 2 messages, 'Unacked' shows 0, and 'Total' shows 2. The 'Message rates' section shows 0.00/s for incoming, deliver, and get.

Overview					Messages			Message rates				
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver	get	ack	+/-
/	kasus_status_updates	classic	D	running	2	0	2	0.00/s				

Gambar 5. 13 Antrean Pesan ketika *Report Service* Dihentikan

Setelah *report service* diaktifkan kembali, nilai *Ready*, *Unacked*, dan *Total* menjadi 0, sebagaimana ditunjukkan pada Gambar 5.14. Hasil ini menunjukkan bahwa kedua pesan tidak lagi berada dalam antrean setelah layanan penerima dipulihkan.



The screenshot shows the RabbitMQ management interface for the same queue 'kasus_status_updates'. The 'Ready' column now shows 0 messages, 'Unacked' shows 0, and 'Total' shows 0. The 'Message rates' section shows 0.00/s for incoming, deliver, and get.

Overview					Messages			Message rates				
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver	get	ack	+/-
/	kasus_status_updates	classic	D	running	0	0	0	0.00/s	0.00/s	0.00/s		

Gambar 5. 14 Antrean Pesan ketika *Report Service* Diaktifkan Kembali

Berdasarkan pengamatan tersebut, RabbitMQ dapat menampung dua pesan selama layanan penerima dihentikan, kemudian antrean menjadi kosong setelah layanan diaktifkan kembali. Namun, antrean kosong belum cukup untuk membuktikan keberhasilan pembaruan status

5.7. Pengujian *User Acceptance Testing*

Pengujian *User Acceptance Testing* (UAT) dilakukan untuk mengetahui penilaian pengguna terhadap fungsi dan kemudahan penggunaan sistem. Pengujian melibatkan dua kelompok pengguna, yaitu 25 responden yang menilai fitur pelaporan dan 3 responden yang menilai fitur admin. Kuesioner dibedakan berdasarkan fitur yang digunakan oleh masing-masing kelompok. Penilaian menggunakan skala Likert dengan bobot Sangat Setuju (SS) sebesar 5, Setuju (S) sebesar 4, Netral (N) sebesar 3, Tidak Setuju (TS) sebesar 2, dan Sangat Tidak Setuju (STS) sebesar 1. Hasil kedua kelompok dianalisis secara terpisah.

5.7.1 Hasil Pengujian UAT Pelapor

Pengujian pada kelompok pelapor menggunakan tujuh pernyataan mengenai pengalaman menggunakan fitur pelaporan. Dari 25 responden dengan 12 pernyataan yang dikelompokkan dalam empat aspek. Setiap aspek terdiri atas tiga pernyataan. Seluruh jawaban terisi, sehingga terdapat 300 jawaban yang dianalisis. Rincian hasil pengujian disajikan pada Tabel 5.16

Tabel 5.16 Hasil Pengujian *User Acceptance Testing* (UAT) Pelapor

No.	Pernyataan	SS	S	N	TS	STS	Subtotal
A. Fungsionalitas							
1	Proses pengiriman laporan kekerasan (termasuk opsi lapor anonim) berjalan lancar dan sesuai yang saya harapkan.	15	3	3	4	0	104
2	Sebagian formulir terisi otomatis berdasarkan cerita yang saya tulis, sehingga saya tidak perlu mengetik ulang semua informasi	7	8	4	4	2	89
3	Fitur cek status laporan memberikan informasi yang cukup jelas mengenai perkembangan penanganan kasus saya.	8	6	10	1	0	96
B. Kemudahan penggunaan (usability)							
4	Instruksi dan langkah-langkah pengisian formulir pelaporan mudah dipahami tanpa membingungkan.	12	4	6	2	1	99

**Tabel 5.16 Hasil Pengujian *User Acceptance Testing* (UAT) Pelapor
(Lanjutan)**

No.	Pernyataan	SS	S	N	TS	STS	Subtotal
5	Navigasi antar menu dan halaman dalam sistem berjalan dengan lancar dan mudah diikuti oleh pengguna.	9	8	7	1	0	100
6	Sistem menampilkan notifikasi atau konfirmasi yang jelas setiap kali operasi berhasil atau gagal dilakukan.	9	8	7	1	0	100
C. Keandalan sistem.							
7	Sistem dapat saya gunakan selama proses pelaporan tanpa mengalami gangguan yang menghambat.	12	3	7	3	0	99
8	Informasi laporan yang ditampilkan setelah pengiriman sesuai dengan data yang saya masukkan.	10	7	8	0	0	102
9	Fitur cek status laporan dapat diakses kembali ketika saya membutuhkannya.	15	5	4	1	0	109
D. Kelayakan implementasi							
10	Saya merasa lebih yakin melaporkan kekerasan lewat sistem ini dibandingkan cara pelaporan manual/datang langsung.	14	7	3	0	1	108
11	Sistem ini sesuai dengan kebutuhan saya untuk menyampaikan laporan kekerasan.	12	2	7	4	0	97
12	Saya bersedia menggunakan sistem ini apabila perlu menyampaikan laporan kekerasan.	14	7	4	0	0	110
Jumlah		137	68	70	21	4	1.213

Subtotal diperoleh dengan mengalikan jumlah jawaban pada setiap pilihan dengan bobotnya, kemudian menjumlahkan seluruh hasil perkalian. Perhitungan indeks keseluruhan kelompok pelapor adalah sebagai berikut.

$$\text{Jumlah skor} = (137 \times 5) + (68 \times 4) + (70 \times 3) + (21 \times 2) + (4 \times 1) = 1.213$$

$$\text{Skor Maksimum} = \text{Total Pertanyaan} \times \text{Responden} \times \text{Skor Tertinggi}$$

$$\text{Skor Maksimum} = 12 \times 25 \times 5$$

$$\text{Skor Maksimum} = 1.500$$

$$\text{Nilai Total} = \left(\frac{\text{Jumlah skor yang diperoleh}}{\text{Skor maksimum}} \right) \times 100\%$$

$$\text{Nilai Total} = \left(\frac{1213}{1500} \right) \times 100\% = 80,87\%$$

Berdasarkan perhitungan tersebut, diperoleh indeks UAT pelapor sebesar 80,87%. Hasil ini menunjukkan bahwa sistem pelaporan kekerasan terhadap perempuan dan anak yang diuji memperoleh penilaian yang baik dari responden. Artinya, sistem dinilai sudah berjalan dengan baik dari aspek fungsionalitas, kegunaan, keandalan sistem, dan kelayakan sistem.

5.7.2 Hasil Pengujian UAT Admin

Pengujian pada kelompok admin melibatkan tiga responden, yaitu kepala unit dan dua petugas. Analisis menggunakan 14 pernyataan dimana jawaban mengenai pengelolaan akun dan data master digunakan untuk kedua responden berdasarkan klarifikasi bahwa keduanya dapat mengakses fitur tersebut. Rincian hasil pengujian disajikan pada Tabel 5.17

Tabel 5.17 Hasil Pengujian *User Acceptance Testing* (UAT) Admin

No.	Pernyataan	SS	S	N	TS	STS	Subtotal
A. Fungsionalitas							
1	Fitur registrasi laporan oleh petugas berfungsi sesuai dengan alur kerja yang seharusnya.	2	1	0	0	0	14
2	Fitur input hasil assessment dan rencana intervensi tersimpan dengan benar ke dalam data kasus.	1	2	0	0	0	13
3	Fitur pengelolaan akun petugas dan data master (kategori kekerasan, metode penanganan) berfungsi dengan benar.	1	2	0	0	0	13
4	Fitur filter dan ekspor data laporan ke format Excel berfungsi sesuai kebutuhan.	0	3	0	0	0	12

Tabel 5.17 Hasil Pengujian *User Acceptance Testing* (UAT) Admin (Lanjutan)

No.	Pernyataan	SS	S	N	TS	STS	Subtotal
5	Fitur pengelolaan data master (kategori kekerasan dan metode penanganan) pada pusat kendali layanan berjalan dengan benar.	2	1	0	0	0	14
B. Kemudahan penggunaan (usability)							
6	Antarmuka sistem, termasuk navigasi antar menu, mudah dipahami dan digunakan tanpa memerlukan pelatihan khusus.	1	2	0	0	0	13
7	Tampilan data laporan dan kasus tersaji secara terstruktur dan mudah dibaca.	2	0	1	0	0	13
8	Navigasi antar menu dan halaman dalam sistem berjalan dengan lancar dan mudah diikuti oleh pengguna.	2	1	0	0	0	14
9	Sistem menampilkan notifikasi atau konfirmasi yang jelas setiap kali operasi berhasil atau gagal dilakukan.	1	2	0	0	0	13
C. Keandalan sistem							
10	Data yang ditampilkan pada setiap halaman konsisten dan sesuai dengan data yang telah diinputkan ke dalam sistem.	1	2	0	0	0	13
11	Sistem mampu menangani perpindahan status kasus secara otomatis tanpa kesalahan.	1	2	0	0	0	13
D. Kelayakan implementasi							
12	Sistem ini membantu mempercepat dan meningkatkan efektivitas proses penanganan kasus.	1	2	0	0	0	13
13	Alur kerja yang diimplementasikan (pelaporan, registrasi, assessment, intervensi, penyelesaian) sesuai dengan prosedur operasional yang berlaku.	2	1	0	0	0	14
Jumlah		17	21	1	0	0	172

Subtotal diperoleh dengan mengalikan jumlah jawaban pada setiap pilihan dengan bobotnya, kemudian menjumlahkan seluruh hasil perkalian. Perhitungan indeks keseluruhan kelompok pelapor adalah sebagai berikut.

$$\text{Jumlah skor} = (17 \times 5) + (21 \times 4) + (3 \times 3) + (0 \times 2) + (0 \times 1) = 172$$

$$\text{Skor Maksimum} = \text{Total Pertanyaan} \times \text{Responden} \times \text{Skor Tertinggi}$$

$$\text{Skor Maksimum} = 13 \times 3 \times 5$$

$$\text{Skor Maksimum} = 195$$

$$\text{Nilai Total} = \left(\frac{\text{Jumlah skor yang diperoleh}}{\text{Skor maksimum}} \right) \times 100\%$$

$$\text{Nilai Total} = \left(\frac{184}{210} \right) \times 100\% = 88,21\%$$

Berdasarkan perhitungan tersebut, diperoleh indeks UAT admin sebesar 88,21%. Hasil ini menunjukkan bahwa sistem manajemen kasus kekerasan terhadap perempuan dan anak yang diuji memperoleh penilaian yang sangat baik dari responden. Artinya, sistem dinilai sudah berjalan dengan baik dari aspek fungsionalitas, kegunaan, keandalan sistem, dan kelayakan sistem.

BAB VI

PENUTUP

6.1. Kesimpulan

Berdasarkan hasil dan pembahasan yang telah diuraikan dalam penelitian ini, disimpulkan bahwa:

1. Sistem pelaporan dan manajemen kasus kekerasan terhadap perempuan dan anak berbasis web berhasil dikembangkan menggunakan metode *Rational Unified Process* (RUP). Sistem mengintegrasikan pelaporan masyarakat, pemantauan status, registrasi, *assessment*, intervensi, dan penyelesaian kasus sesuai hak akses pengguna.
2. Arsitektur *microservices* diterapkan melalui *report-service* dan *case-service* dengan RabbitMQ sebagai perantara pesan perubahan status. Pengujian *fault injection* menunjukkan bahwa dua pesan tetap tertampung ketika *report-service* dihentikan dan antrean menjadi kosong setelah layanan diaktifkan kembali.
3. Pada *load testing* dengan 100, 500, dan 1.000 pengguna virtual, *microservices* memiliki rata-rata waktu respons lebih rendah pada Login, Submit Laporan, dan Master Kekerasan, sedangkan monolitik lebih cepat pada Get Penanganan. *Microservices* mencatat *error rate* 0%, sementara monolitik mengalami satu kegagalan Submit Laporan pada 1.000 pengguna virtual dengan *error rate* 0,10%. Hasil ini berlaku pada konfigurasi dan skenario yang diuji.
4. Pengujian *User Acceptance Testing* (UAT) memperoleh indeks 80,87% dari 25 responden pelapor dan 88,21% dari tiga responden admin. Berdasarkan interval penilaian yang digunakan, hasil tersebut termasuk kategori Setuju dan Sangat Setuju, yang menunjukkan penerimaan positif terhadap fitur dan penggunaan sistem.

6.2. Saran

Adapun beberapa saran untuk pengembangan lebih lanjut sistem pelaporan dan manajemen kasus kekerasan ini, yaitu:

1. Pengelolaan data master perlu dikembangkan dengan pemilahan pilihan bentuk kekerasan terhadap anak dan perempuan dewasa berdasarkan usia serta jenis kelamin korban. Rincian bentuk kekerasan, isian pelaporan, dan kebutuhan penanganan perlu disesuaikan dengan karakteristik masing-masing kelompok serta divalidasi bersama DPPP Kota Kendari. Jenis kekerasan yang berlaku pada kedua kelompok tetap dapat digunakan, sedangkan penyajian data dan rekapitulasi perlu dibedakan agar kasus anak perempuan tidak tercampur dengan kategori perempuan dewasa.
2. Kinerja Login dan Get Penanganan perlu ditingkatkan melalui evaluasi penggunaan CPU, *query* dan indeks basis data, ukuran respons, serta penerapan *pagination* sesuai kebutuhan. Pengaruh setiap perbaikan perlu diperiksa melalui pengujian ulang agar penurunan waktu respons dapat dibuktikan.
3. Pengujian *fault injection* perlu dilanjutkan dengan pemeriksaan perubahan status pada basis data setelah layanan pulih serta skenario gangguan RabbitMQ dan jaringan. Penanganan pesan perlu dievaluasi untuk mencegah kehilangan pesan dan pemrosesan berulang.
4. Pengujian penerimaan pengguna selanjutnya perlu melibatkan lebih banyak petugas dengan kewenangan yang jelas. Instrumen perlu ditinjau untuk menghilangkan pernyataan terduplikasi serta diuji validitas dan reliabilitasnya. Fitur pengisian formulir otomatis perlu dievaluasi lebih lanjut karena memperoleh indeks terendah pada UAT pelapor, yaitu 71,20%

DAFTAR PUSTAKA

- Achsan, S. A., Susetyo, Y. A. 2022. Restful Web Service Implementation Using Spring Framework In Room Assets Management System. *Jurnal Teknik Informatika*, 3(2), 395–403.
- Adham, M. F. 2024. Analisis Implementasi Sistem Informasi: Studi Literatur. *Jurnal Teknologi Sistem Informasi*, 5(1), 264–275.
- Adi, Terttiavini, Dona Marcelina, D. A. 2023. Sistem Informasi Pelayanan Pengaduan Kekerasan Terhadap Perempuan Dan Anak Berbasis Web. *Jurnal Ilmiah Binary*, 5(2), 174–184.
- Al-Said Ahmad, A., Andras, P. 2022. Scalability Resilience Framework Using Application-Level Fault Injection For Cloud-Based Software Services. *Journal Of Cloud Computing*, 11, 1-13.
- Andriati, W. 2023. Sistem Informasi Pelaporan Realisasi E-Order Berbasis Web Pada Pemerintah Kota Jakarta Timur. *Jurnal Prosisko*, 10(1), 24–31.
- Annisa, R., Adharani, Y. 2023. Sistem Informasi Penanganan Kekerasan Terhadap Perempuan Dan Anak Berbasis Website (Studi Kasus Pimpinan Wilayah 'Aisyiyah DKI Jakarta). *Just It: Jurnal Sistem Informasi, Teknologi Informasi Dan Komputer*, 14(1). 36-45.
- Aprianto, A. N., Girsang, A. S., Nugroho, Y., Putra, W. K. 2024. Analisis Kinerja Streaming RabbitMQ Dan Nats Untuk Komunikasi Di Layanan Mikro. *Teknologi: Jurnal Ilmiah Sistem Informasi*, 14(1), 37–47.
- Arianti, T., Fa'izi, A., Adam, S., Wulandari, M. 2022. Perancangan Sistem Informasi Perpustakaan Menggunakan Diagram UML (Unified Modelling Language). *Jurnal Ilmiah Komputer Terapan Dan Informasi*, 1(1), 19–25.
- Arief, M., Chaniago, A. U. 2025. Penerapan Arsitektur Microservices Pada Web E-Commerce Menggunakan Metode GRPC. *Jurnal Networking System And Security System*, 2(1), 1–9.
- Balahadia, F., Astoveza, Z. J., Jamolin, G., Astoveza, N. E. A. 2022. Development And Implementation Of Violence Against Women And Their Children

- Report System Mobile Application. *International Journal Of Science, Technology, Engineering And Mathematics*, 2(3), 17–42.
- Bentaleb, O., Belloum, A. S. Z., Sebaa, A., El Maouhab, A. 2022. Containerization Technologies: Taxonomies, Applications And Challenges. *The Journal Of Supercomputing*, 78(1), 1144–1181.
- Blinowski, G., Ojdowska, A., Przybylek, A. 2022. Monolithic Vs. Microservice Architecture: A Performance And Scalability Evaluation. *IEEE Access*, 10, 20357–20374.
- Cahyono, S. A. B., Sucipto, Firliana, R. 2023. Implementasi Otentikasi Website Node JS Express Menggunakan Passport. *Jurnal Sistem Informasi dan Teknologi Informasi Komputer*, 2(1), 33–40.
- Calderón-Gómez, H., Mendoza-Pittí, L., Vargas-Lombardo, M., Gómez-Pulido, J. M., Rodríguez-Puyol, D., Sención, G., Polo-Luque, M. L. 2021. Evaluating Service-Oriented And Microservice Architecture Patterns To Deploy eHealth Applications In Cloud Computing Environment. *Applied Sciences*, 11(10), 1-28.
- Ćatović, A., Buzadžija, N., Lemeš, S. 2022. Microservice Development Using RabbitMQ Message Broker. *Science, Engineering And Technology*, 2(1), 30–37.
- Dinova, C. A., Utomo, I. C. 2023. Pengembangan Arsitektur Microservice Pada Learning Management System E-Learning Menggunakan Metode Web Service. *Jurnal Ilmu Komputer Dan Informatika*, 3(2), 125–141.
- Dirgantara, D. P., Kusumo, D. S., Utomo, R. G. 2024. Docker-Based Monolithic And Microservices Architecture Performance Comparison. *Jurnal Teknik Informatika*, 5(2), 357–365.
- Effendi, E., Sagalai, R. Si. A., Rezeki, S. 2023. Jenis-Jenis Sistem Informasi Dan Model Sistem Informasi. *Jurnal Pendidikan Dan Konseling*, 5(2), 4944–4952.
- Farchani, S. B., Hermanto, N., Kusuma, B. A. 2025. Implementasi Rest API Dalam Pengembangan Backend Inventory Peminjaman. *Jurnal Ilmiah Penelitian Dan Pembelajaran Informatika*, 10(2), 1404–1413.

- Fitriana, A. R. D., Kriswibowo, A. 2022. Case Management Approach To Overcoming The Violence Against Child (Study At The Regional Technical Implementation Unit For The Women And Children Protection, Sidoarjo Regency). *Journal Of Economics, Business, And Government Challenges*, 5(1), 66–74.
- Galahartlambang, Y., Khotiah, T., Jumain. 2021. Analisa Performa Aplikasi Web Berbasis Manipulasi Dom Dan Virtual Dom. *Seminar Nasional Inovasi Teknologi*, 36–40.
- Gea, I. Z., Mardhiyah, L., Jasri, R. H. 2025. Faktor Sosial-Budaya, Dampak, Dan Upaya Pencegahan Kekerasan Seksual Terhadap Anak Di Indonesia. *Jurnal Abdimas Indonesia*, 5(4). 2386–2390.
- Ginasari, N. L. A. S., Wibawa, K. S., Wirdiani, N. K. A. 2021. Pengujian Stress Testing Api Sistem Pelayanan Dengan Apache Jmeter. *Jurnal Ilmiah Teknologi Dan Komputer*, 2(3), 552–557.
- Gudelli, V. R. 2023. Containerization Technologies: ECR And Docker For Microservices Architecture. *International Journal Of Innovative Research In Engineering & Multidisciplinary Physical Sciences*, 11(3), 1–10.
- Hanapi, A., Moonti, R. M., Ahmad, I. 2025. Kekerasan Berbasis Gender Terhadap Anak Dan Perempuan: Antara Stigma Sosial Dan Ketimpangan Perlindungan Hukum. *Jurnal Hukum, Pendidikan Dan Sosial Humaniora*, 2(4), 143–156.
- Ramli, H., Santi, I. H., Romadhona, R. D. 2026. Optimalisasi Sistem Informasi Layanan Jasa Konsultan Dengan Metode Search Engine Optimization (SEO). *Jurnal Informatika Dan Teknologi Komputer*, 6(1), 249–266.
- Saputra, M. H. K., Nabil, L. 2021. Penerapan Arsitektur Microservice Pada Sistem Tata Kelola Matakuliah Proyek Politeknik Pos Indonesia. *Jurnal Teknik Informatika*, 13(3), 22–28.
- Hidayat, A. 2021. Kekerasan Terhadap Anak Dan Perempuan. AL-MURABBI: *Jurnal Studi Kependidikan Dan Keislaman*, 8(1), 22–33.
- Huh, I., Gim, J. 2025. Exploration Of Likert Scale In Terms Of Continuous Variable With Parametric Statistical Methods. *BMC Medical Research Methodology*, 25(1), 1-10.

- Iqbal, M., Syahputra, A. K., Handoko, W. 2022. Penerapan Monolithic Architecture Pada Aplikasi Ujian Online Berbasis Web. *Jurnal Pemberdayaan Sosial Dan Teknologi Masyarakat*, 2(2), 213–218.
- Irawan C, Y., Ruri I, D., Riastuti, M. 2024. Penerapan Model Rational Unified Process (RUP) Dalam Membangun Aplikasi Promosi Dan Penjualan Berbasis Web (Studi Kasus: Toko Mie Ayam "Neng Kene Kiyeh"). *Ikraith-Informatika*, 8(3), 107–120.
- Jannah, M., Nasrullah, Rachmad. 2024. Implementasi Kebijakan Penanganan Tindak Kekerasan Perempuan Di Dinas Pemberdayaan Perempuan Dan Perlindungan Anak Kota Palu. *Journal Of Publicness Studies*, 1(2), 48–55.
- Kartaputra, D. P., Sulistiawati, N. 2024. Pelaporan Dan Pencegahan Kekerasan Seksual Berbasis Web. *Jurnal Bangkit Indonesia*, 13(2), 58–64.
- Maharani, A. S., Alifa, C. F., Febriga, A. L. 2022. Perancangan Data Base Kasir dan Persediaan Barang Menggunakan MongoDB. *Jurnal Data Mining dan Sistem Informasi*, 3(1), 31–37.
- Malavolta, I., Nirghin, K., Scoccia, G. L., Romano, S., Lombardi, S., Scanniello, G., Lago, P. 2023. JavaScript Dead Code Identification, Elimination, And Empirical Assessment. *IEEE Transactions On Software Engineering*, 49(7), 3692–3714.
- Neelan, A. 2025. The Evolving Role Of API Gateways In Scalable Microservices Architecture. *International Journal Of Emerging Trends In Computer Science And Information Technology*, 6(4), 4–15.
- Novianto, M. A., Munir, S. 2022. Analisis Dan Implementasi Restful Api Guna Pengembangan Sistem Informasi Akademik Pada Perguruan Tinggi. *Jurnal Informatika Terpadu*, 8(1), 47–61.
- Nurrahman, A., Dimas, M., Falakhuddin Ma'sum, M., Farhan Ino, M. 2021. Pemanfaatan Website Sebagai Bentuk Digitalisasi Pelayanan Publik Di Kabupaten Garut. *Jurnal Teknologi Dan Komunikasi Pemerintahan*, 3(1), 78–93.

- Pradigi, C. H., Harlina, T., Solehatin. 2022. Implementasi Express JS Untuk Membangun REST API Website STIKOM PGRI Banyuwangi: *Jurnal Informatika dan Komputer*, 9(2), 118–122.
- Pragitha Rahmawati, C., Hertati, D. 2023. Collaborative Governance Dalam Penanganan Kekerasan Seksual Pada Anak Di Kota Surabaya. *Jurnal Noken: Ilmu-Ilmu Sosial*, 9(1), 1–10.
- Prasetyo, S. E., Wijaya, A. 2021. Analisa Dan Implementasi Microservice Pada Container Menggunakan Docker. *Conference On Management, Business, Innovation, Education And Social Sciences*, 1(1), 557–564.
- Prayogi, A. A., Niswar, M., Indrabayu, Rijal, M. 2020. Design And Implementation Of REST API For Academic Information System. *IOP Conference Series: Materials Science And Engineering*, 875(1), 1-6.
- Putri, D., Fahmi, S., Ardiansah. 2025. Perlindungan Hak Anak Perempuan Kekerasan Seksual Di Kota Pekanbaru Berdasarkan Undang-Undang Nomor 12 Tahun 2022 Tentang Tindak Pidana Kekerasan Seksual. *Sakola: Journal Of Sains Cooperative Learning And Law*, 2(1), 17–29.
- Rahmawati, D., Setiawan, A. 2025. Implementasi Sistem Informasi Pengaduan Masyarakat Berbasis Website: Studi Kasus Balai Desa Pedawang Kudus. *Bima Abdi: Jurnal Pengabdian Masyarakat*, 5(1), 236–247.
- Putra, R. K., Nurjaman, R. A., Haniifah, A. F. 2025. Perancangan Dan Evaluasi Arsitektur Microservices Menggunakan Microservices Migration Pattern Dan Microservices Scorecard. *Jurnal Aplikasi Teknologi Informasi Dan Manajemen*, 6(2), 70–83.
- Putri. A. A., Ritonga. F. U. 2024. Proses Penanganan Kasus Kekerasan Seksual Pada Anak Berkebutuhan Khusus Di Unit Pelaksana Teknis Daerah Perlindungan Perempuan Dan Anak (UPTD PPA) Kota Medan. *Sosmaniora: Jurnal Ilmu Sosial Dan Humaniora*, 3(1), 15–30.
- Ramdany, S. W., Kaidar, S. A., Aguchino, B., Amelia, C., Putri, A. 2024. Penerapan UML Class Diagram Dalam Perancangan Sistem Informasi Perpustakaan Berbasis Web. *Journal Of Industrial And Engineering System*, 5(1), 30–41.

- Rigotti, C., McGlynn, C. 2025. Online And Technology-Facilitated Violence Against Women: The EDVAW Platform's Contribution To Human Rights Protection And Monitoring. *Netherlands Quarterly Of Human Rights*, 43(2), 82–103.
- Riyanto, D. J., Safaat, N. H., Affandes, M. 2022. Implementasi Service Choreography Pattern Arsitektur Microservice Classroom Akademik Menggunakan Docker. *Jurnal Ilmiah Penelitian Dan Pembelajaran Informatika*, 7(3), 768–779.
- Rumondor, A. Y., Wensen, V. A., Budiman, M. J., Kapoh, H. 2025. Case Reporting System With Tracking And Chat Machine Learning For The Department Of Women's Empowerment And Child Protection Manado. *Journal La Multiapp*, 6(3), 496–507.
- Santoso, M. F. 2021. Teknik Single Page Application (SPA) Layout Web dengan Menggunakan React JS dan Bootstrap. *Jurnal Khatulistiwa Informatika*, 9(2), 107–114.
- Setiawan, H., Enda, D. 2025. Pengujian Load testing Website Jurusan Teknik Informatika Politeknik Negeri Bengkalis. *Jurnal Teknik Informatika*, 5(1), 1–12.
- Sinaga, T. L., Charibaldi, N., Cahyana, N. H. 2023. Perbandingan Waktu Respon Aplikasi Database NoSQL Elasticsearch dan MongoDB pada Pengujian Operasi CRUD. *Jurnal Informatika Sunan Kalijaga*, 8(1), 22–35.
- Tejaya, W., Rahman, S., Munir, A. 2023. Pengujian Website Invitees Menggunakan Metode Load testing Dengan Apache Jmeter. *Jurnal Kharisma Tech*, 18(1), 99–112.
- Thabibi, H., Wati, S. F. A., Rinjeni, T. P. 2025. Implementasi *User Acceptance Testing* (UAT) Pada Website E-Commerce UMKM BBhealthy. *Adopsi Teknologi Dan Sistem Informasi*, 4(1), 19–26.
- Tjahyono, A., Muslim, A., Anggraini, D. 2022. Perancangan Sistem Informasi Laporan Kegiatan Penanaman Modal Dengan Menggunakan Arsitektur Microservices Pada Kementerian Investasi/Badan Koordinasi Penanaman Modal. *Indonesia Journal Of Computing*, 7(3), 33–51.

- Triana, Y., Fadhilah, A., Efendi, A., Hanoselina, Y. 2025. Analisis Bentuk-Bentuk Sistem Informasi Manajemen Dalam Mendukung Kinerja Di Dinas Pemberdayaan Perempuan, Perlindungan Anak, Pengendalian Penduduk, Dan Keluarga Berencana (DP3AP2KB) Provinsi Sumatera Barat. *Jurnal Intelek Insan Cendikia*, 2(5), 7977–7990.
- Yulius, Susetyo, Y. A. 2023. Analisis dan Penerapan Database MongoDB pada Aplikasi Manajemen Dokumen di PT. XYZ. *Jurnal Teknologi Informasi dan Komunikas*, 7(4), 570–578.
- Wati, R., Andriyani, N., Priyono, P., Susilowati, T. 2025. Rancang Bangun Sistem Informasi Akademik Berbasis Microservices. *Riggs: Journal Of Artificial Intelligence And Digital Business*, 4(4), 4906–4912.
- Wijaningsih, D., Farida, E. 2022. Violence Against Women And Children As A Violation Of Human Rights. *International Journal Of Multicultural And Multireligious Understanding*, 9(11), 339–345.
- Wu, H., Yu, S., Niu, X., Nie, C., Pei, Y., He, Q., Yang, Y. 2023. Enhancing Fault Injection Testing Of Service Systems Via Fault-Tolerance Bottleneck. *IEEE Transactions On Software Engineering*, 49(8), 4097–4114.

LAMPIRAN

Lampiran 1 *Form User Acceptance Testing*

bit.ly/KusinoerUatPelaporan

Kuesioner Evaluasi Pengguna (User Acceptance Testing) – Modul Penanganan Kasus SIPEKA Kota Kendari (Petugas/Admin UPTD PPA)

Kuesioner ini bertujuan mengevaluasi pengalaman Bapak/Ibu menggunakan modul penanganan kasus pada sistem SIPEKA (Sistem Pelaporan dan Penanganan Kasus Kekerasan terhadap Perempuan dan Anak) Kota Kendari, sebagai bagian dari penelitian skripsi. Mohon isi berdasarkan pengalaman Bapak/Ibu menggunakan sistem dalam pekerjaan sehari-hari. Pengisian bersifat sukarela, data dijaga kerahasiaannya, dan hanya digunakan untuk kepentingan akademik. Estimasi waktu pengisian: 10 menit.

cikoh536@gmail.com [Ganti akun](#)

Tidak dibagikan

* Menunjukkan pertanyaan yang wajib diisi

Nama *

Jawaban Anda

Usia *

Jawaban Anda

Jenis Kelamin *

☐ Laki laki

☐ Perempuan

Pekerjaan

Jawaban Anda

Saya bersedia mengisi kuesioner ini secara sukarela untuk keperluan penelitian skripsi, dan memahami bahwa data yang saya berikan akan dijaga kerahasiaannya serta hanya digunakan untuk kepentingan akademik *

☐ Centang untuk lanjut

Penilaian Pengguna						
Proses pengiriman laporan kekerasan (termasuk opsi lapor anonim) berjalan lancar dan sesuai yang saya harapkan. *						
	1	2	3	4	5	
Sangat Tidak Setuju	☐	☐	☐	☐	☐	Sangat Setuju
Instruksi dan langkah-langkah pengisian formulir pelaporan mudah dipahami tanpa membingungkan. *						
	1	2	3	4	5	
Sangat Tidak Setuju	☐	☐	☐	☐	☐	Sangat Setuju
Sebagian formulir terisi otomatis berdasarkan cerita yang saya tulis, sehingga saya tidak perlu mengetik ulang semua informasi *						
	1	2	3	4	5	
Sangat Tidak Setuju	☐	☐	☐	☐	☐	Sangat Setuju
Sistem dapat saya gunakan selama proses pelaporan tanpa mengalami gangguan yang menghambat. *						
	1	2	3	4	5	
Sangat Tidak Setuju	☐	☐	☐	☐	☐	Sangat Setuju
Informasi laporan yang ditampilkan setelah pengiriman sesuai dengan data yang saya masukkan.						
	1	2	3	4	5	
	☐	☐	☐	☐	☐	

bit.ly/KusionerUatPetugas

Kuesioner Evaluasi Pengguna (User Acceptance Testing) – Modul Penanganan Kasus SIPEKA Kota Kendari (Petugas/Admin UPTD PPA)

Kuesioner ini bertujuan mengevaluasi pengalaman Bapak/Ibu menggunakan modul penanganan kasus pada sistem SIPEKA (Sistem Pelaporan dan Penanganan Kasus Kekerasan terhadap Perempuan dan Anak) Kota Kendari, sebagai bagian dari penelitian skripsi. Mohon isi berdasarkan pengalaman Bapak/Ibu menggunakan sistem dalam pekerjaan sehari-hari. Pengisian bersifat sukarela, data dijaga kerahasiaannya, dan hanya digunakan untuk kepentingan akademik. Estimasi waktu pengisian: 10 menit.

cikoh536@gmail.com [Ganti akun](#)



Tidak dibagikan

* Menunjukkan pertanyaan yang wajib diisi

Jabatan/peran di UPTD PPA *

Jawaban Anda

Apakah Anda memiliki akses sebagai Super Admin (pengelolaan akun & data master)? *

- ☐ Ya
- ☐ Tidak

Fitur registrasi laporan oleh petugas berfungsi sesuai dengan alur kerja yang seharusnya. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Fitur input hasil assessment dan rencana intervensi tersimpan dengan benar ke dalam data kasus. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Fitur input hasil assessment dan rencana intervensi tersimpan dengan benar ke dalam data kasus. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Fitur pengelolaan akun petugas dan data master (kategori kekerasan, metode penanganan) berfungsi dengan benar. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Fitur pengelolaan data master (kategori kekerasan dan metode penanganan) pada pusat kendali layanan berjalan dengan benar. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Fitur filter dan ekspor data laporan ke format Excel berfungsi sesuai kebutuhan. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Antarmuka sistem, termasuk navigasi antar menu, mudah dipahami dan digunakan tanpa memerlukan pelatihan khusus. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Navigasi antar menu dan halaman dalam sistem berjalan dengan lancar dan mudah diikuti oleh pengguna. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Data yang ditampilkan pada setiap halaman konsisten dan sesuai dengan data yang telah diinputkan ke dalam sistem. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Tampilan data laporan dan kasus tersaji secara terstruktur dan mudah dibaca. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Sistem menampilkan notifikasi atau konfirmasi yang jelas setiap kali operasi berhasil atau gagal dilakukan. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Sistem mampu menangani perpindahan status kasus secara otomatis tanpa kesalahan. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Sistem ini membantu mempercepat dan meningkatkan efektivitas proses penanganan kasus. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Alur kerja yang diimplementasikan (pelaporan, registrasi, assessment, intervensi, penyelesaian) sesuai dengan prosedur operasional yang berlaku. *

1 2 3 4 5

Sangat Tidak Setuju ☐ ☐ ☐ ☐ ☐ Sangat Setuju

Lampiran 2 Hasil Penilaian UAT Responden

Timestamp	Nama													
25/08/2026 20:34:28	Ratu Humaira	4	5	4	3	4	4	4	4	3	4	3	4	
31/08/2026 10:06:19	ozora annisa putri	5	5	5	5	5	5	5	5	5	5	4	5	
31/08/2026 10:06:37	Sari Puspita	2	3	5	5	5	5	5	5	5	5	4	5	
31/08/2026 10:08:35	Diana Evelink Wong	2	1	3	4	2	4	5	4	4	3	5	5	
31/08/2026 10:09:23	Atapu mutiara	2	3	2	3	3	2	3	2	3	5	3	3	
31/08/2026 10:09:24	ERLIN	3	3	3	3	3	3	3	3	3	3	3	3	
31/08/2026 10:09:35	Amri rahardi	3	2	2	3	3	3	4	3	3	3	2	4	
31/08/2026 10:09:38	Safa Salsabila Amir	2	3	2	2	3	4	5	4	2	4	3	5	
31/08/2026 10:10:09	ADNAN	3	3	4	2	3	3	4	3	2	3	5	4	
31/08/2026 10:11:07	dhea putri arafah	5	5	4	3	3	4	4	4	3	4	5	4	
31/08/2026 10:11:12	Nabila safitri	5	5	4	3	3	4	4	4	3	4	5	4	
31/08/2026 10:11:19	Asyer Rapli	5	3	3	2	4	4	5	4	2	4	5	5	
31/08/2026 10:11:59	raihan	5	5	5	5	5	5	5	5	5	5	4	5	
31/08/2026 10:12:15	Windy Agustin	4	5	4	5	4	3	5	3	5	3	5	5	
31/08/2026 10:12:22	JULIA KHAULAH AZHAF	5	2	4	4	3	3	1	3	2	3	5	3	
31/08/2026 10:12:24	Muhammad Rezky Amal	5	5	5	5	5	5	5	5	5	5	5	5	
Jabatan/peran di UPTD PPA														
Kepala UPTD PPA	5	5	4	4	4	5	4	3	4	5	5	4	4	5
Petugas	4	4	4	5	4	4	5	5	5	4	4	5	5	4
Petugas	5	4	4	4	4	4	4	5	4	5	5	5	4	4

Lampiran 3 Wawancara Dengan Petugas DPPPA



Lampiran 4 Dokumentasi Responden UAT



Lampiran 5 Bimbingan Dosen Pembimbing 2